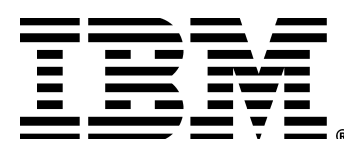


Containers & OpenShift Workshop

Alexander Mamani - IBM
Kate Queiroz - Red Hat
Christian Zaccaria - Red Hat
Winnie Imafidon - IBM

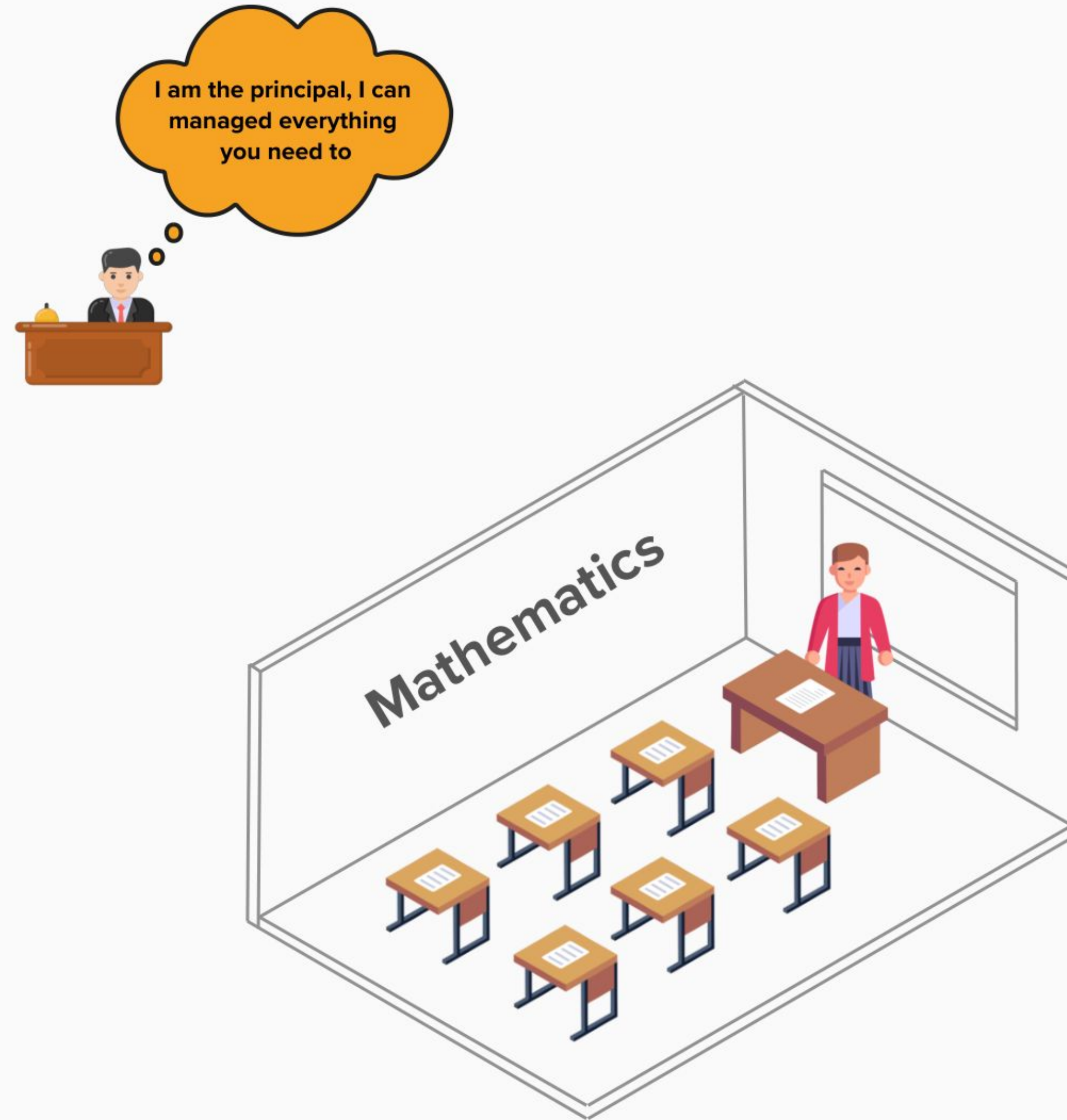


What is kubernetes?

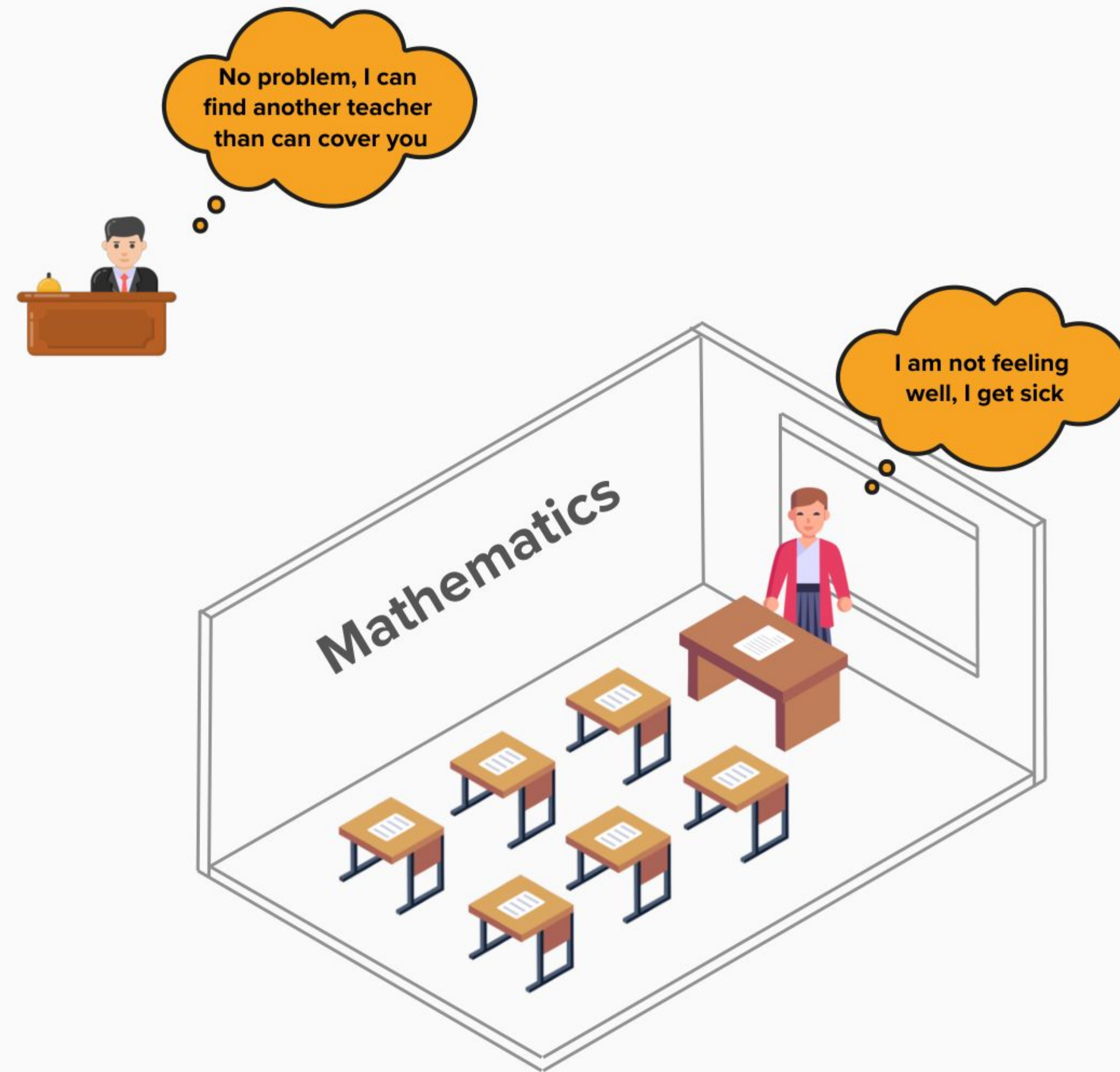
Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.

Let's compare it with a real-life example

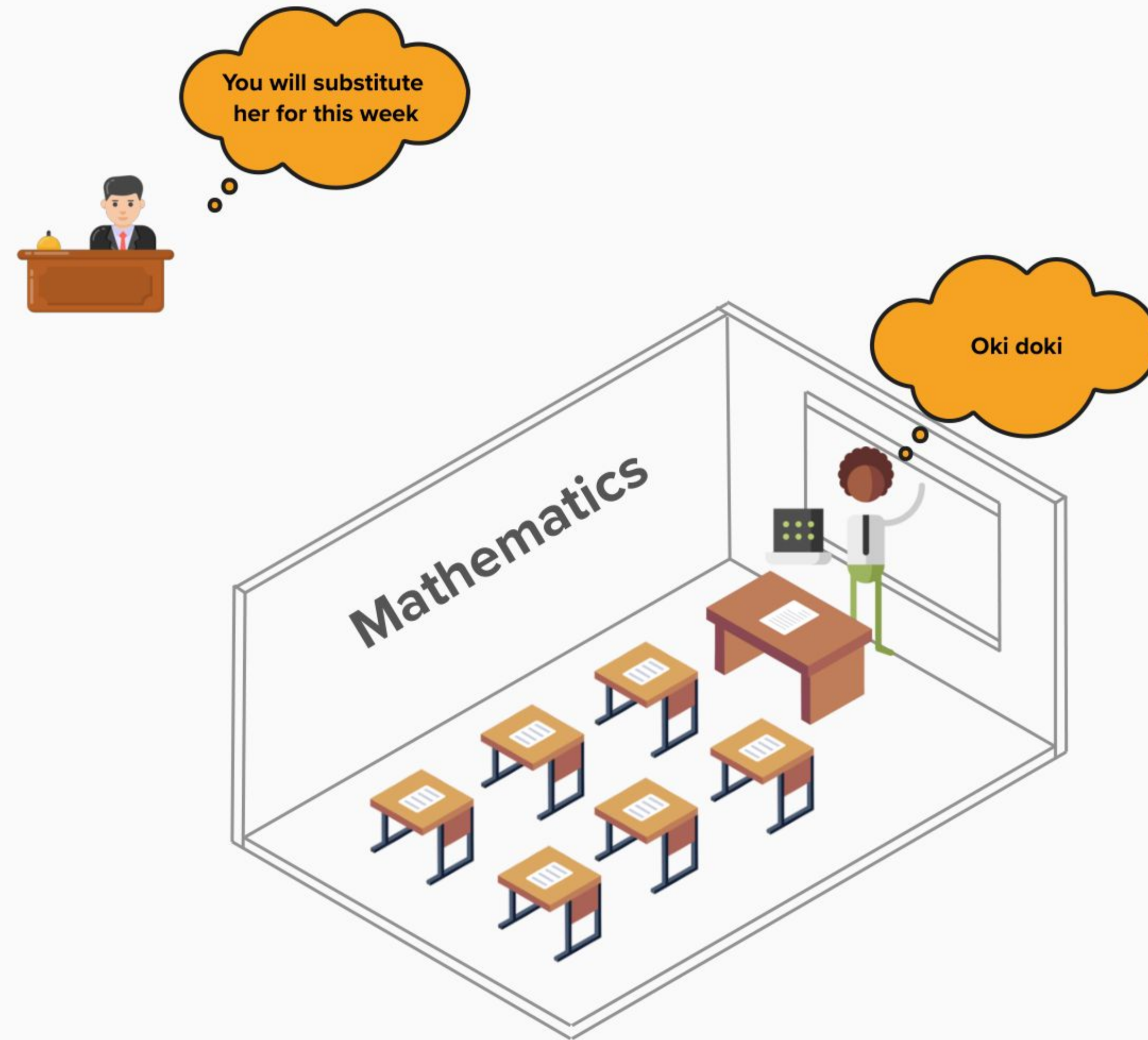
1. KUBERNETES



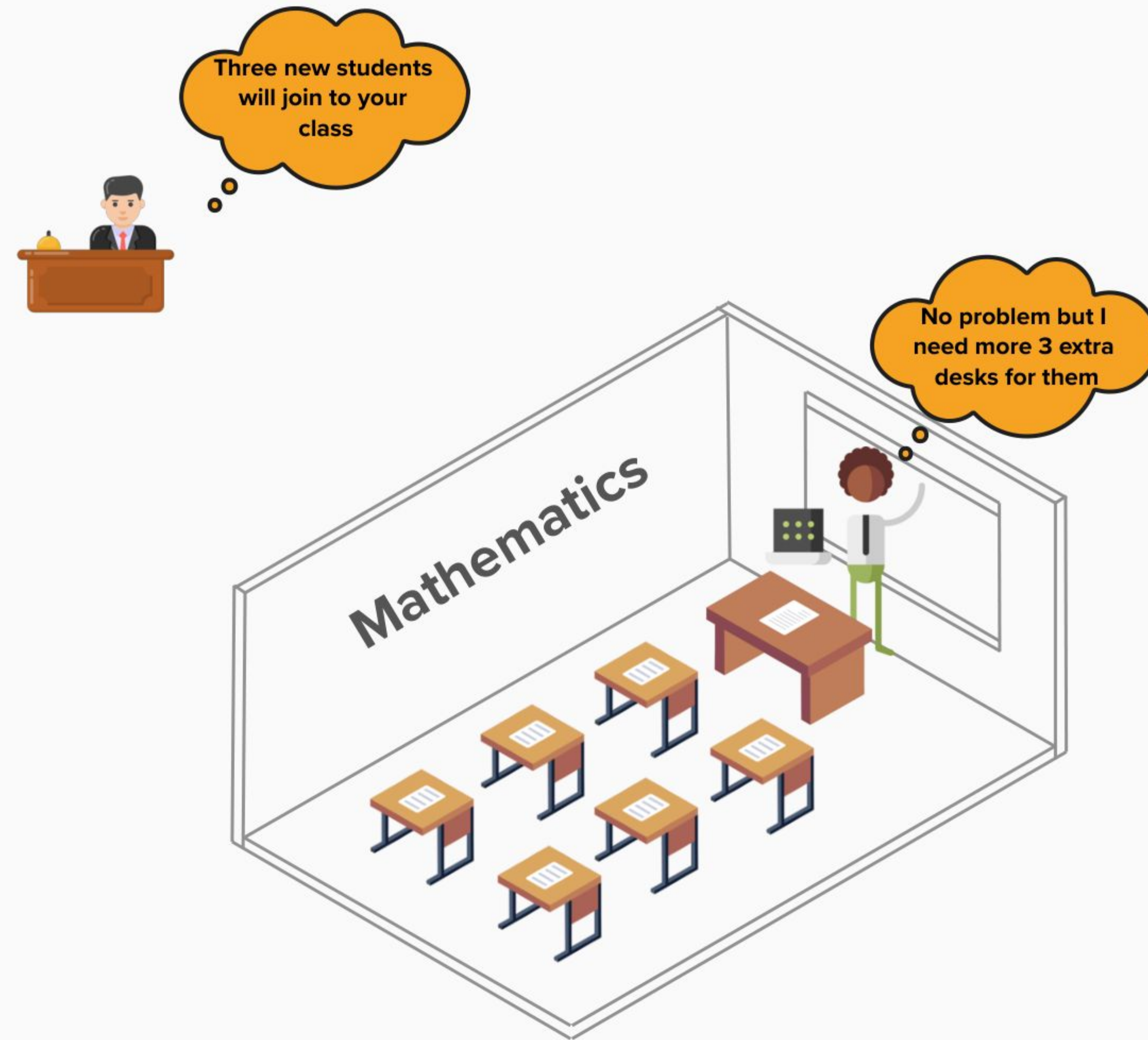
1. KUBERNETES



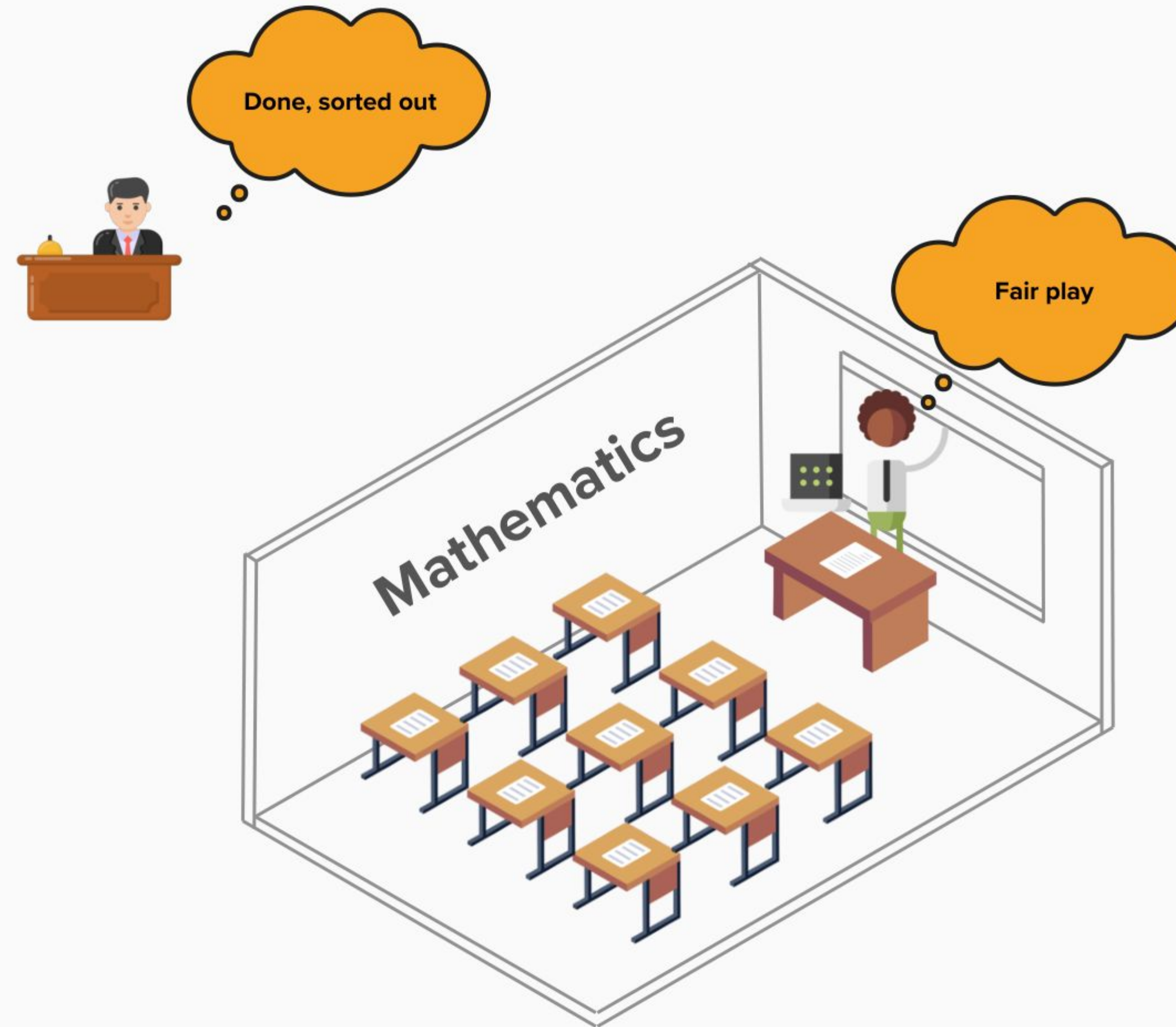
1. KUBERNETES



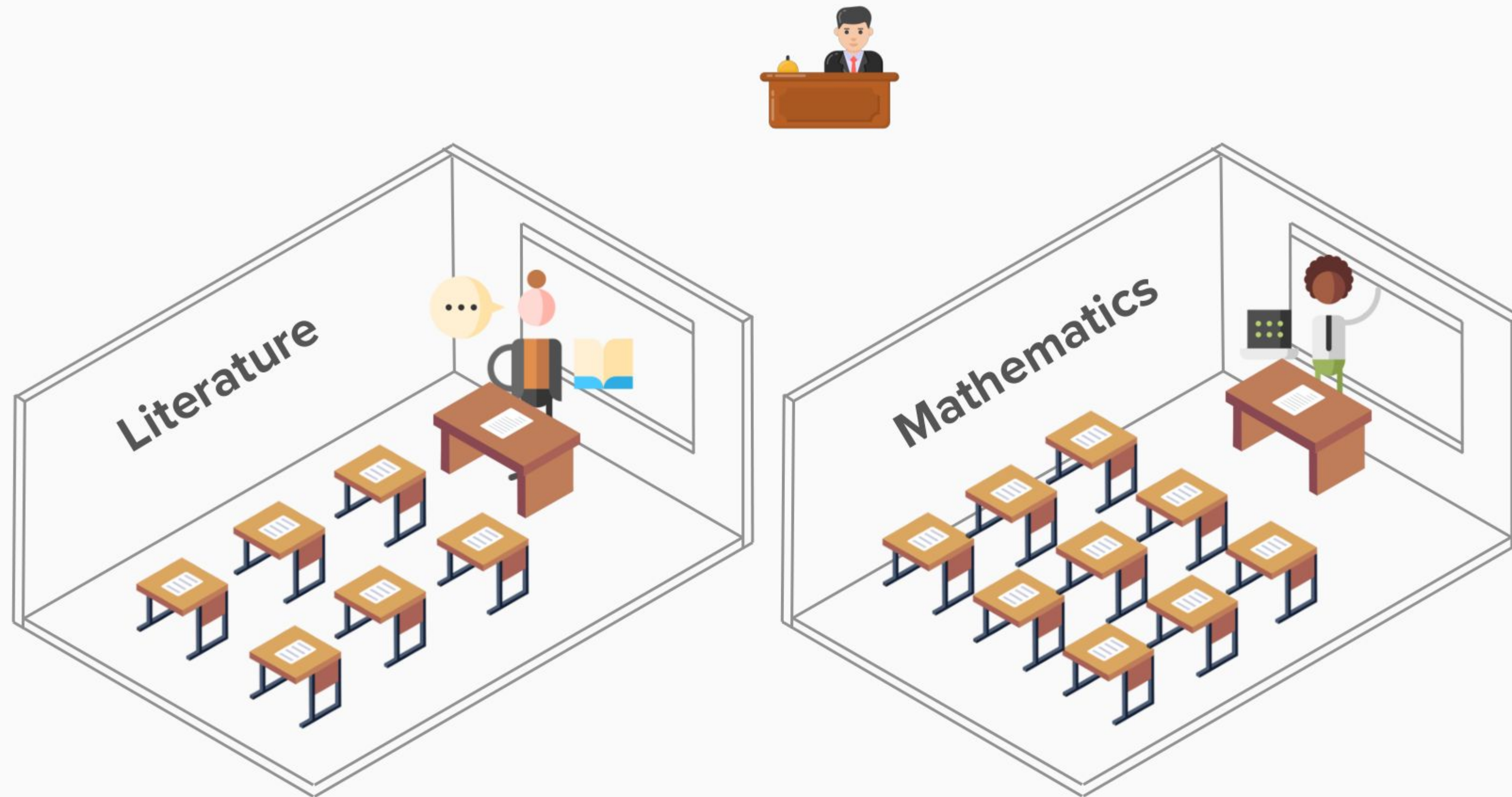
1. KUBERNETES



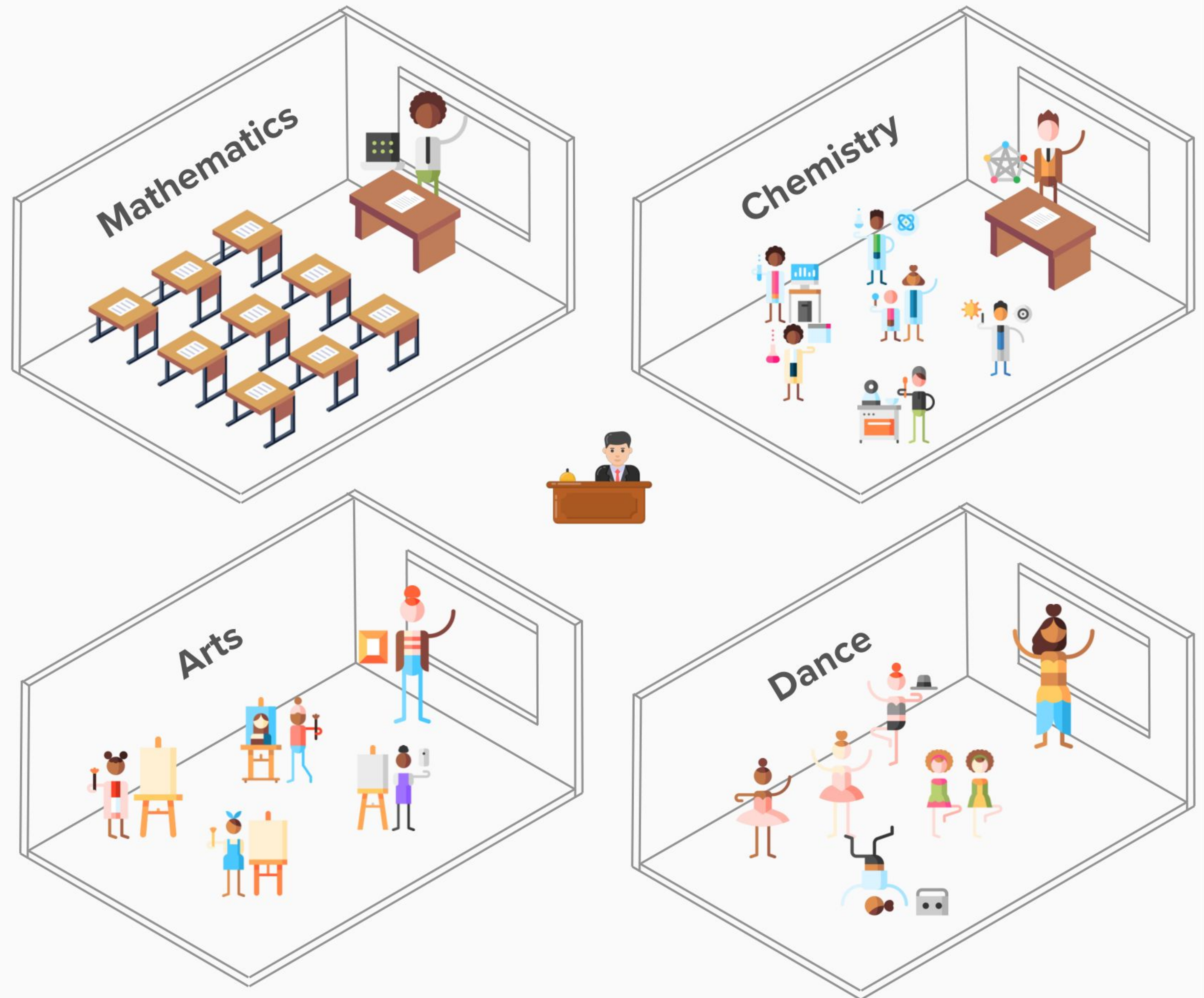
1. KUBERNETES



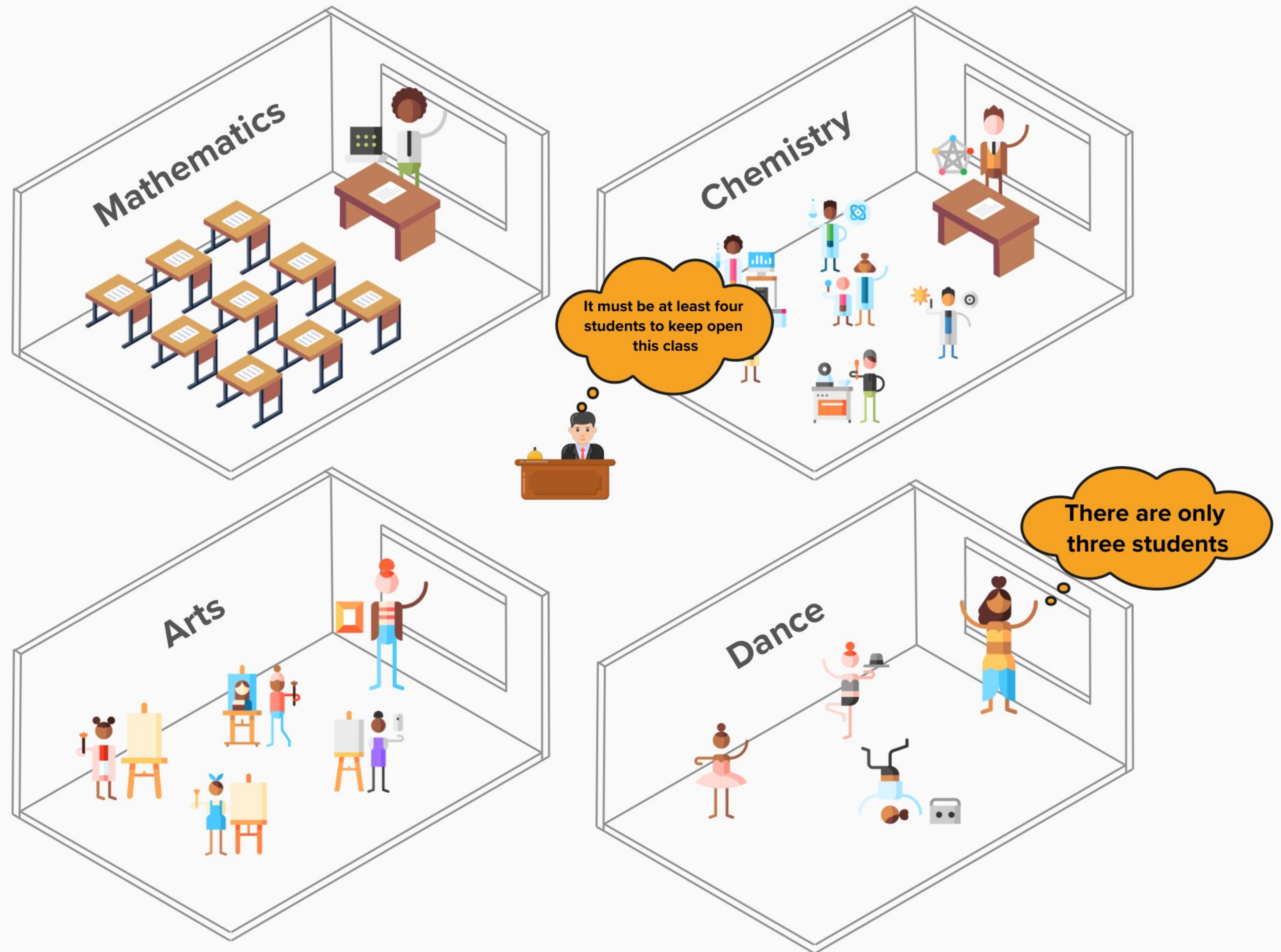
1. KUBERNETES



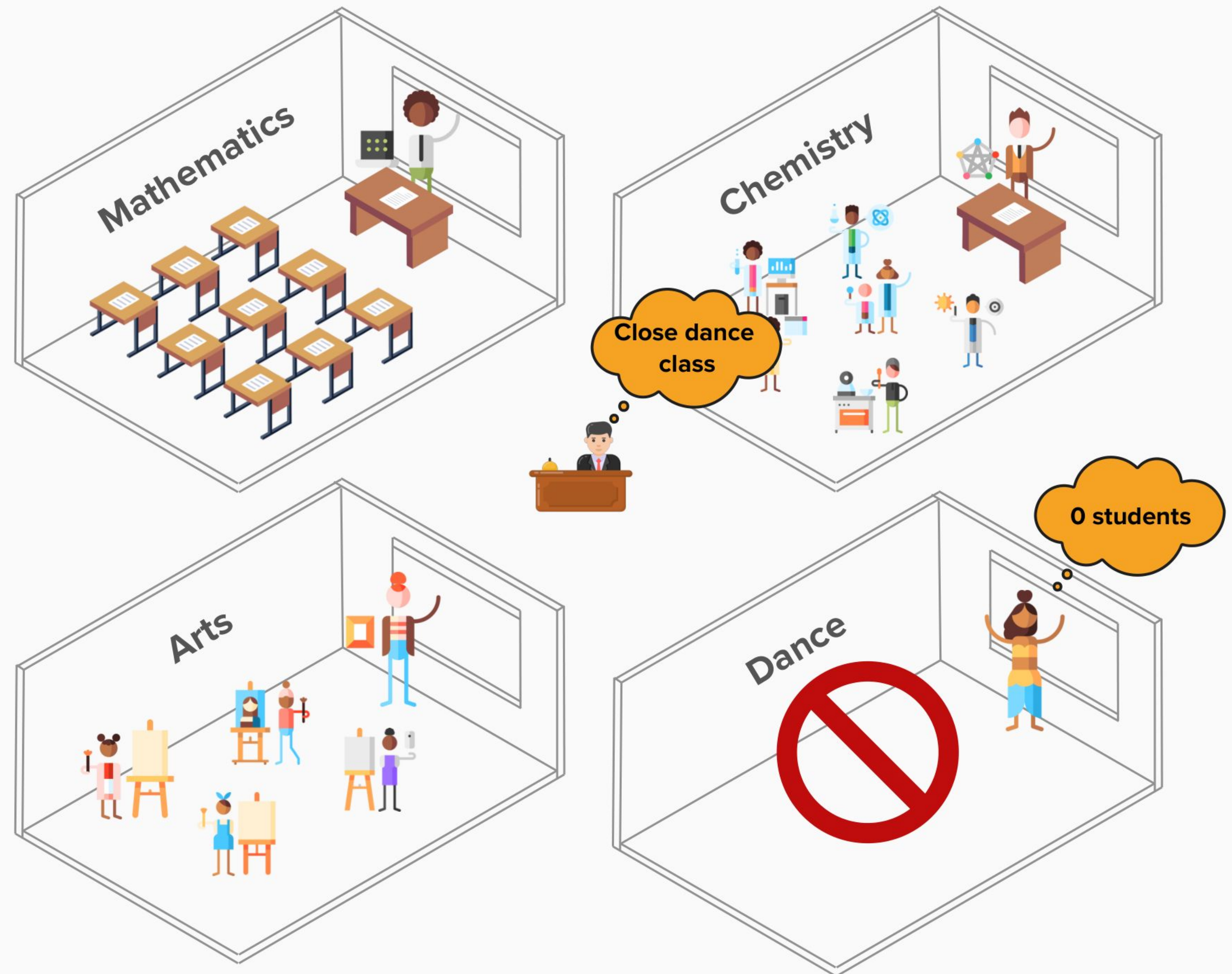
1. KUBERNETES



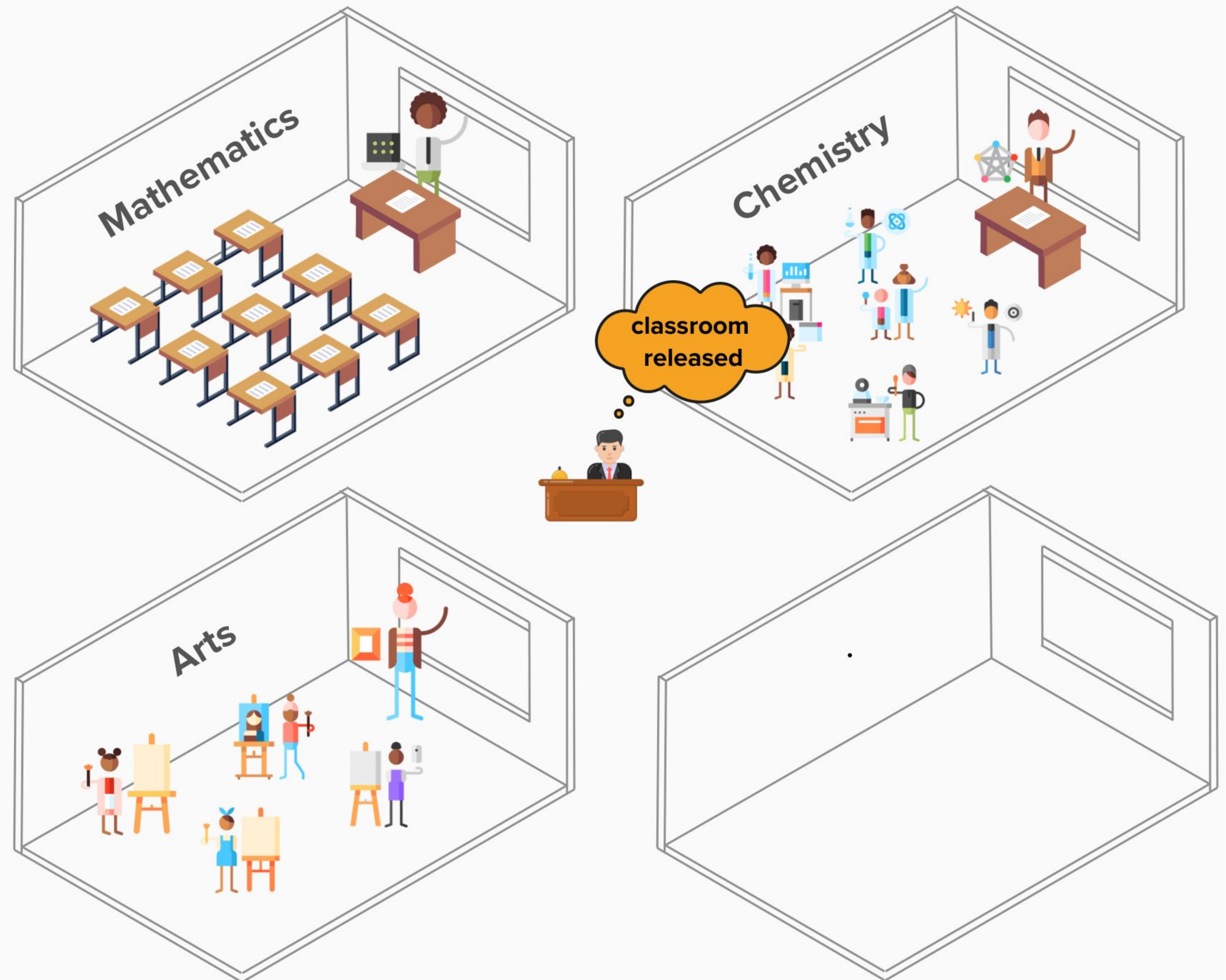
1. KUBERNETES



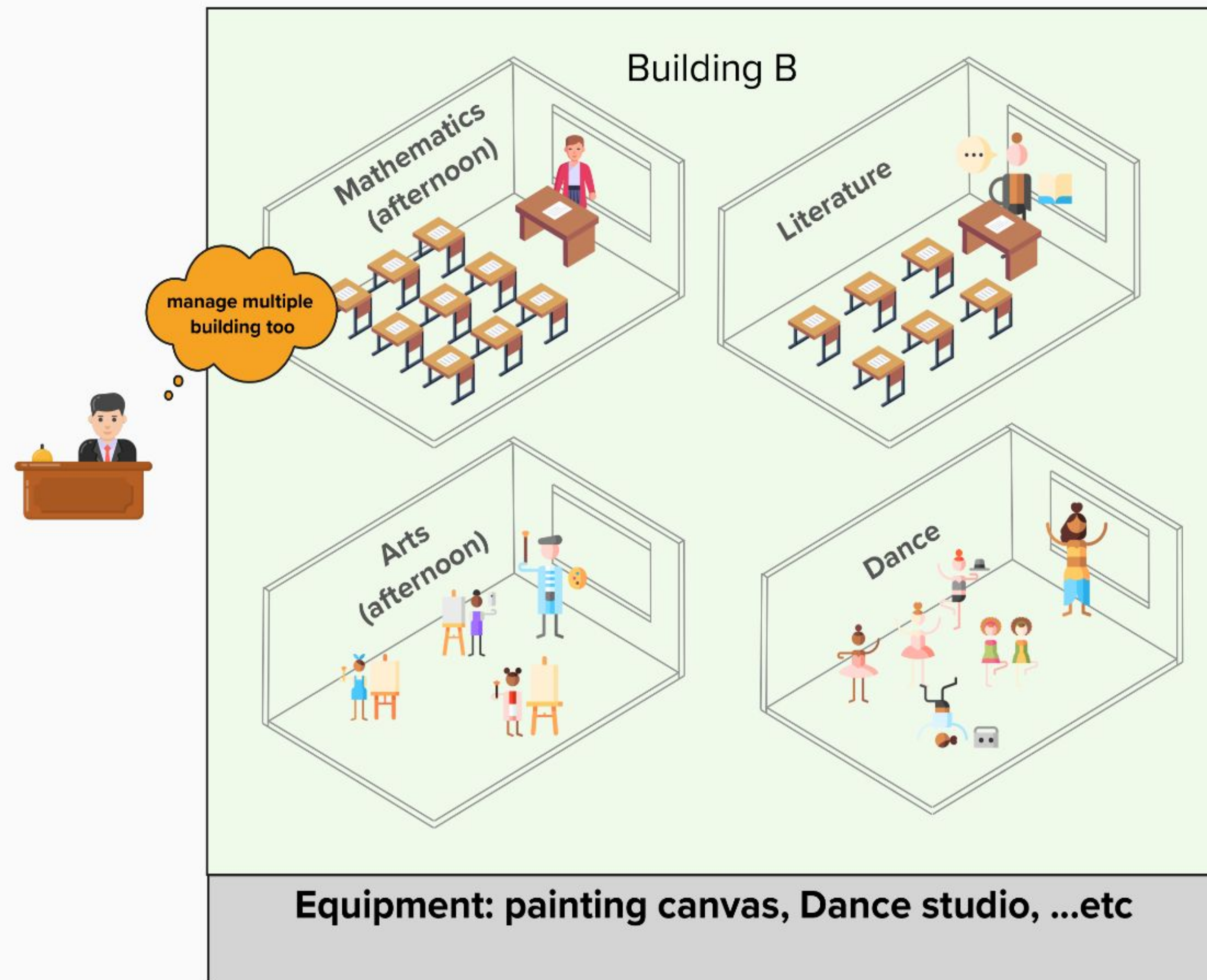
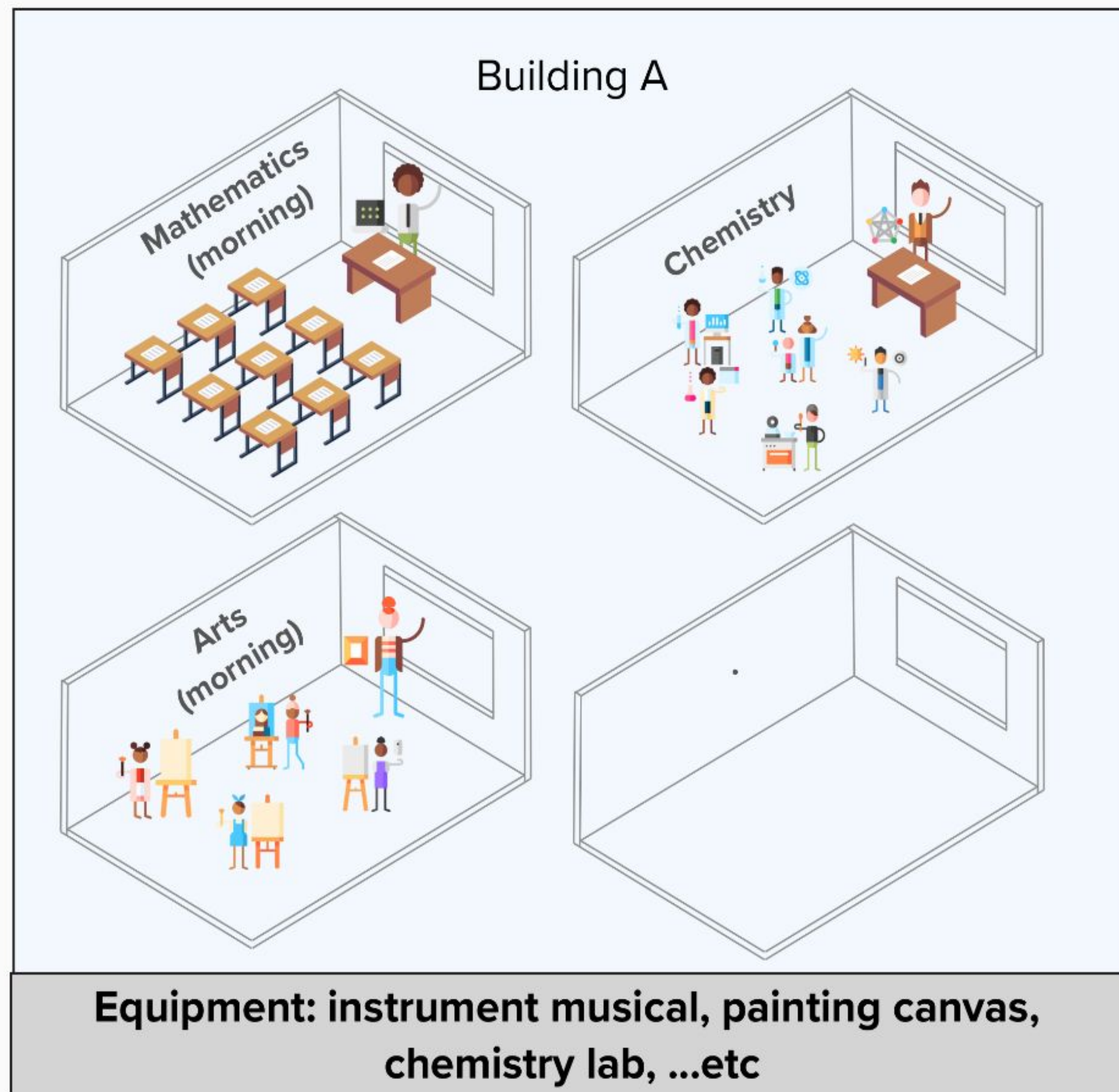
1. KUBERNETES



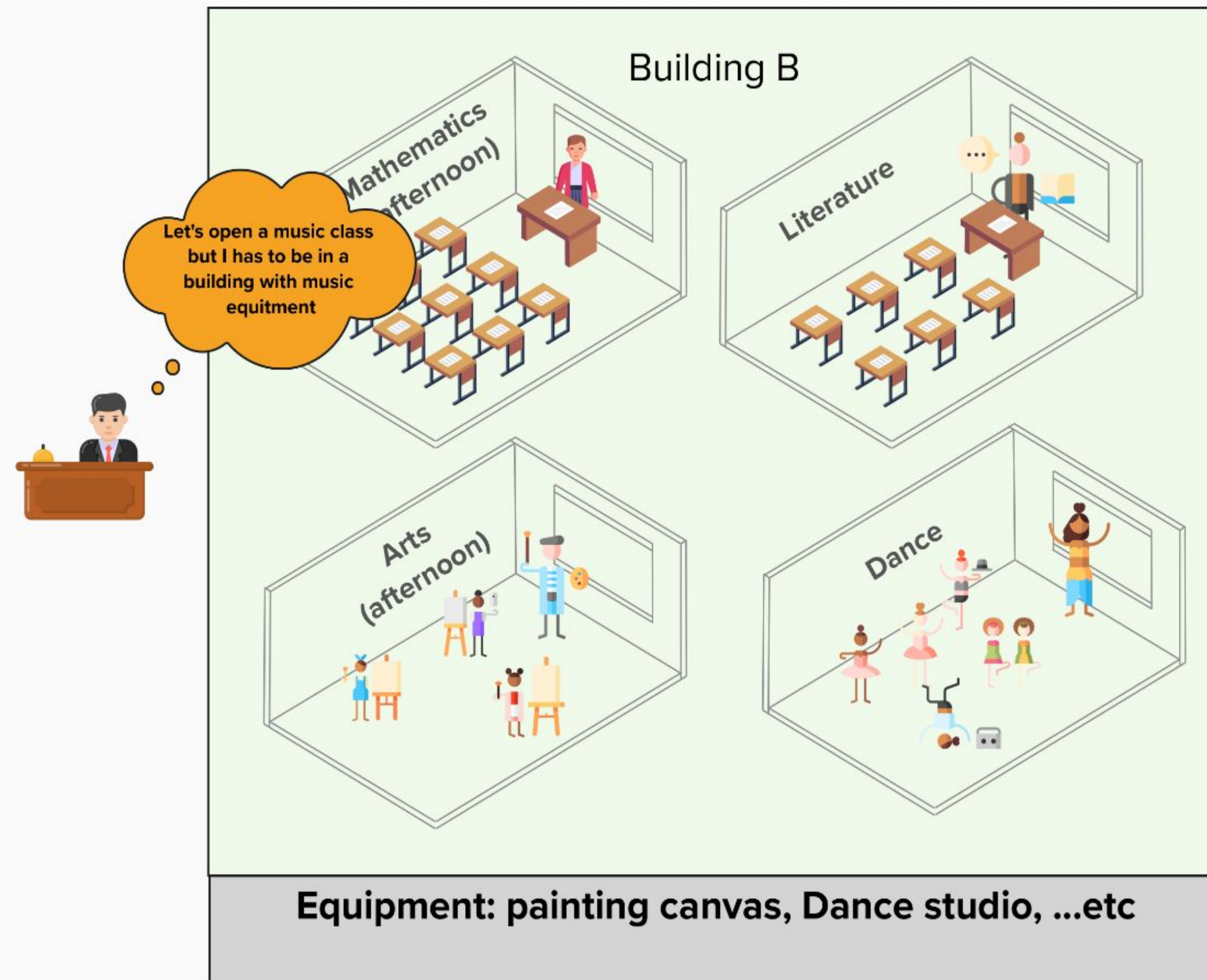
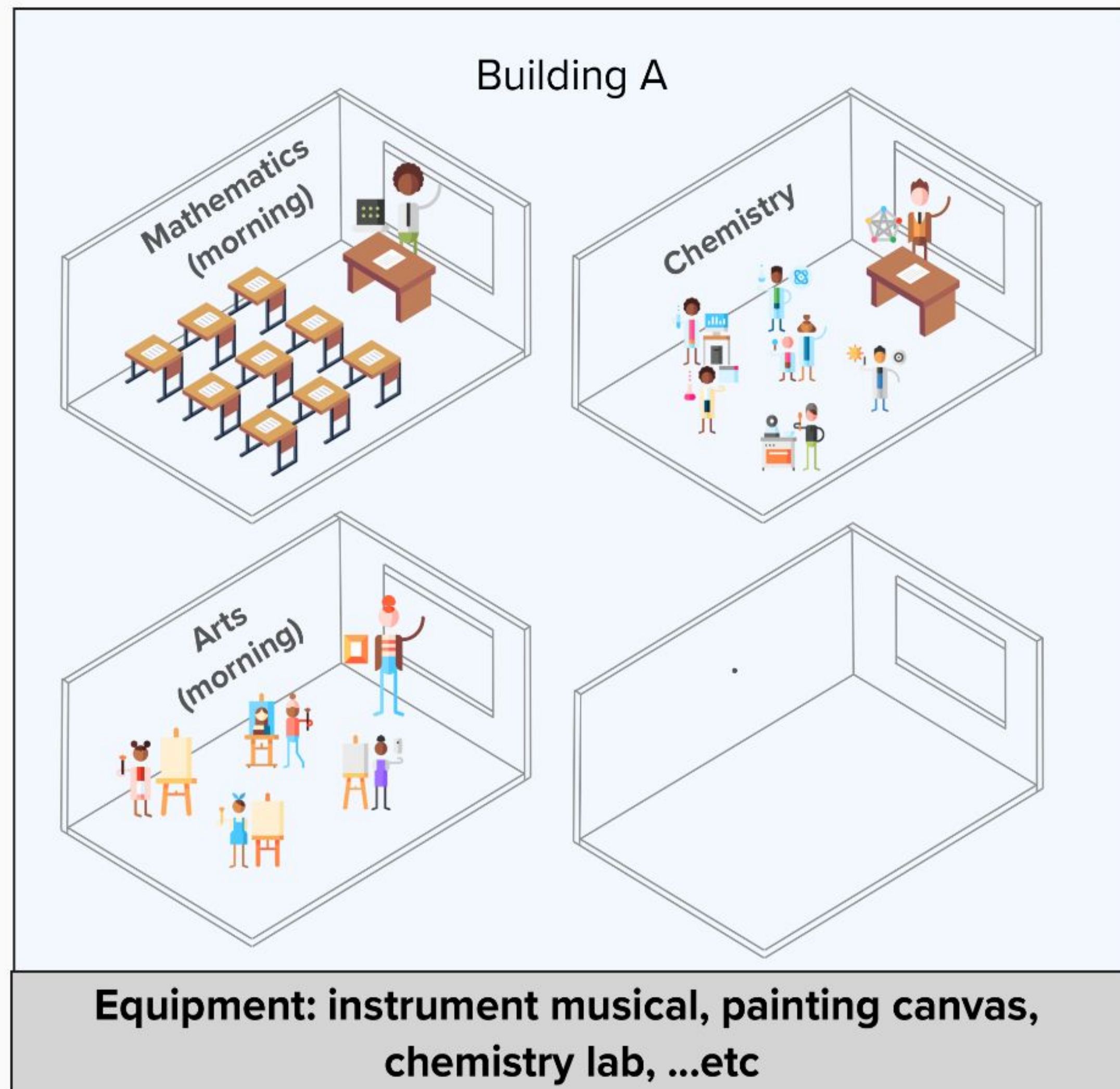
1. KUBERNETES



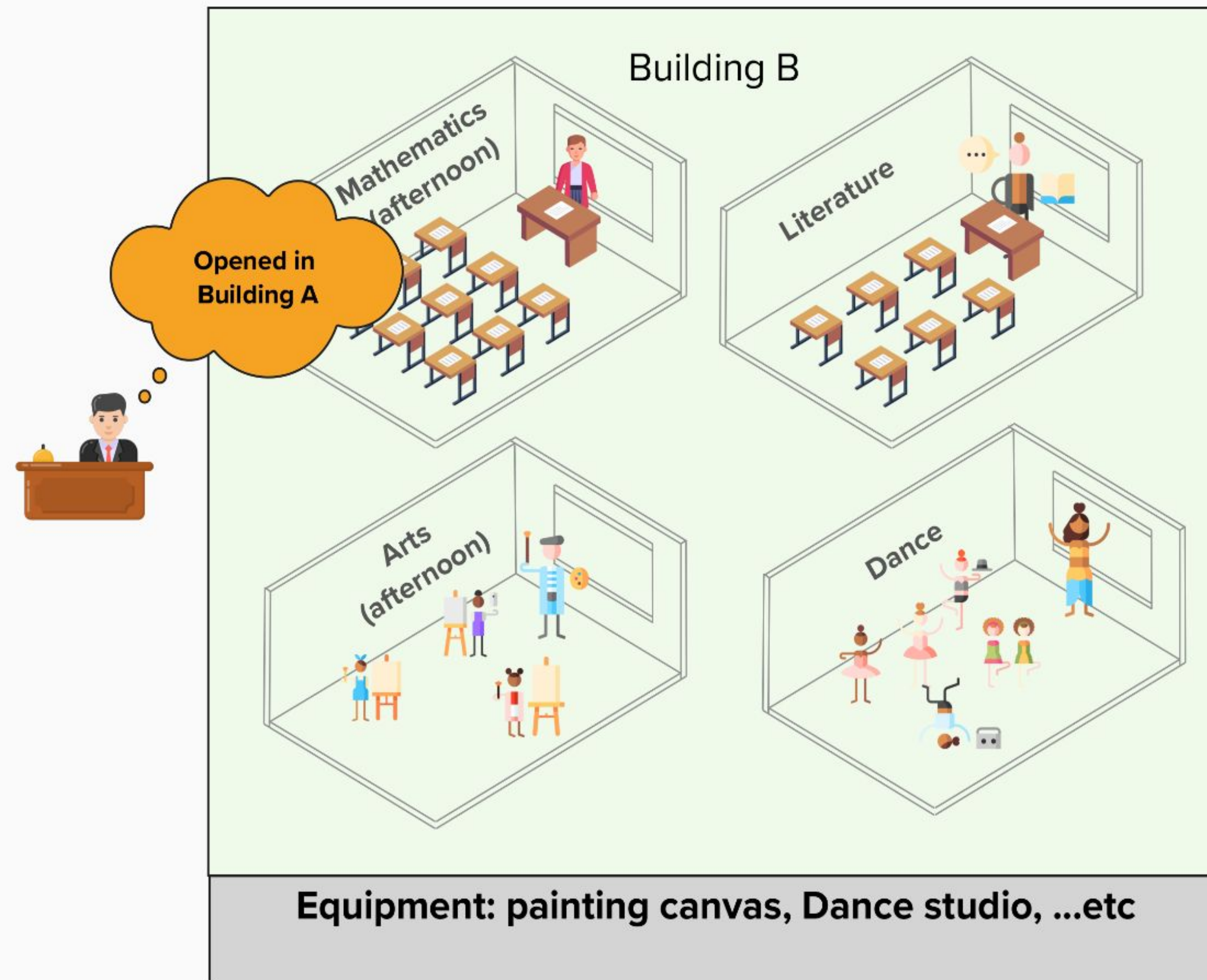
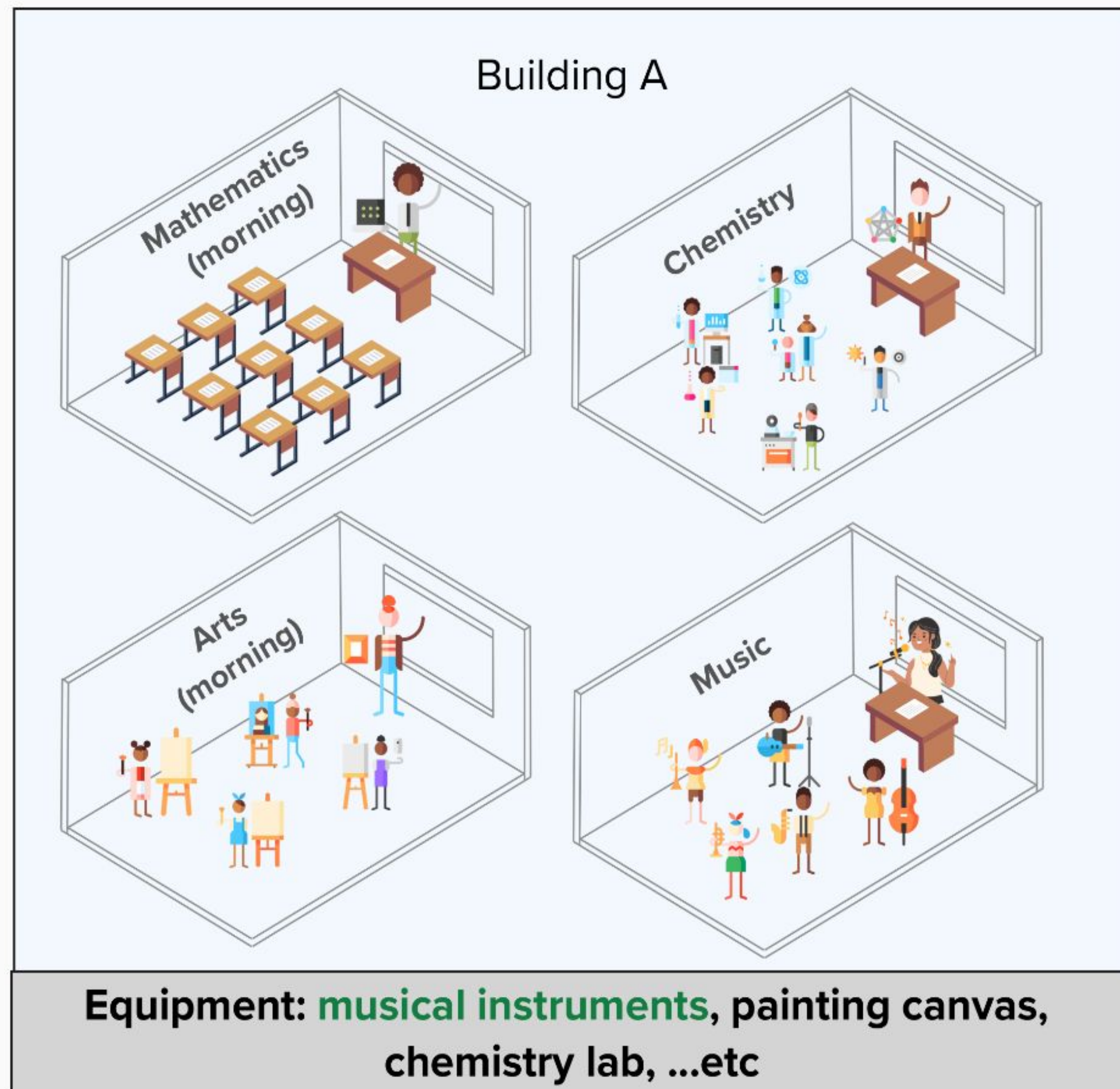
1. KUBERNETES



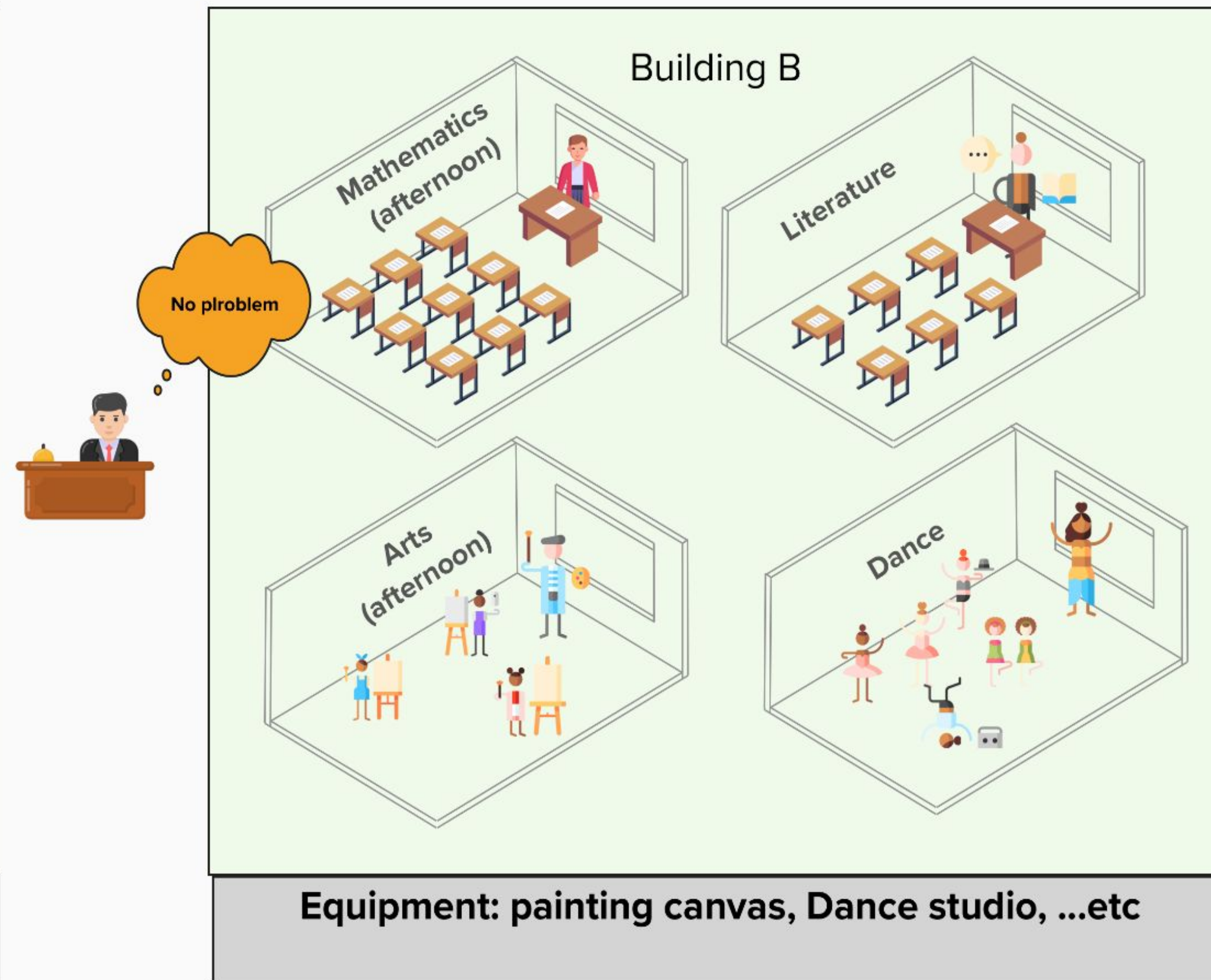
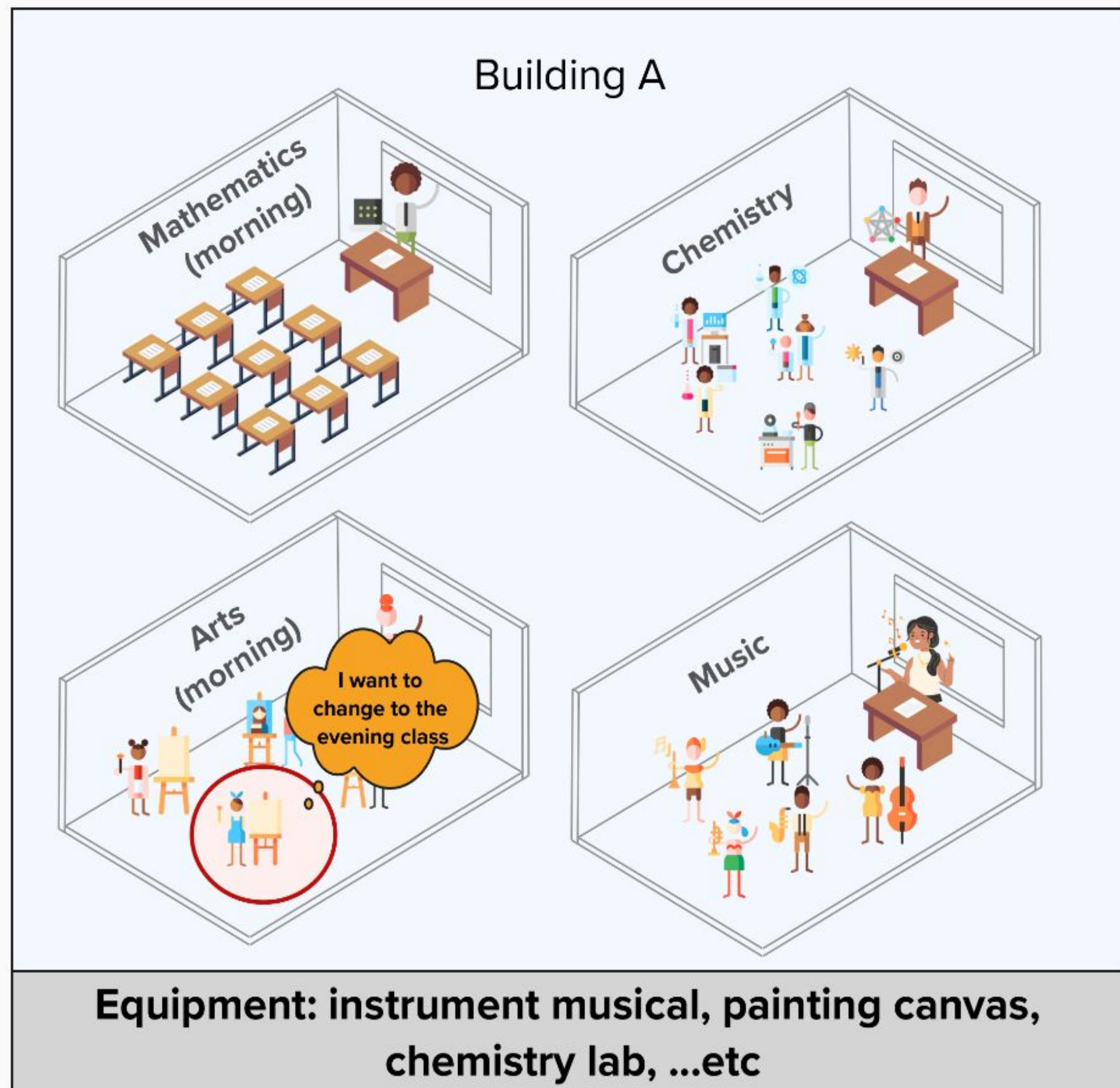
1. KUBERNETES



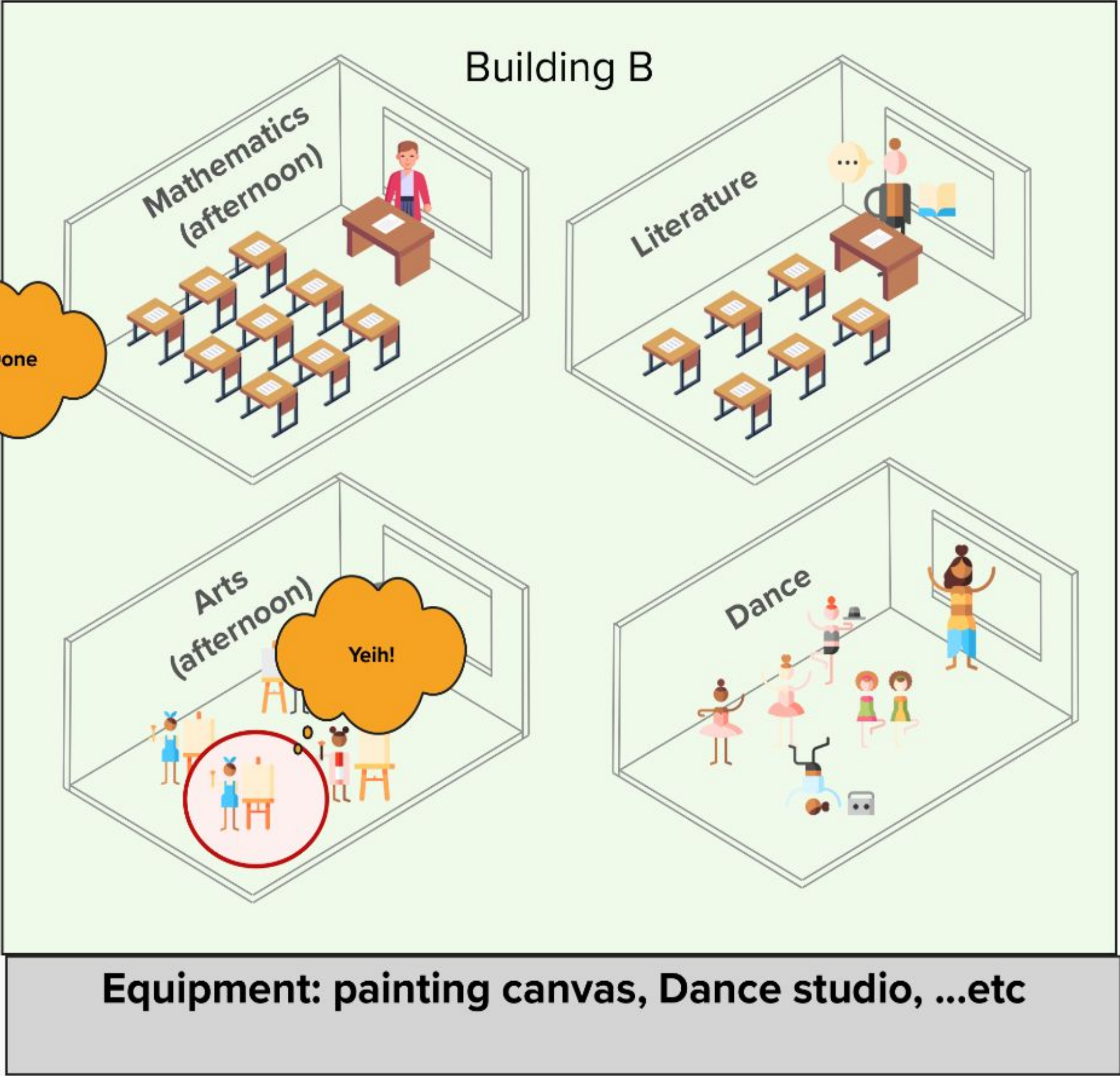
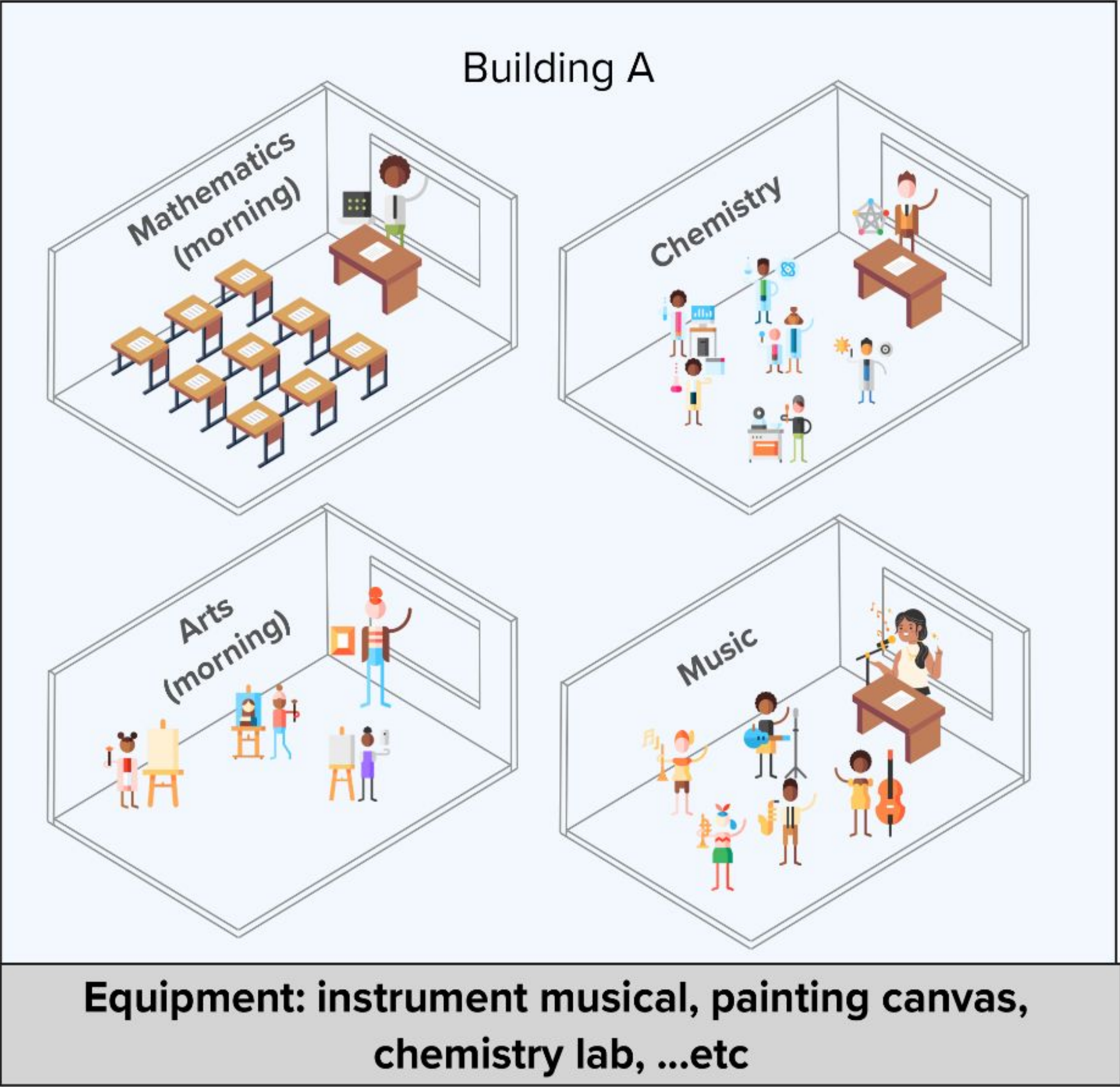
1. KUBERNETES



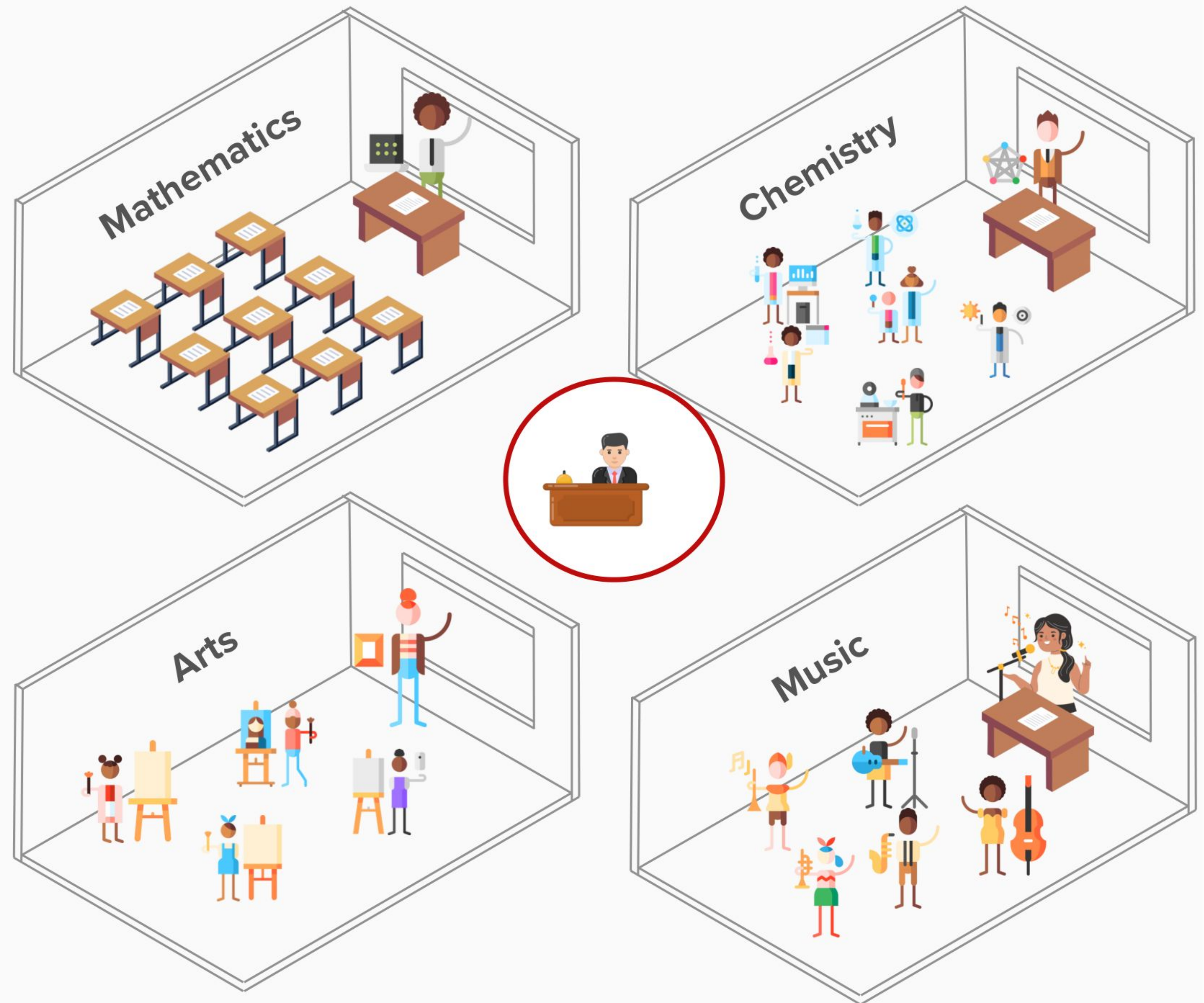
1. KUBERNETES



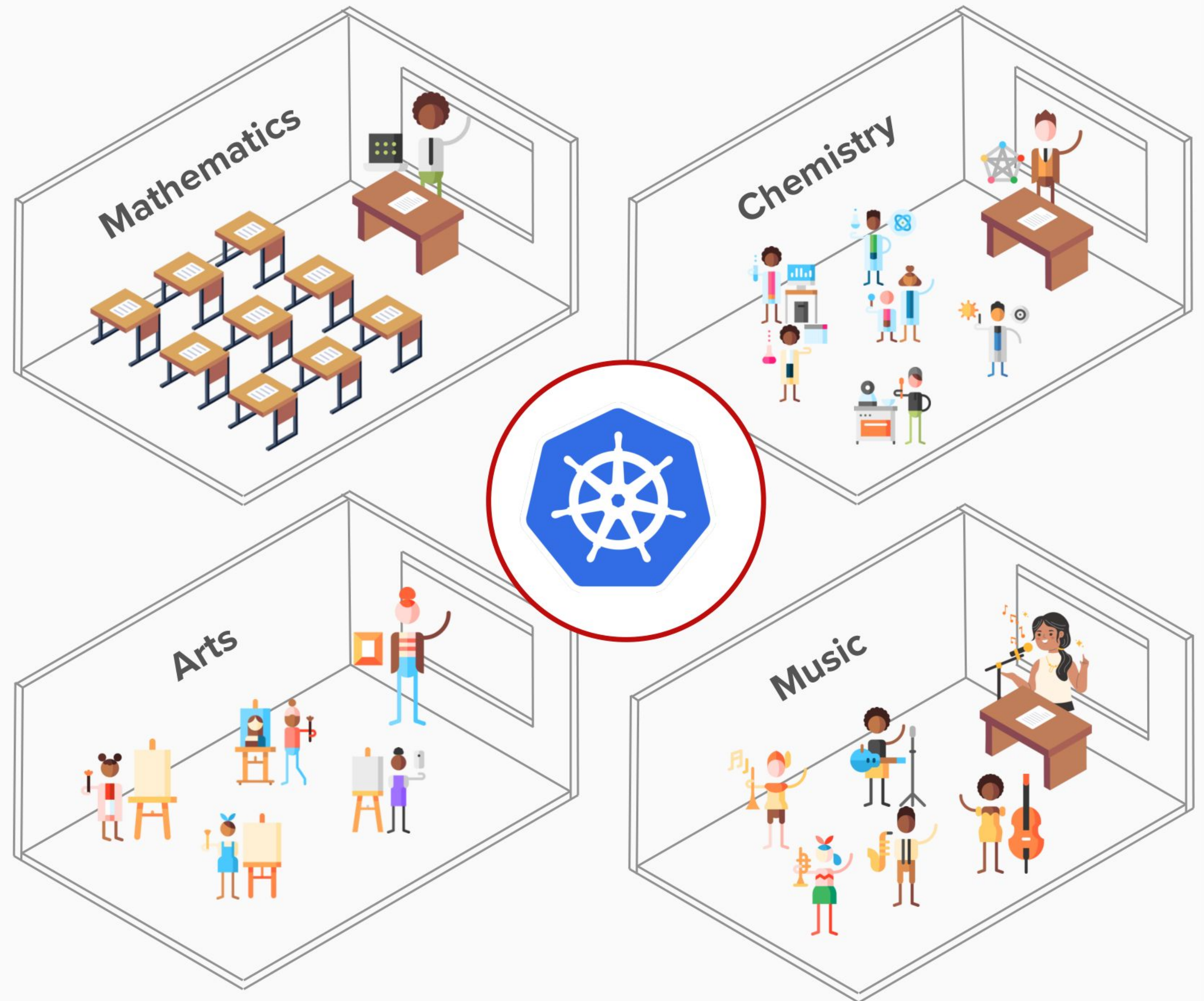
1. KUBERNETES



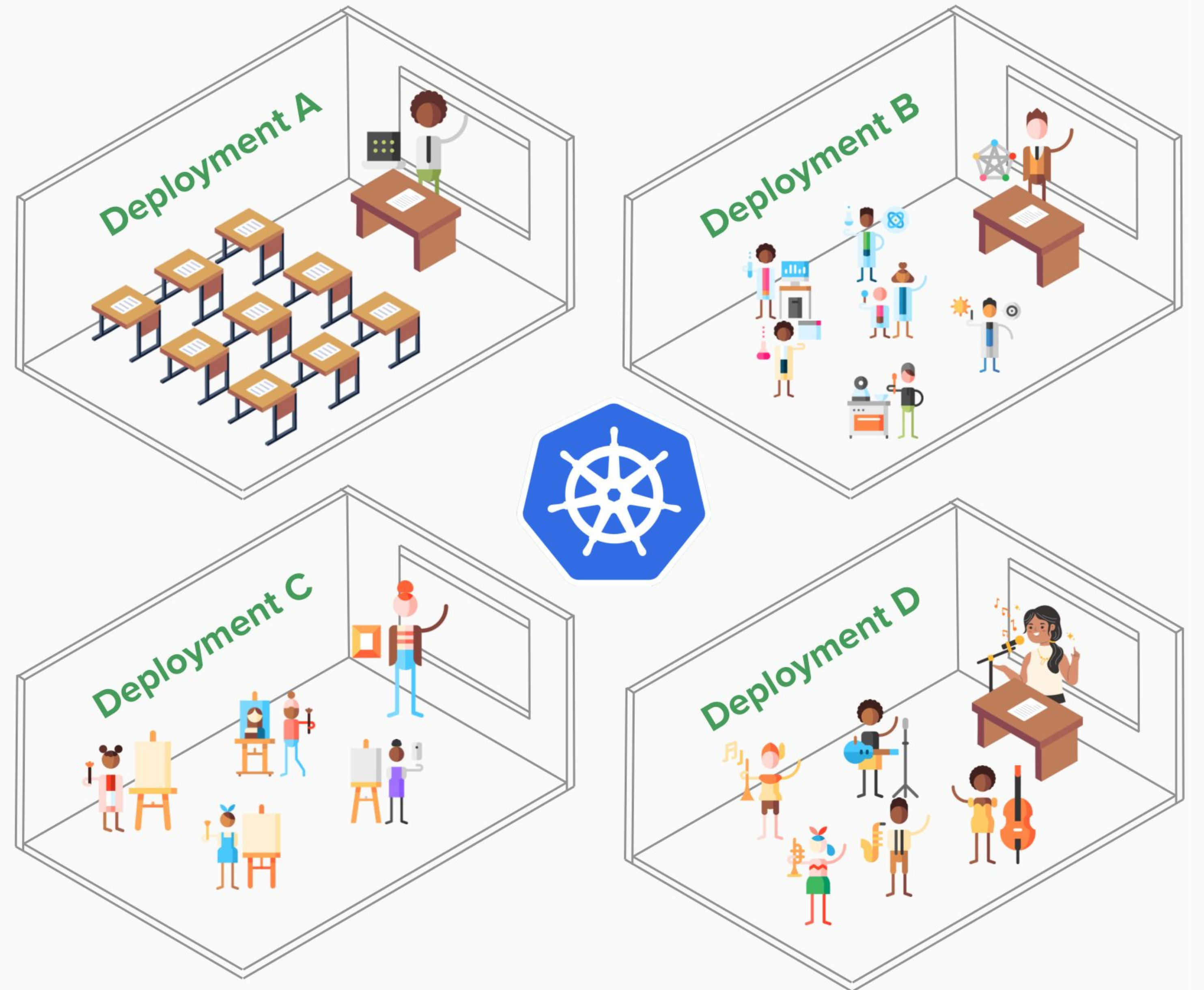
1. KUBERNETES



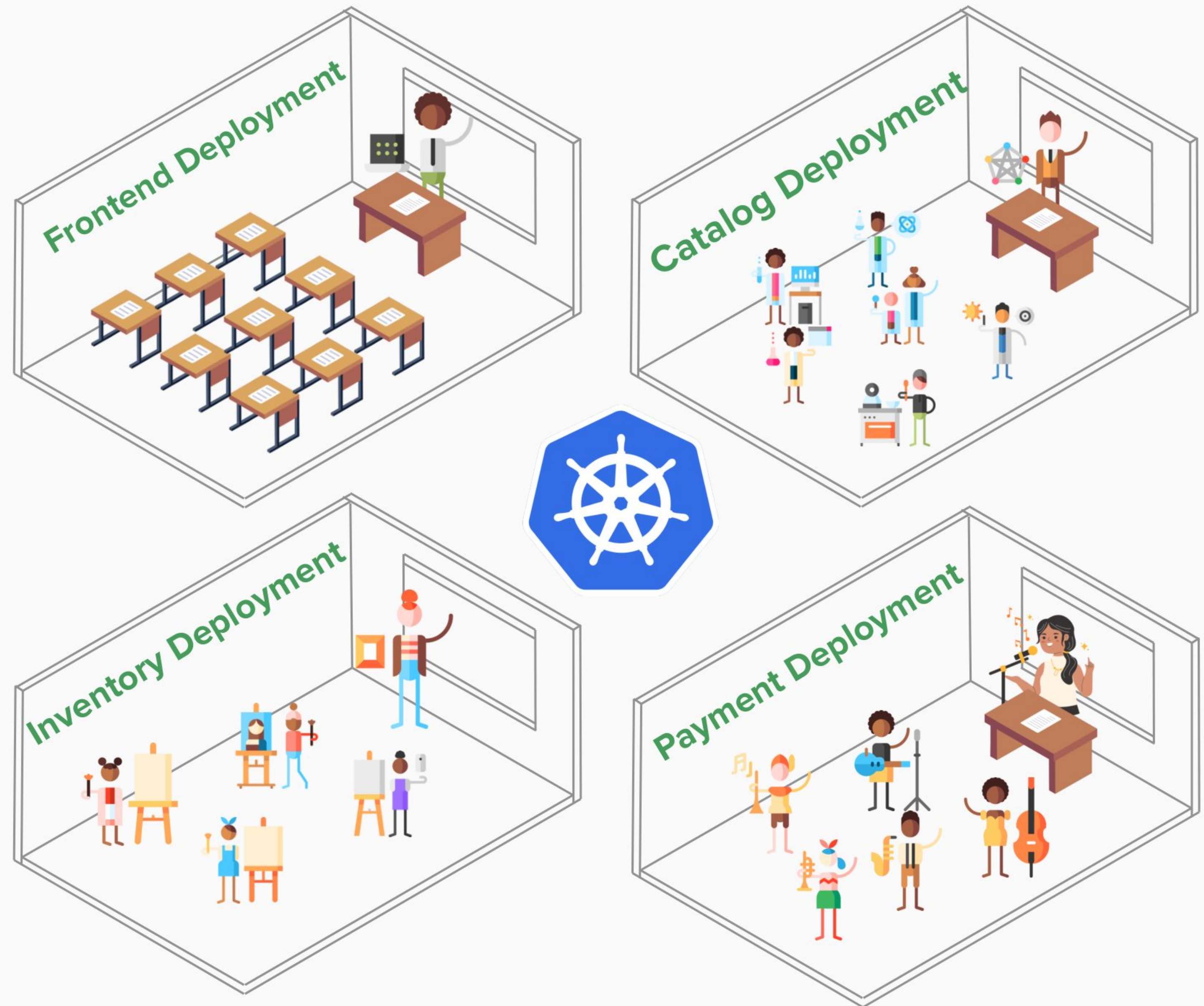
1. KUBERNETES



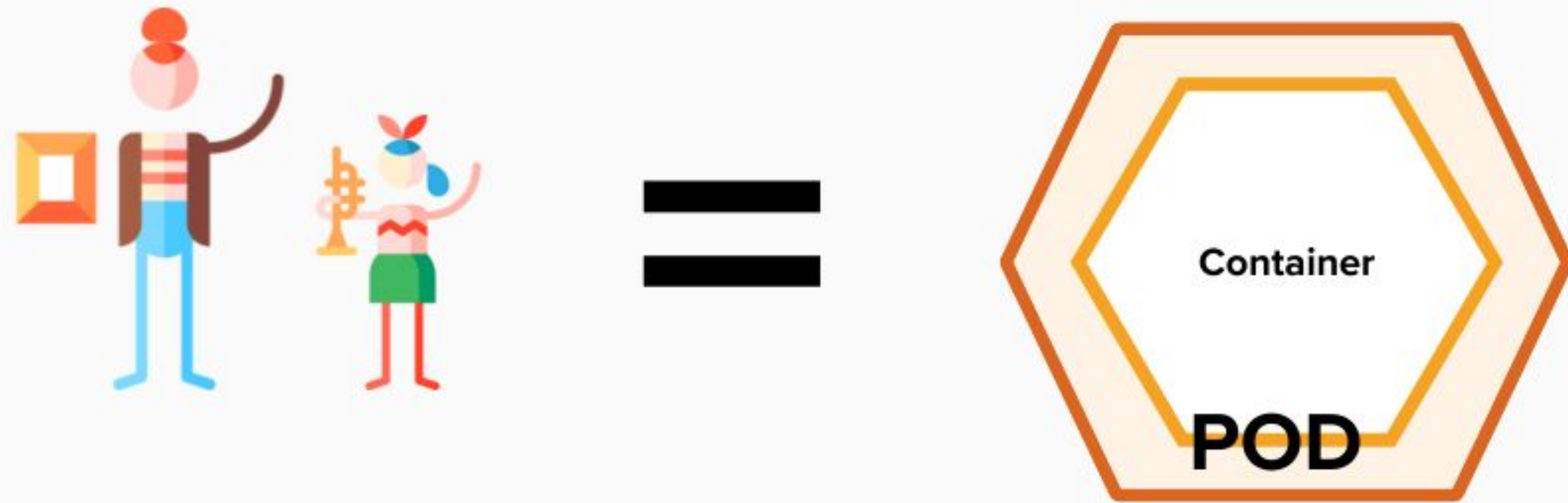
1. KUBERNETES



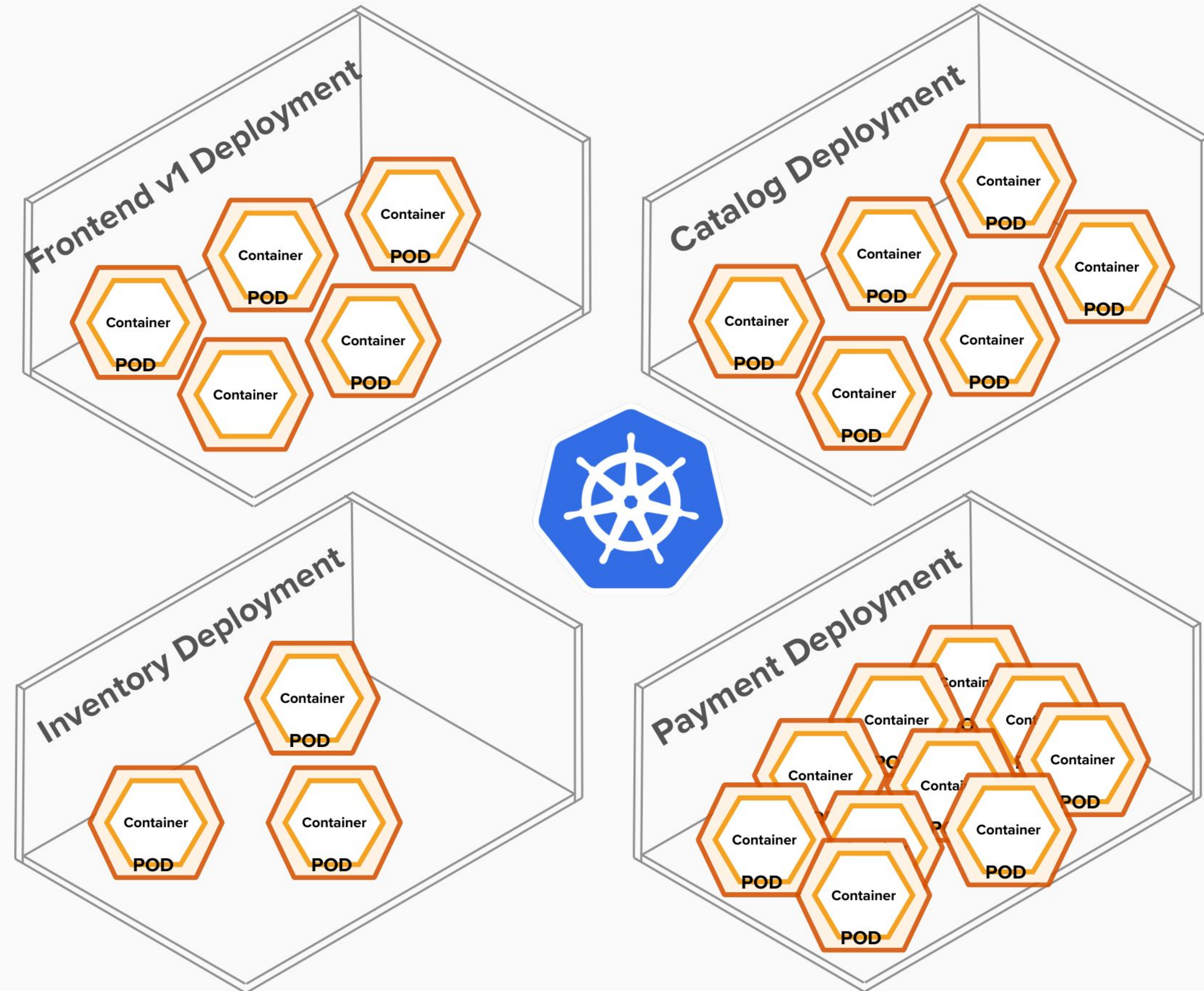
1. KUBERNETES



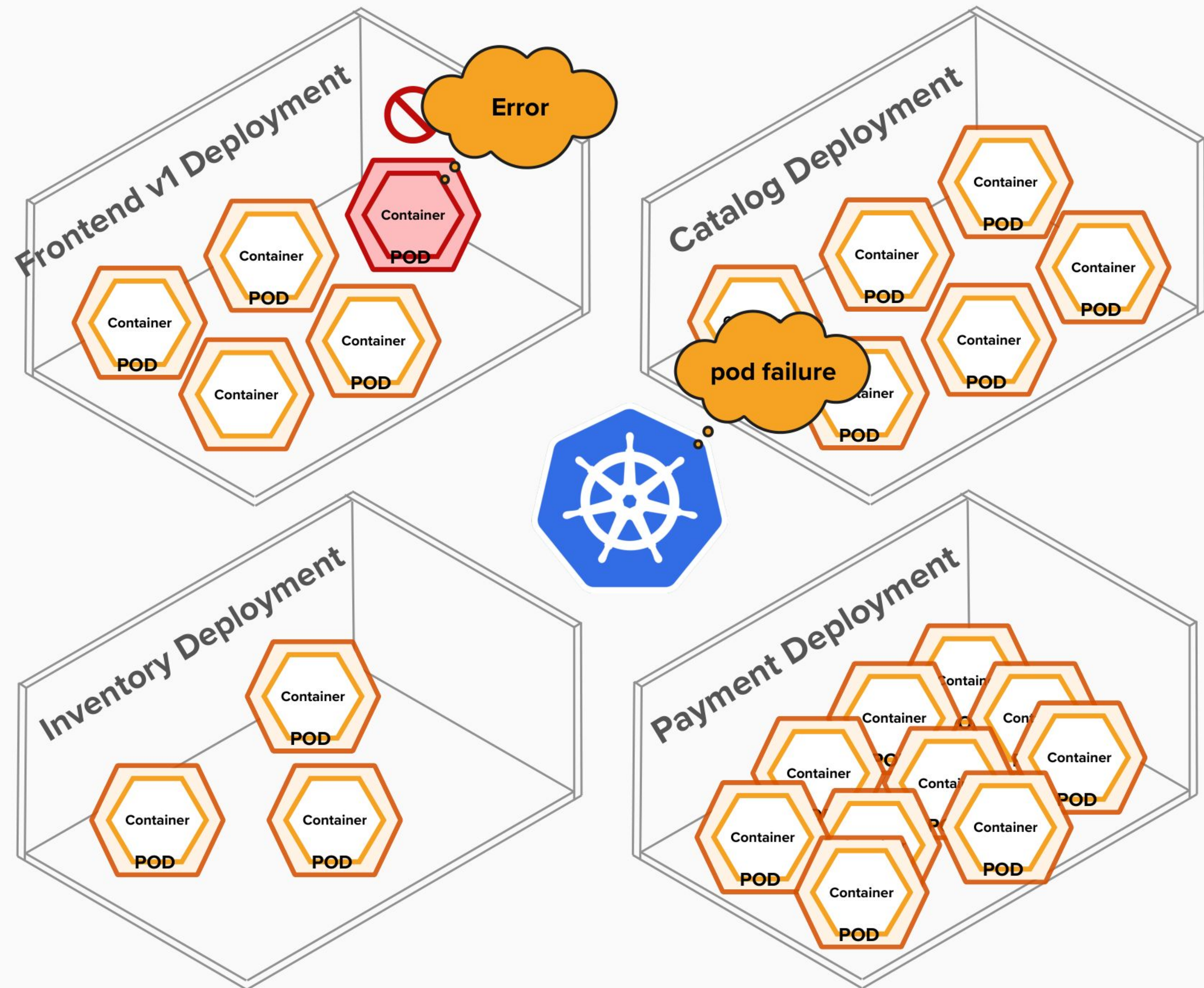
1. KUBERNETES



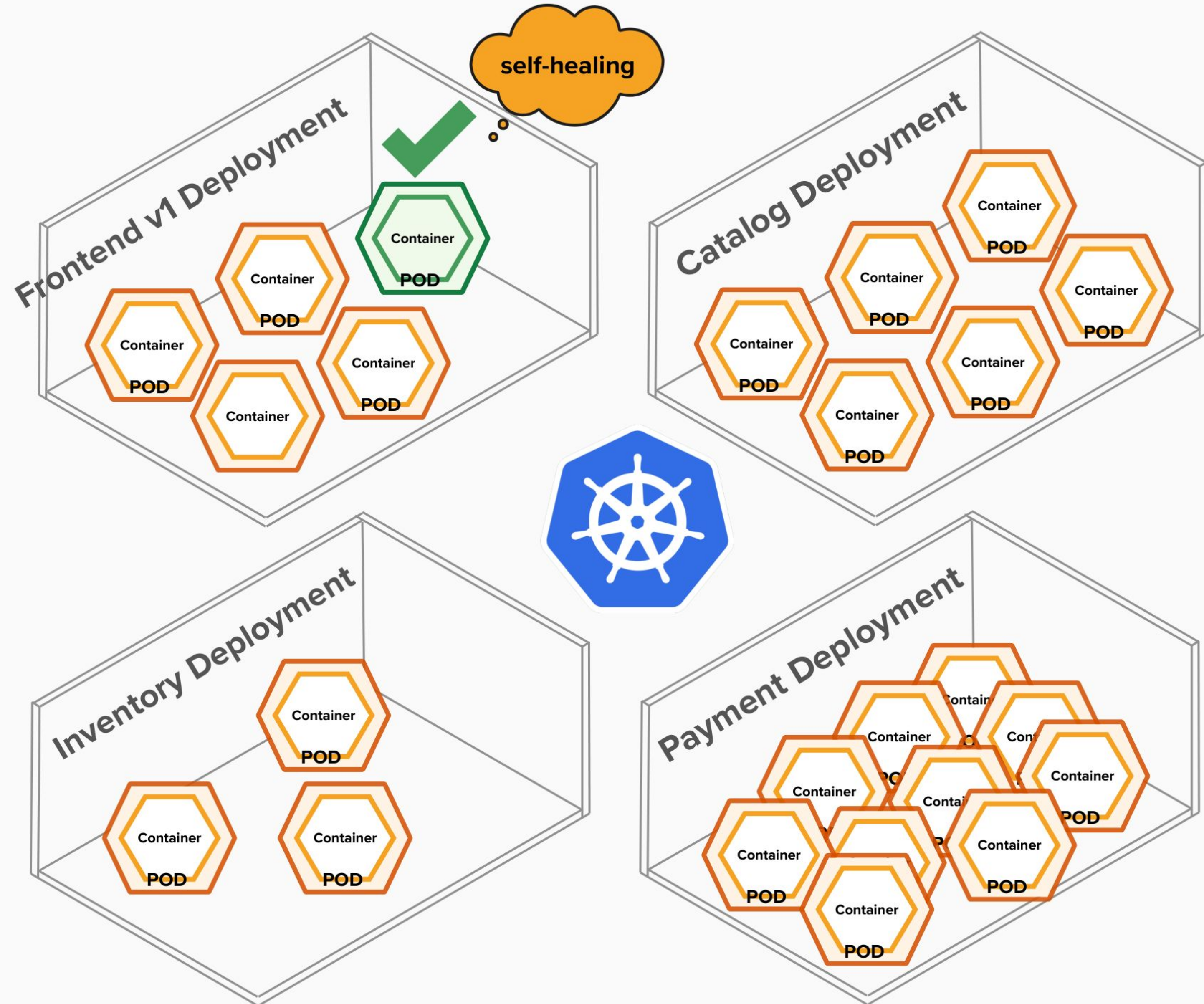
1. KUBERNETES



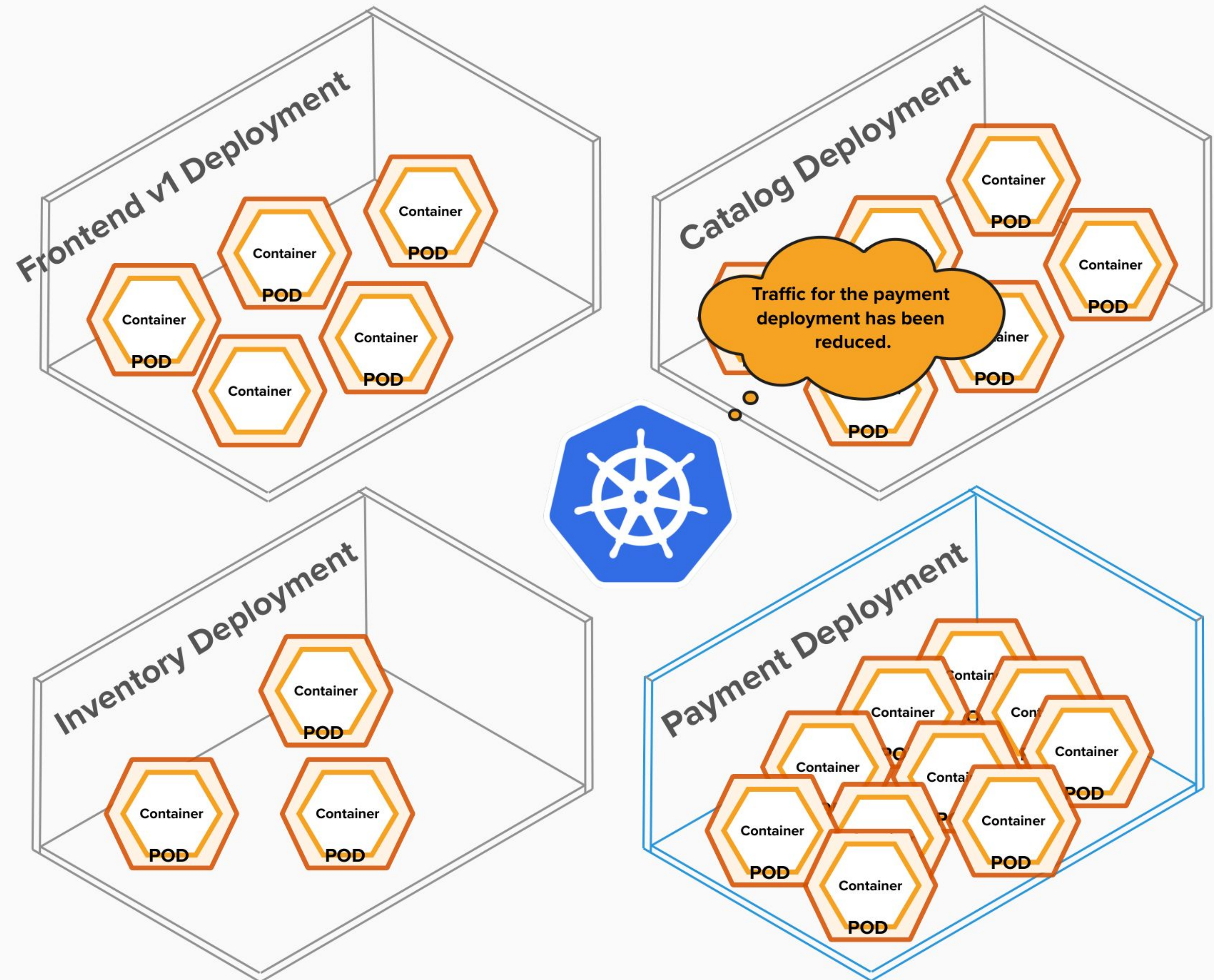
1. KUBERNETES



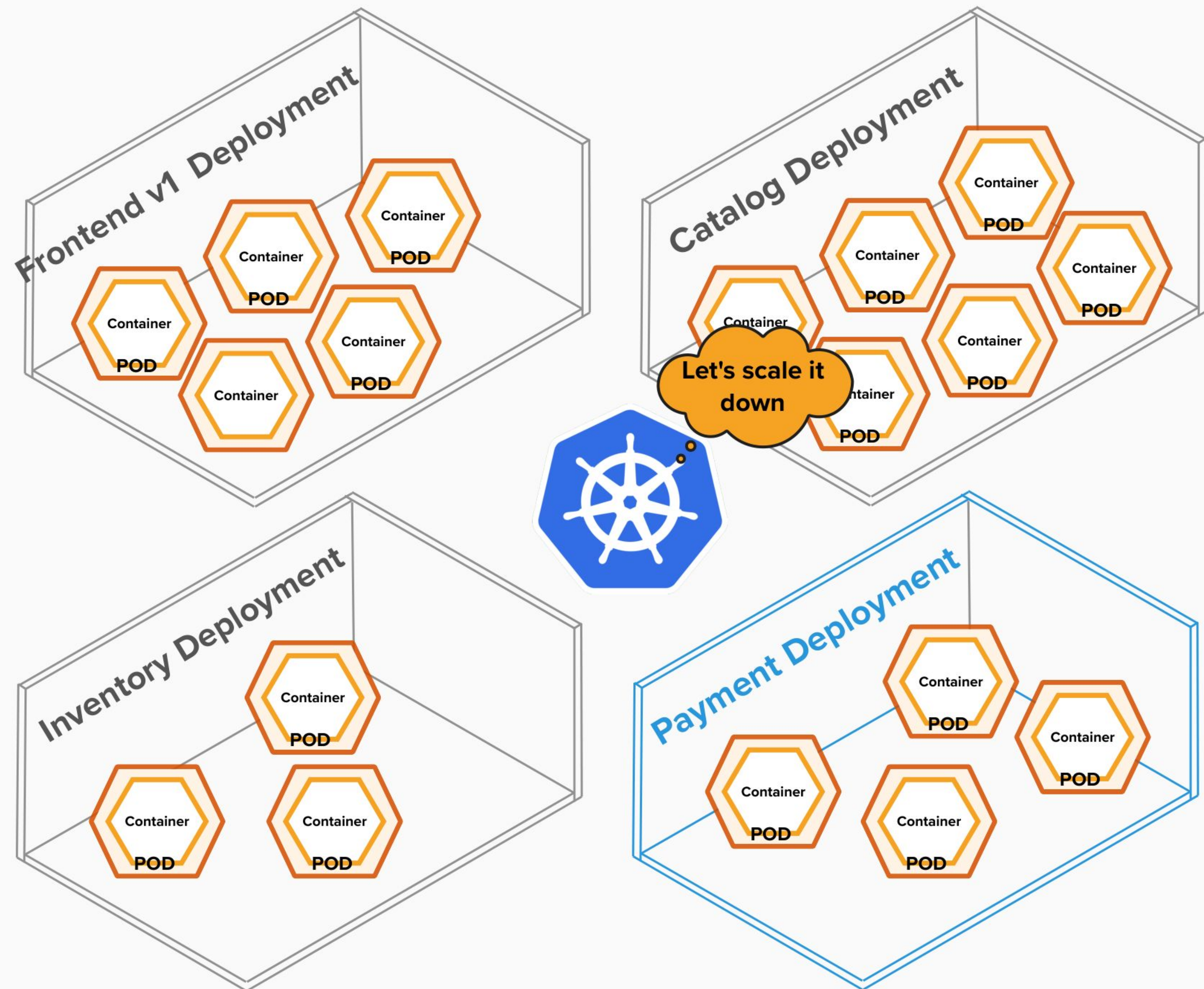
1. KUBERNETES



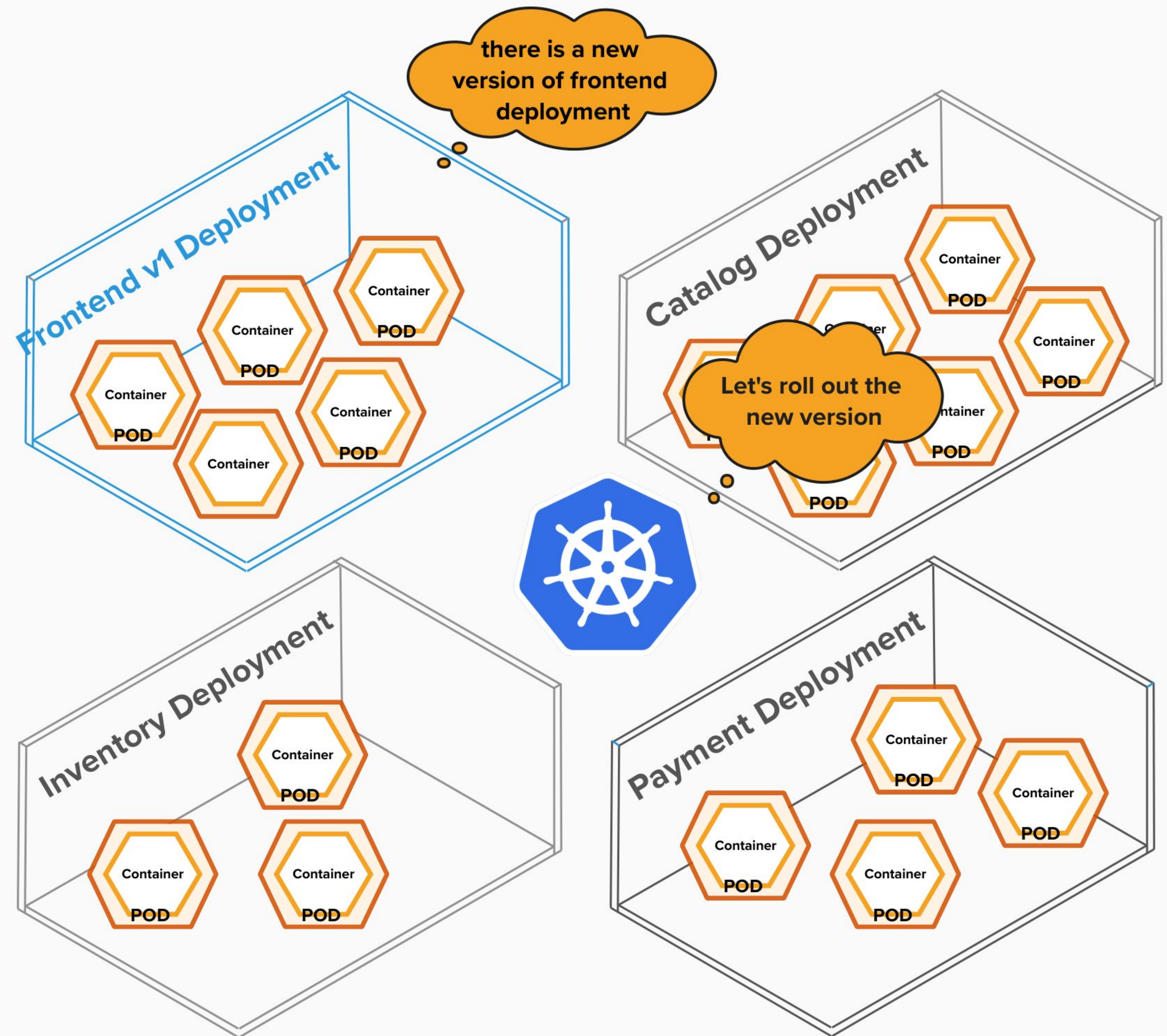
1. KUBERNETES



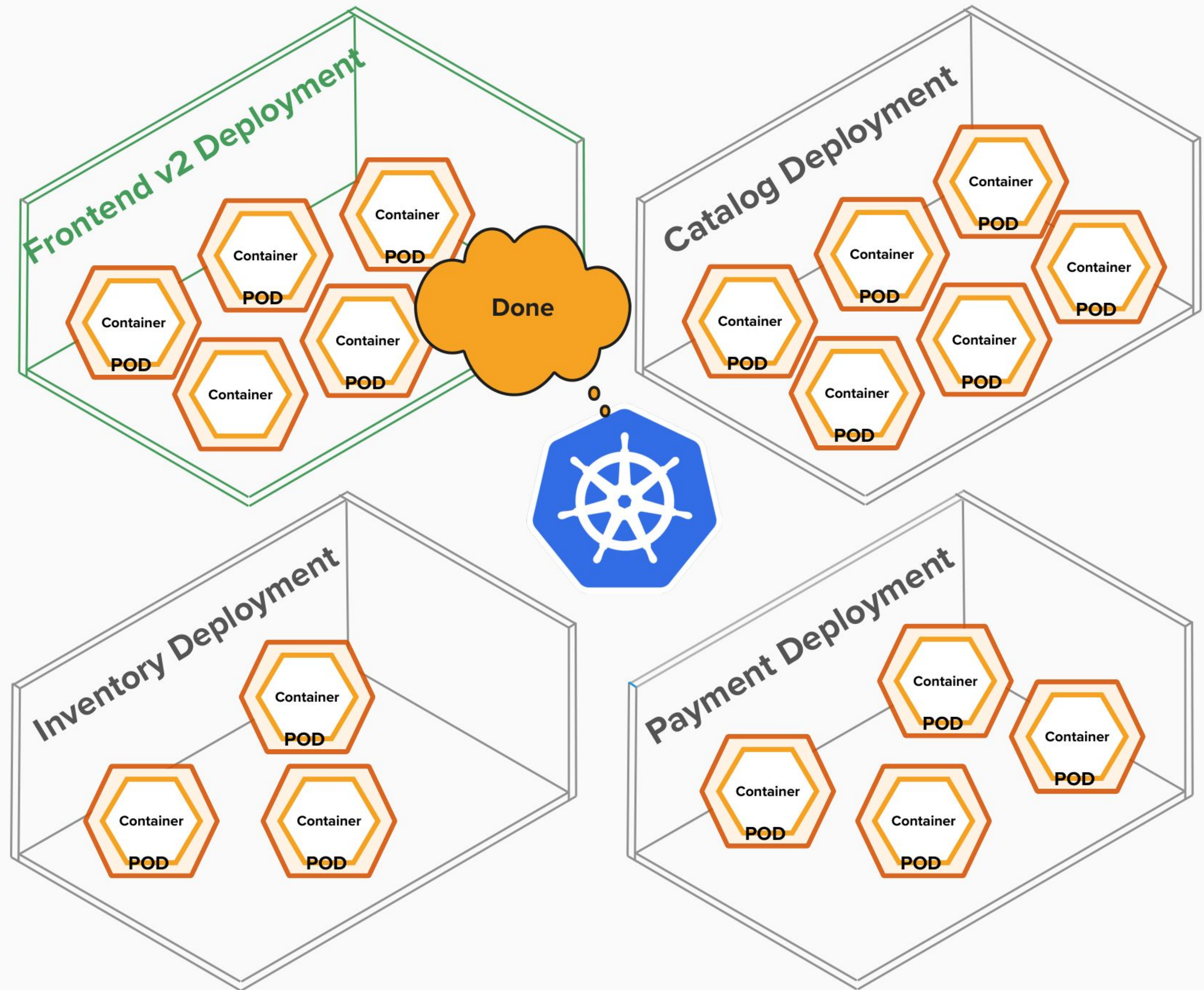
1. KUBERNETES



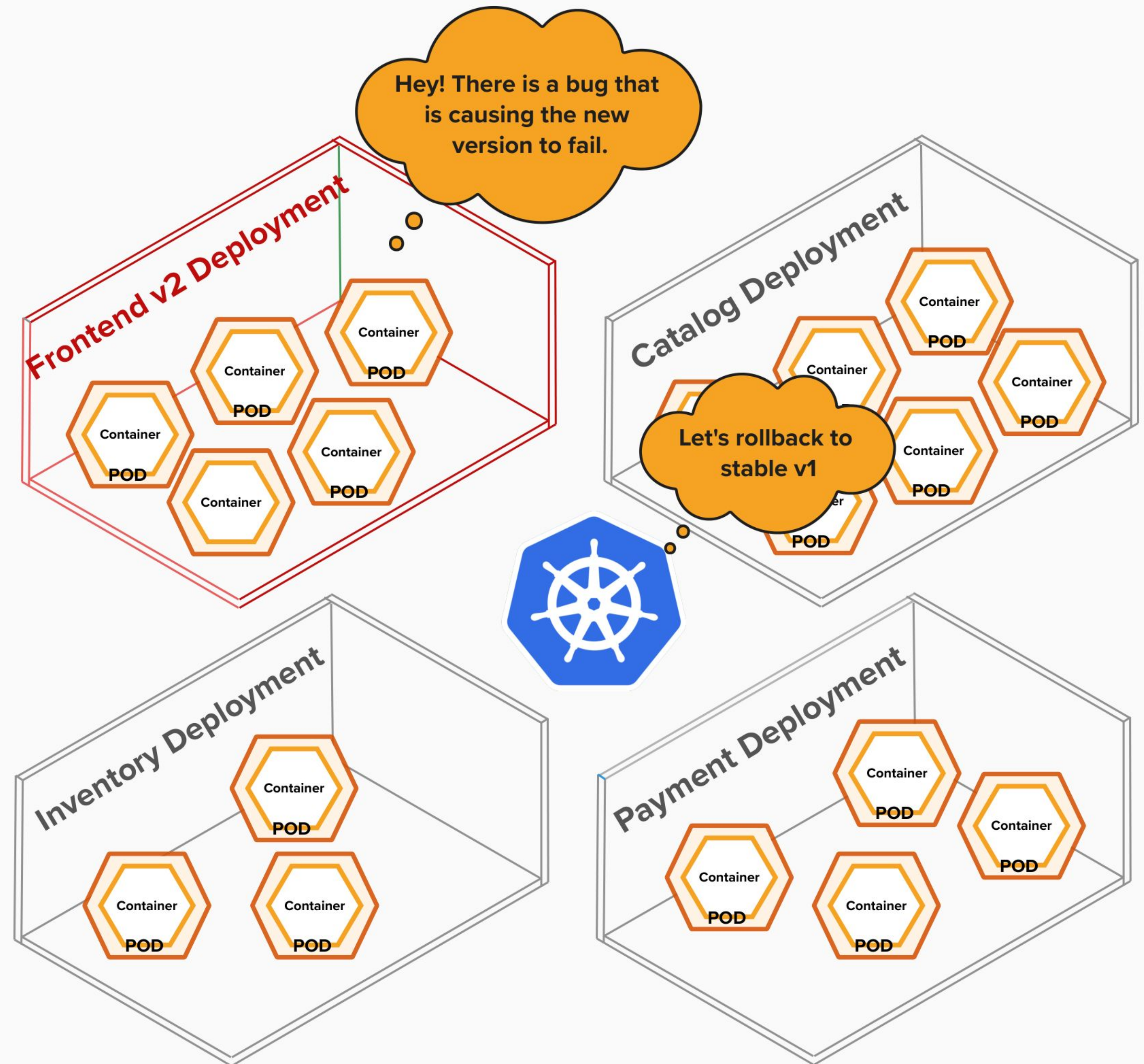
1. KUBERNETES



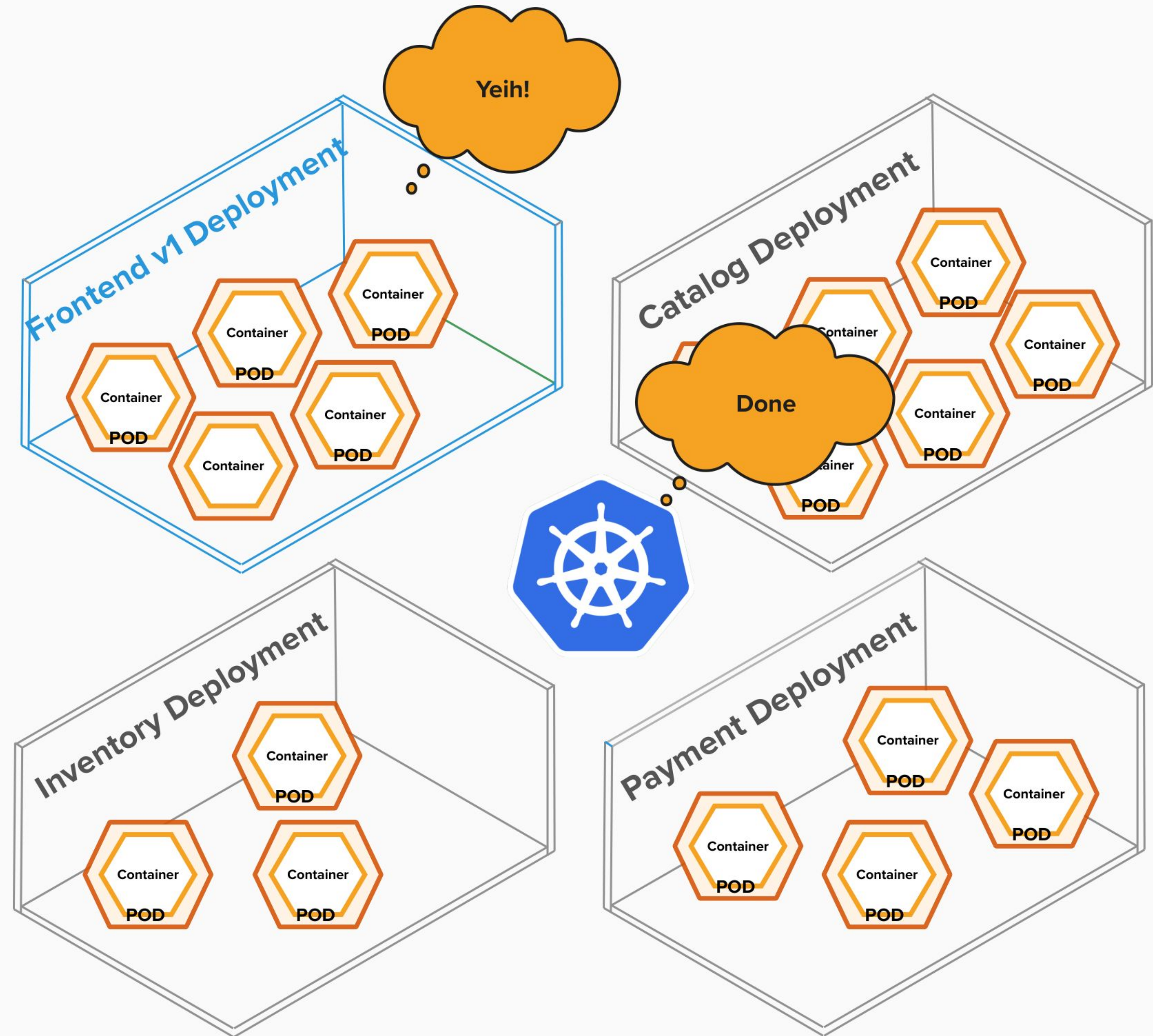
1. KUBERNETES



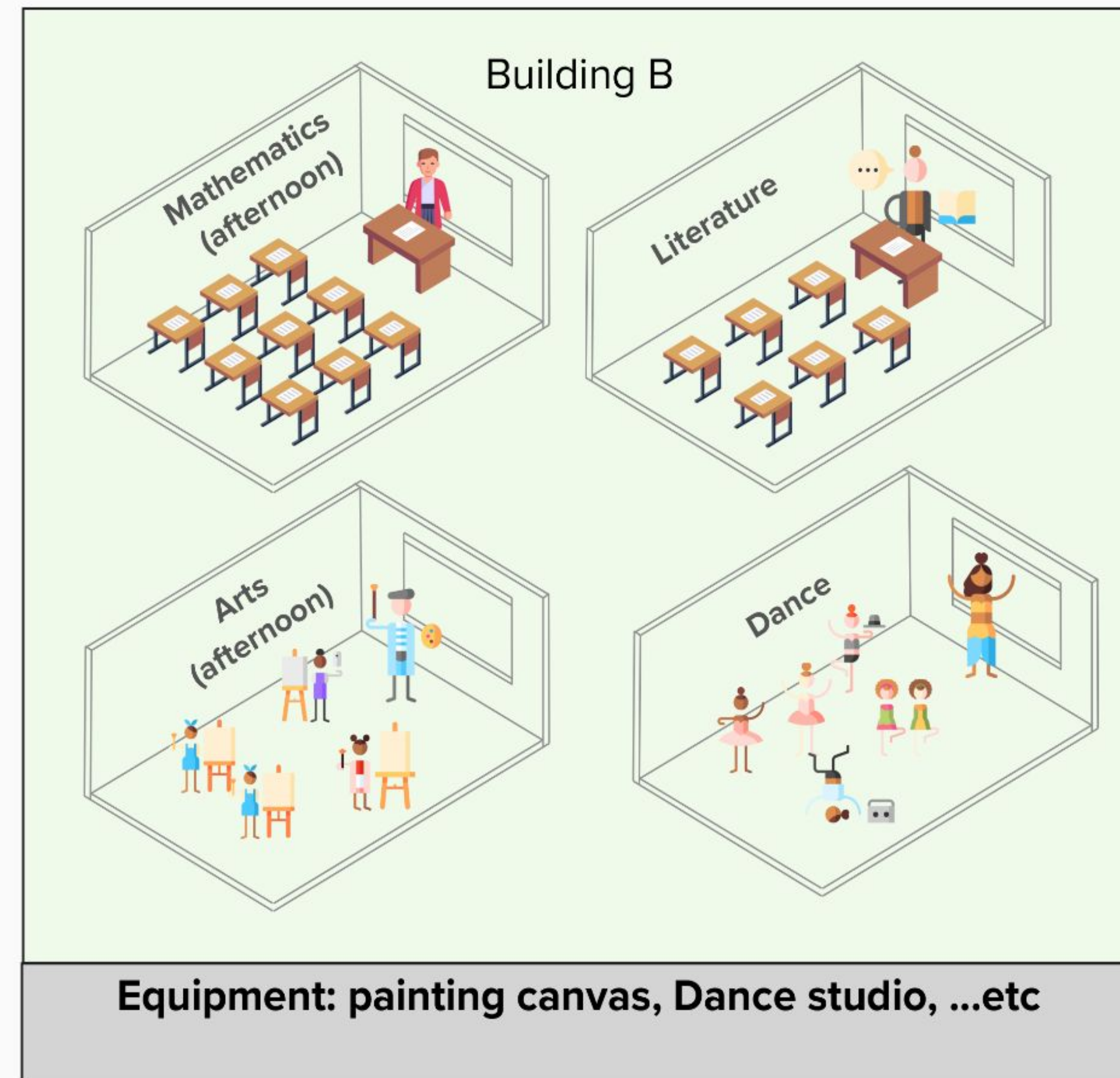
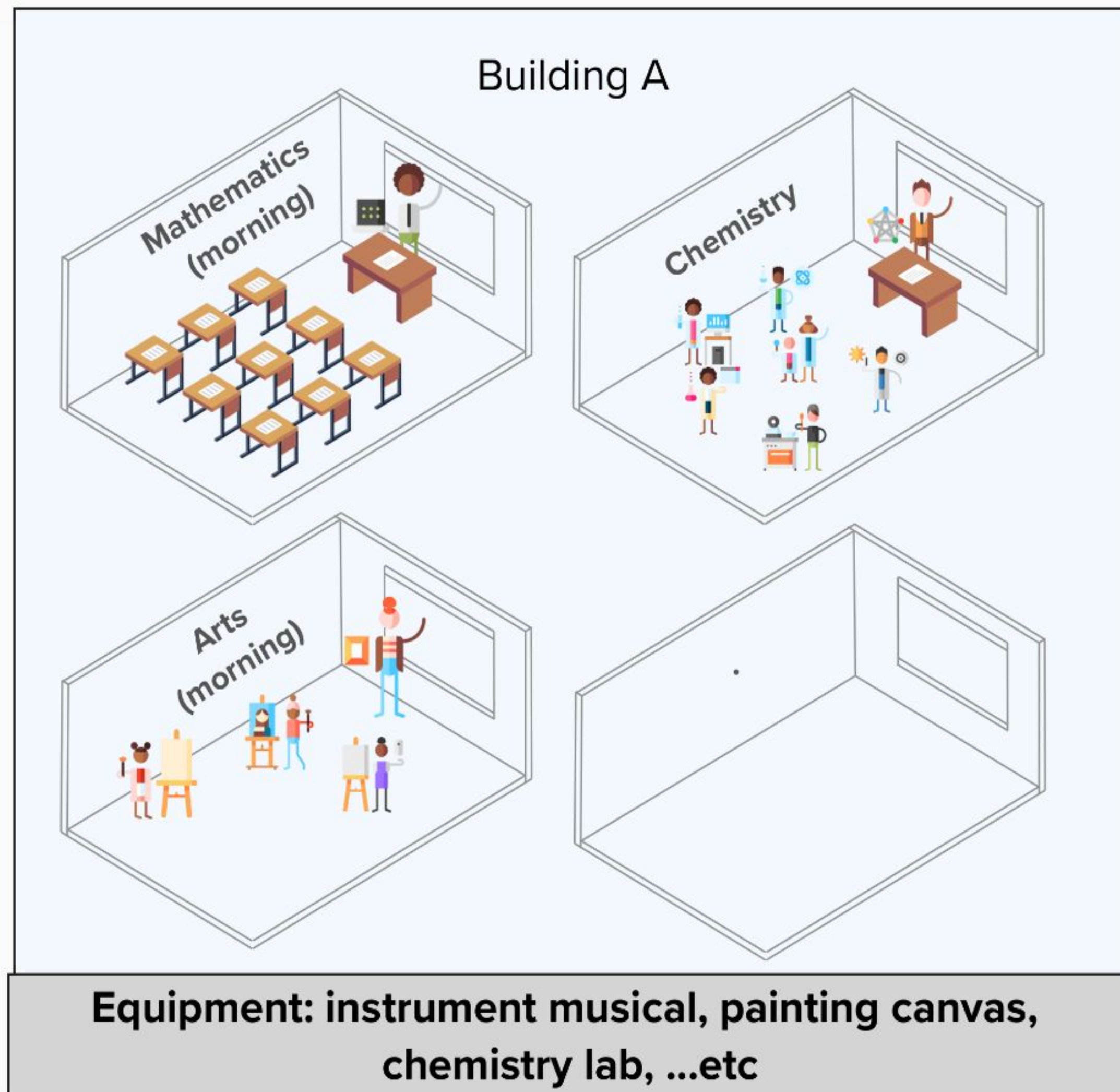
1. KUBERNETES



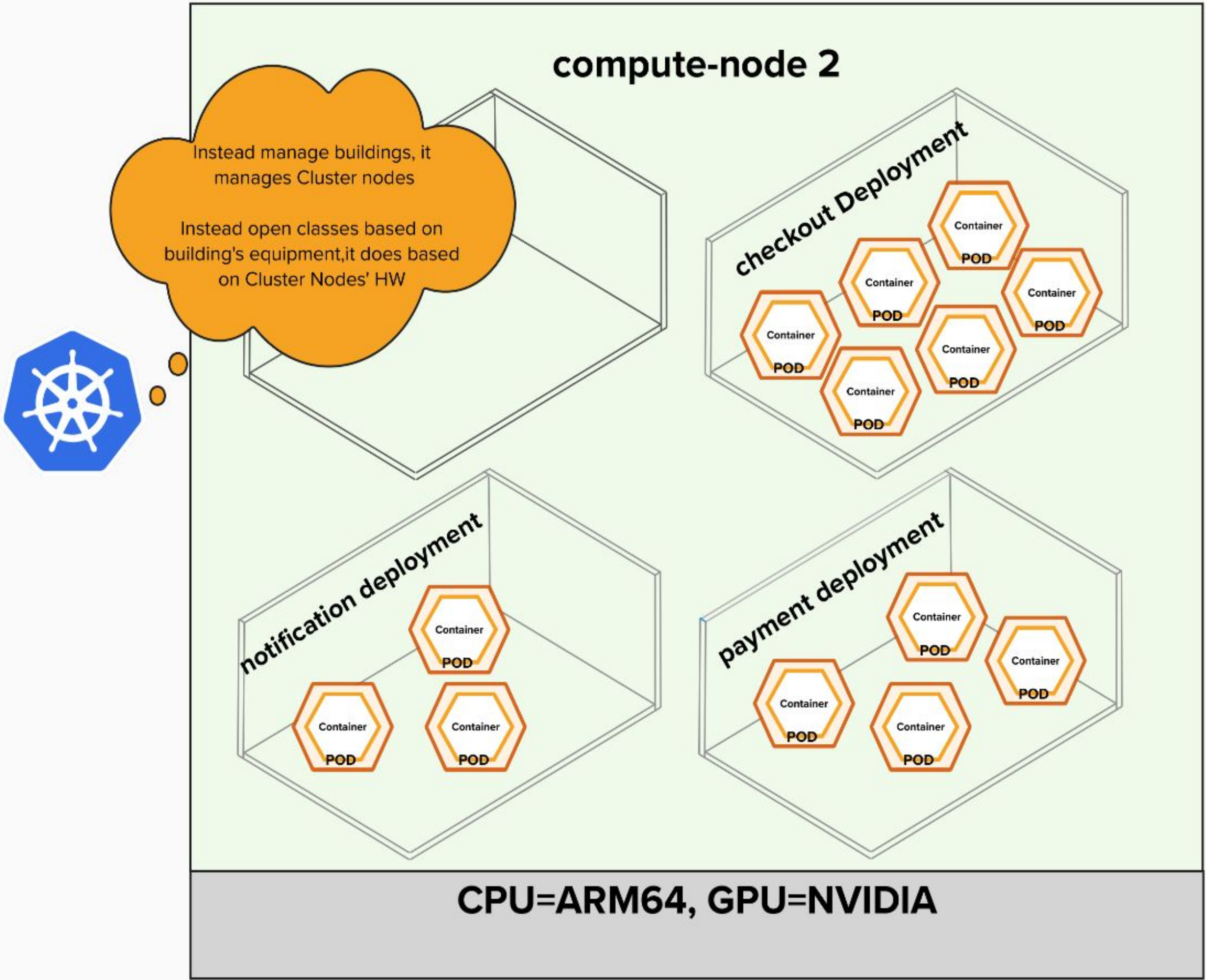
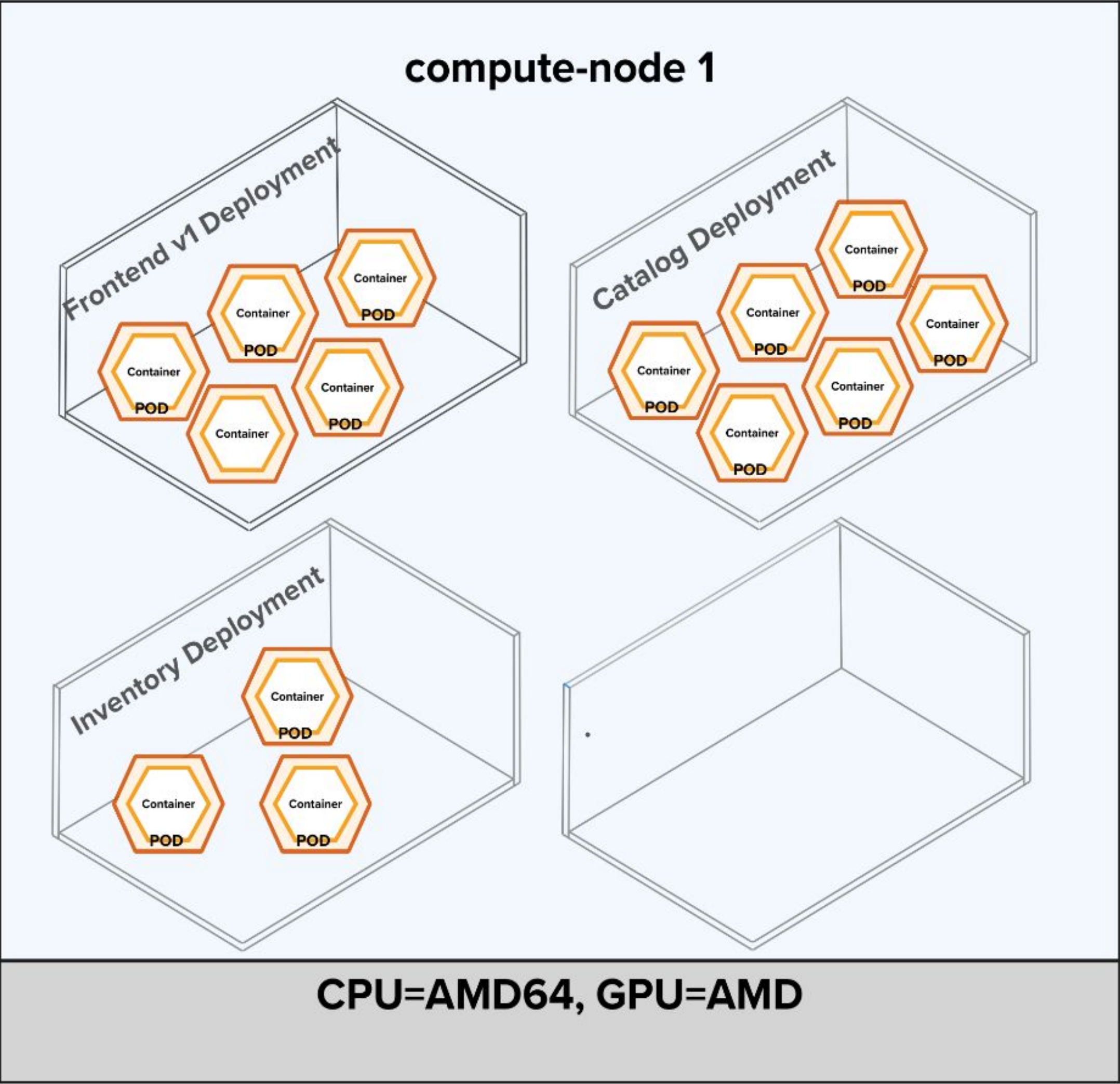
1. KUBERNETES



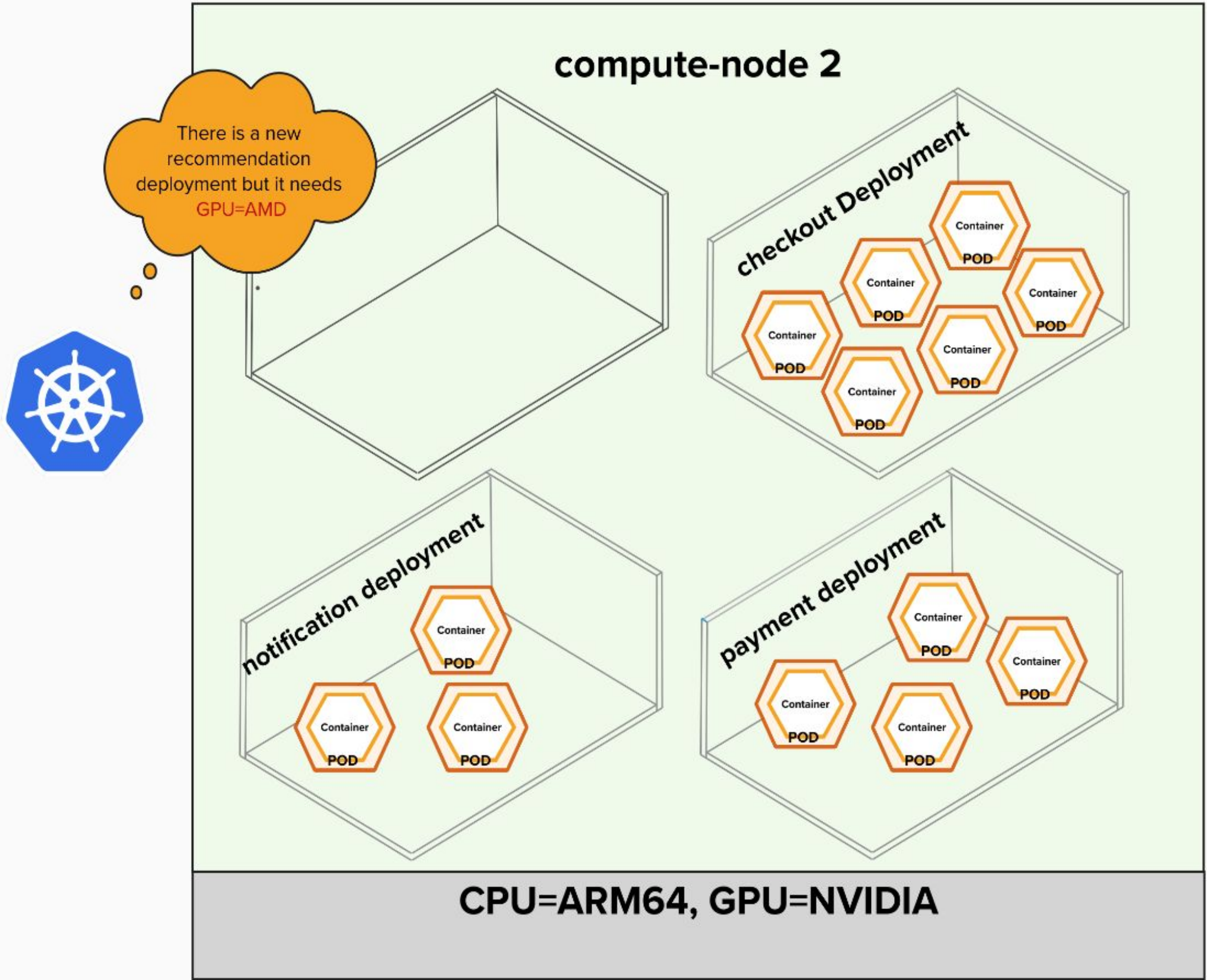
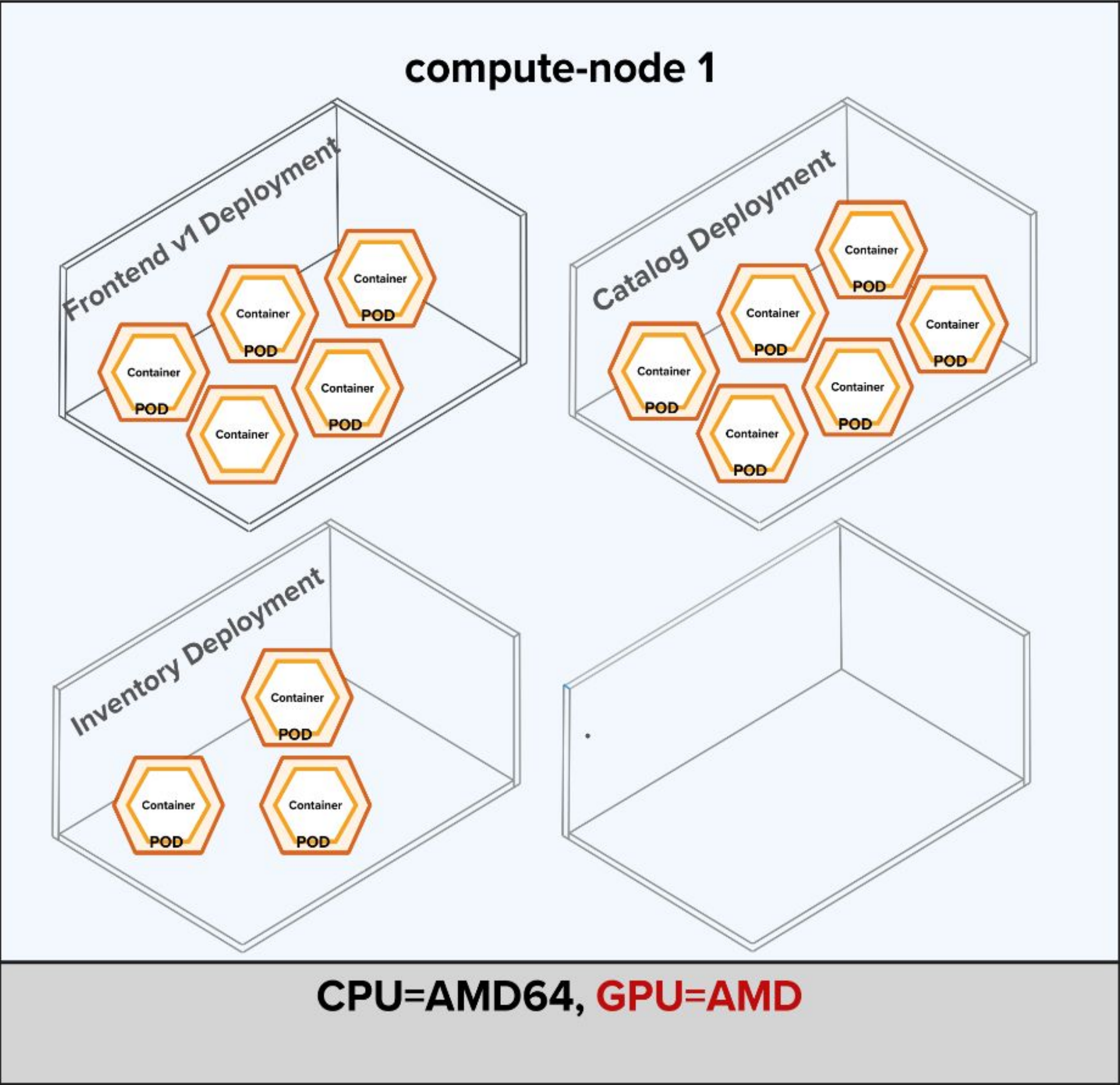
1. KUBERNETES



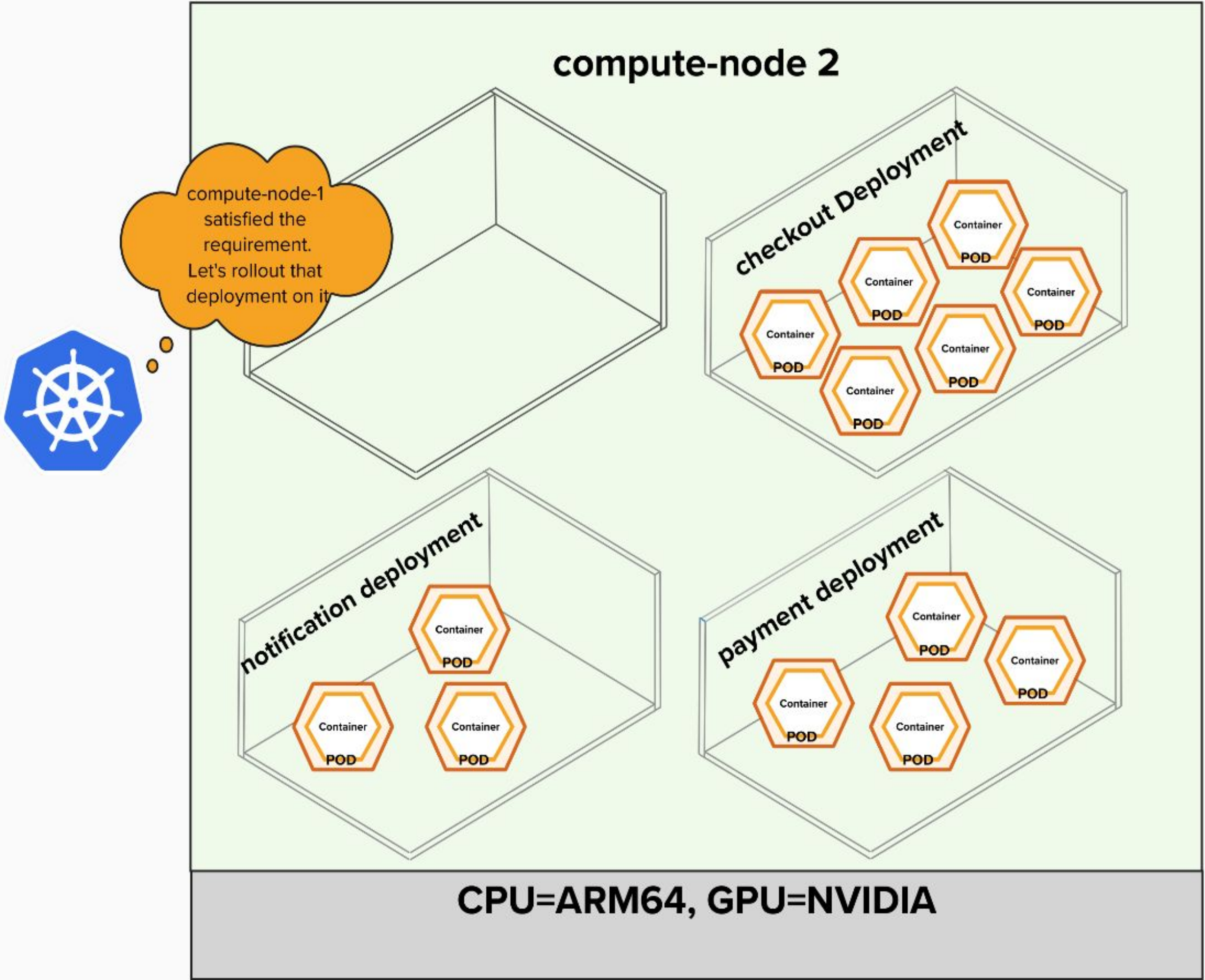
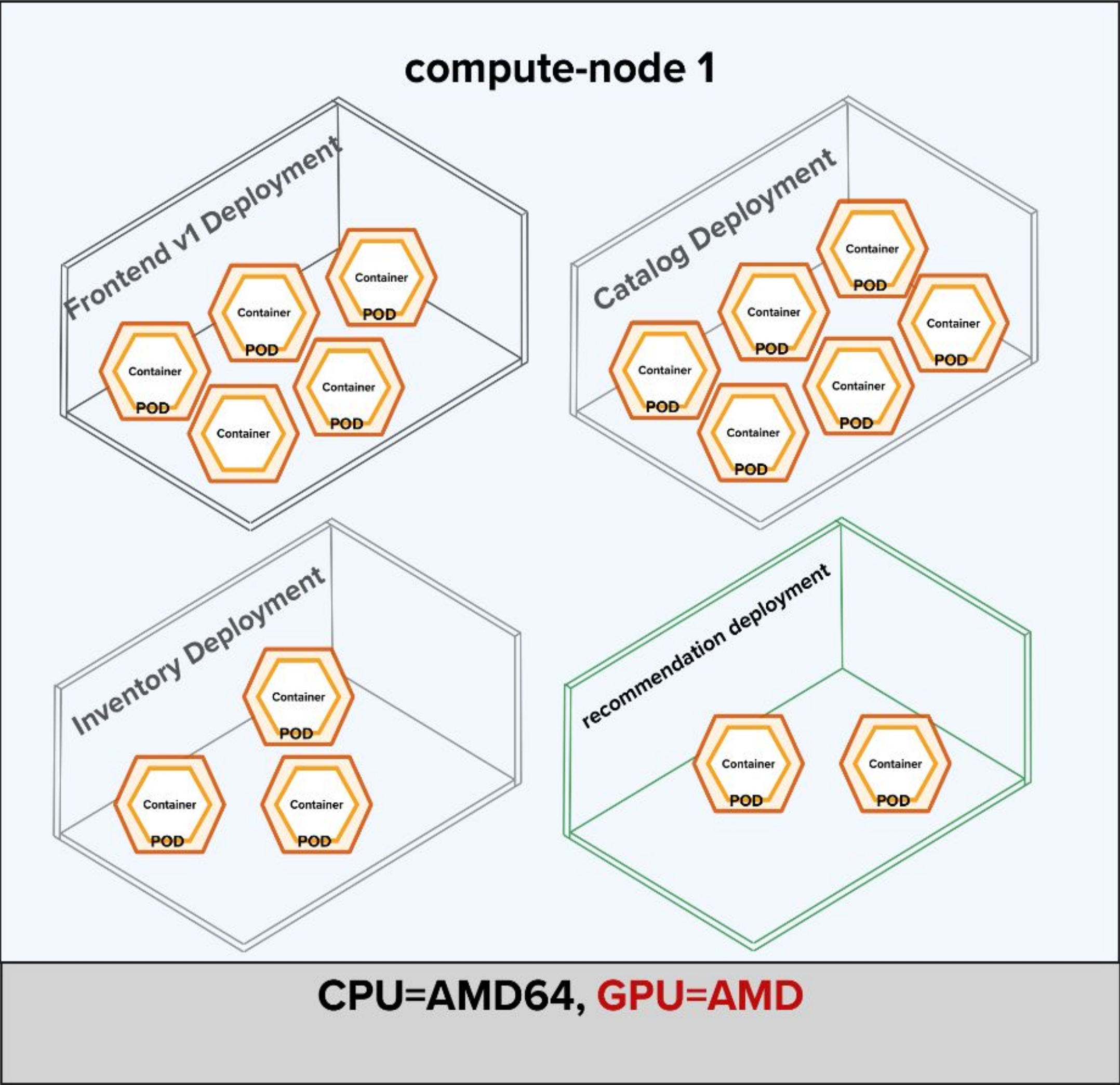
1. KUBERNETES



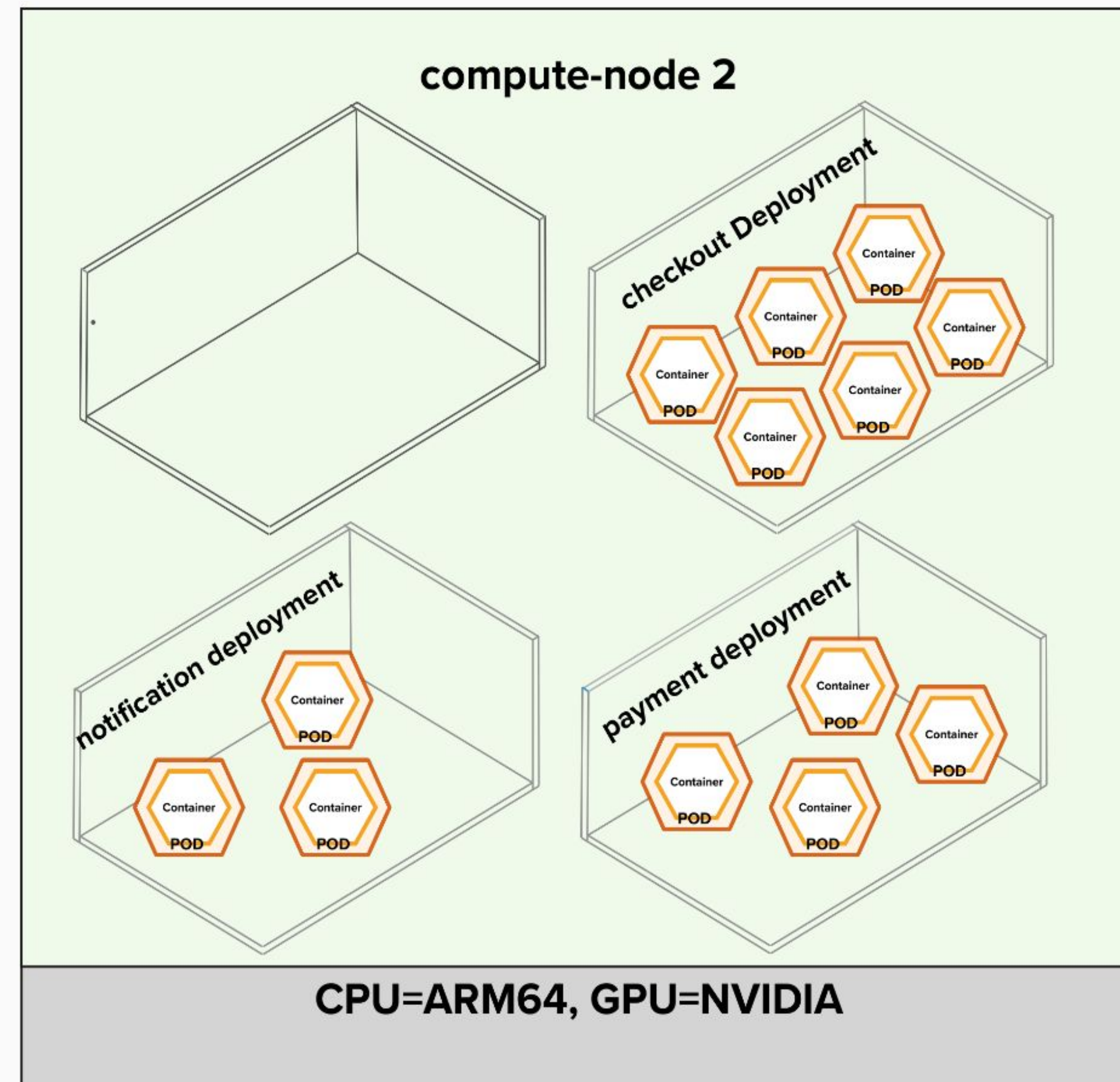
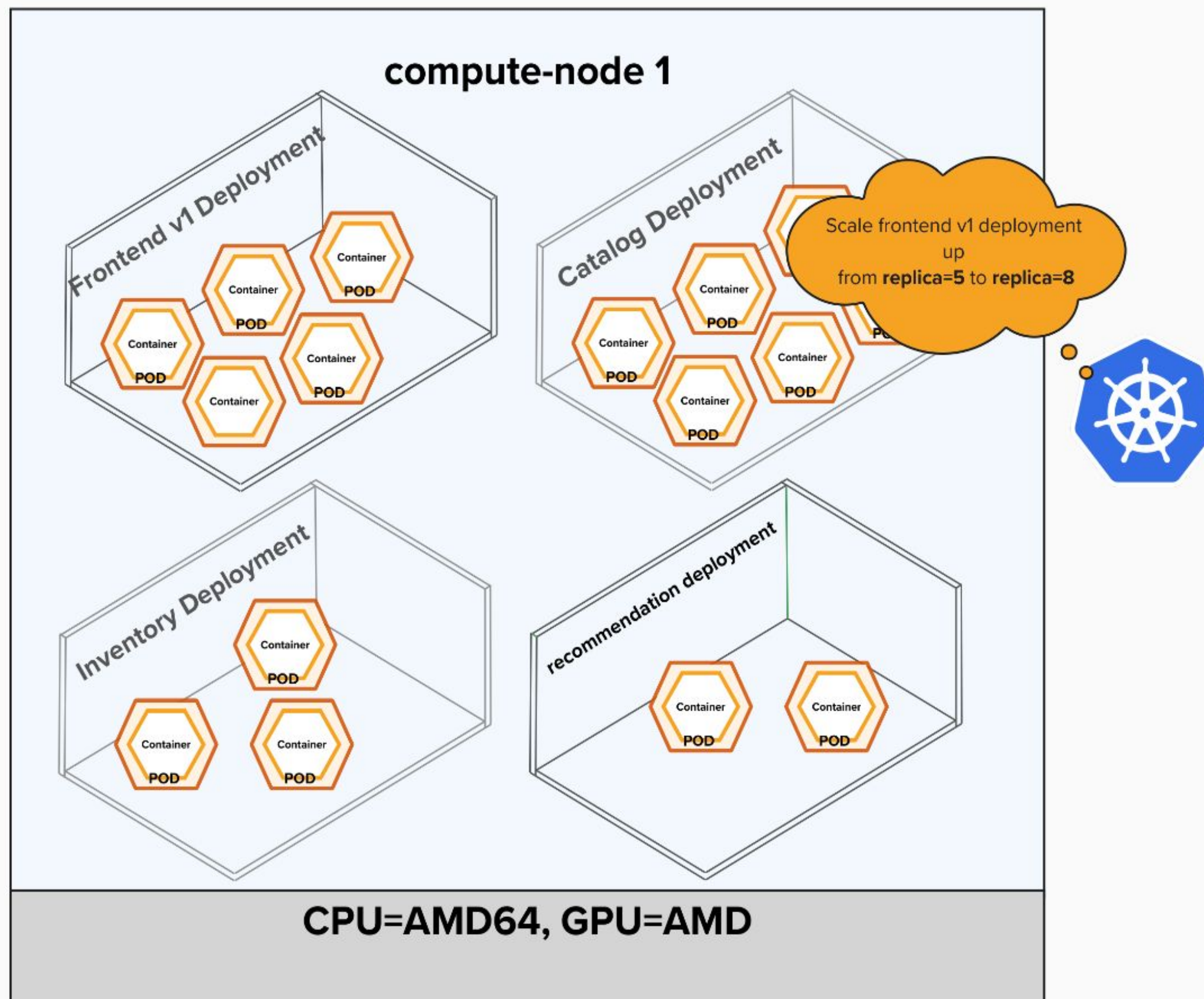
1. KUBERNETES



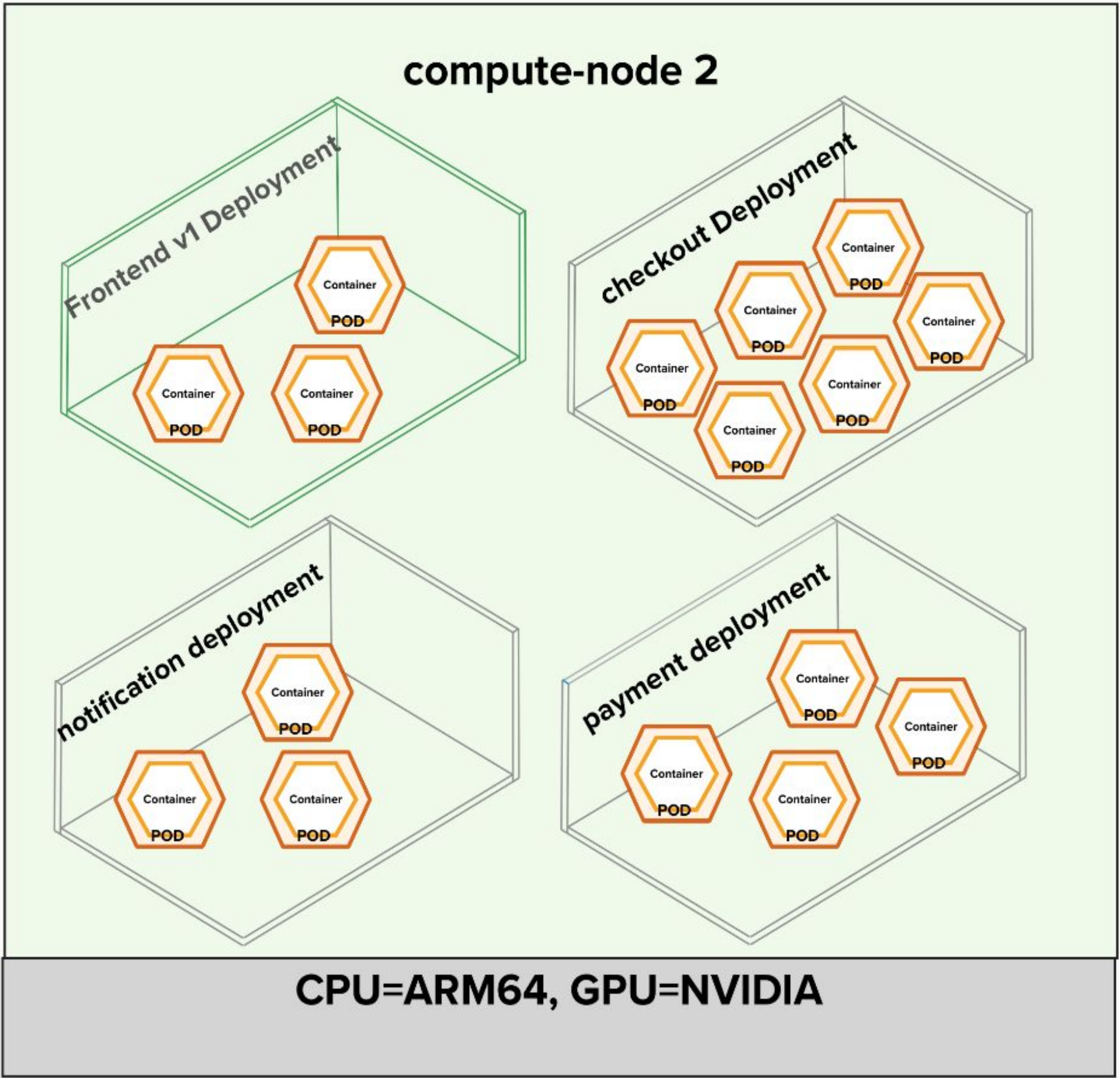
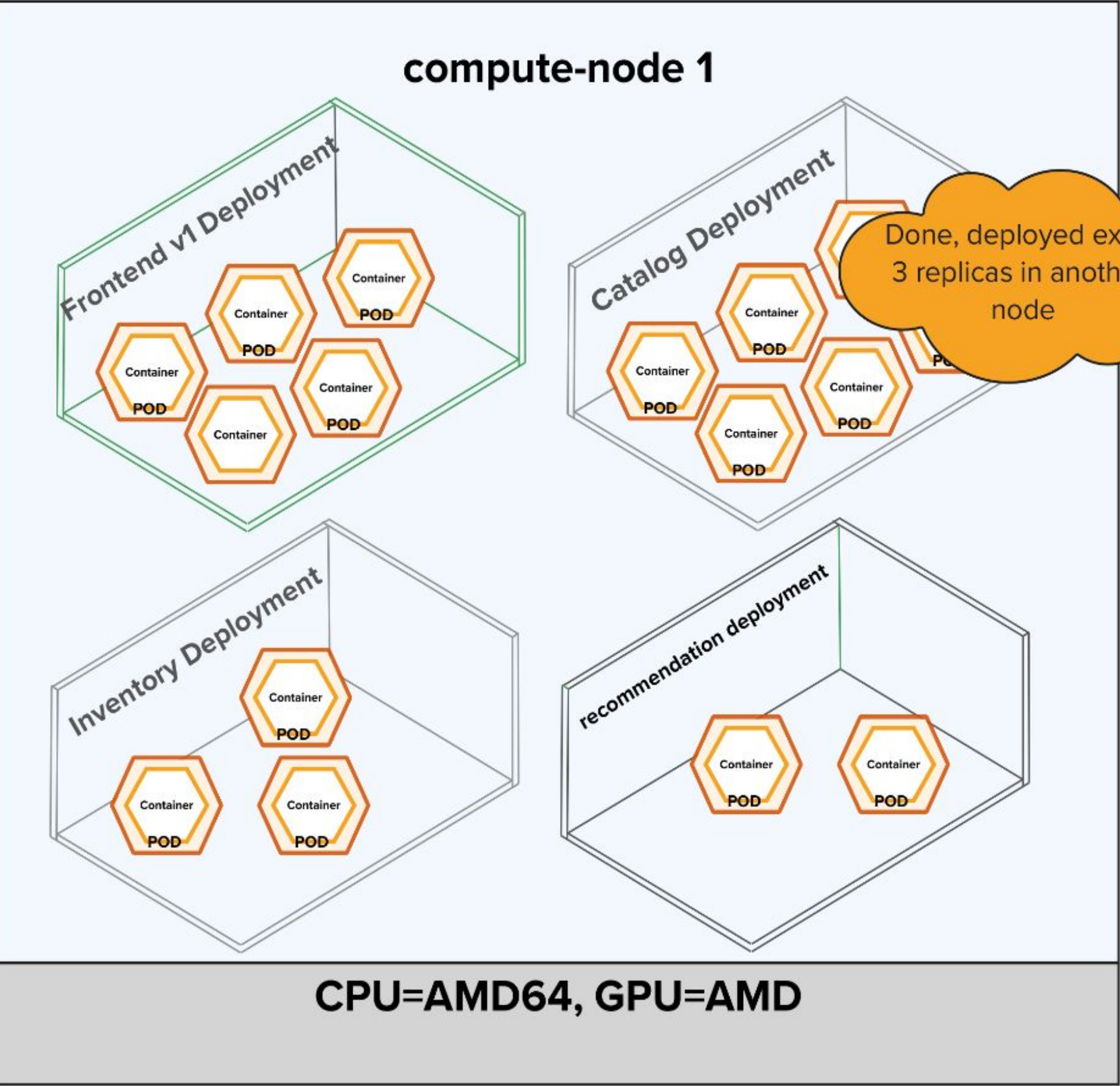
1. KUBERNETES



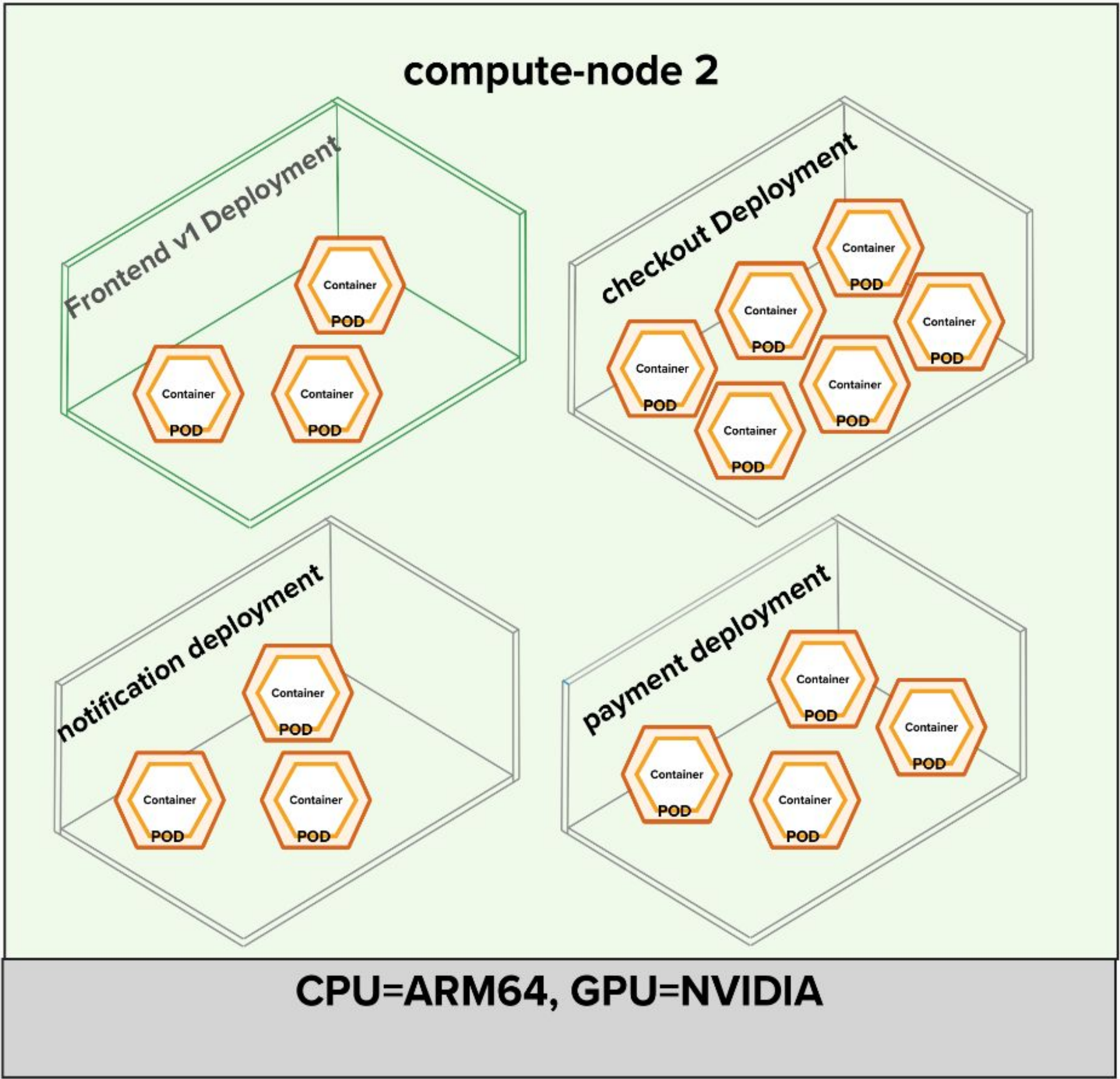
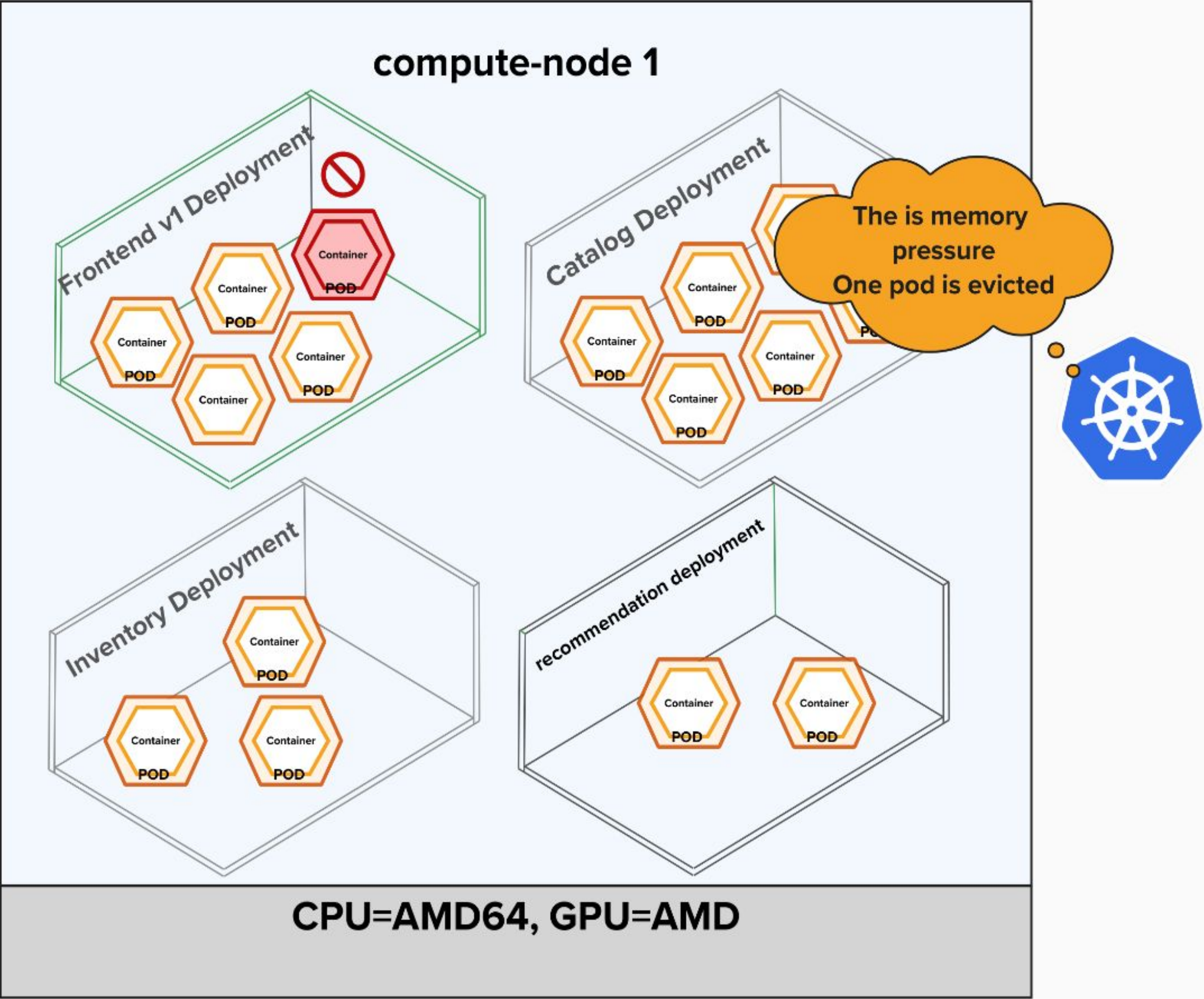
1. KUBERNETES



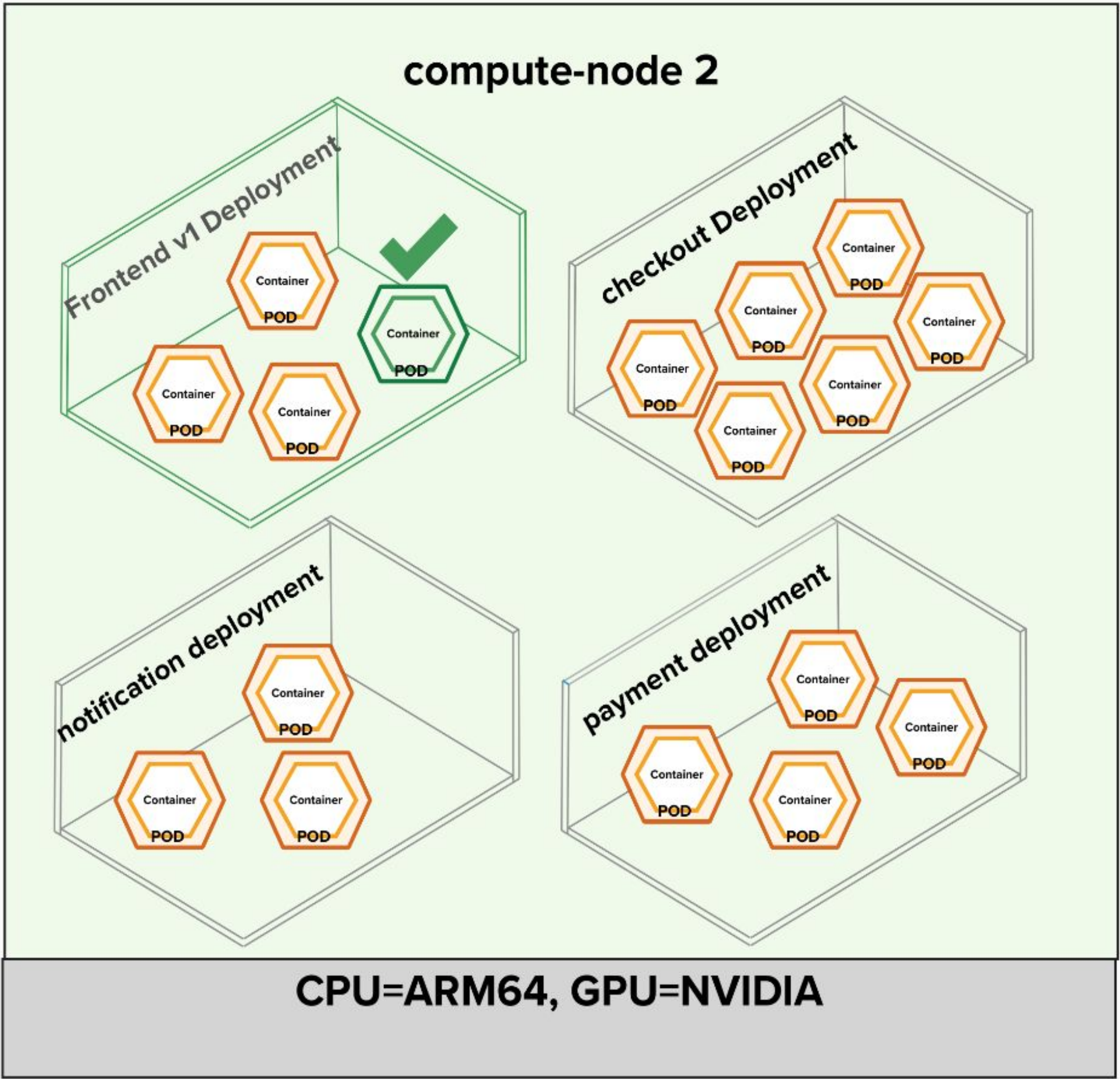
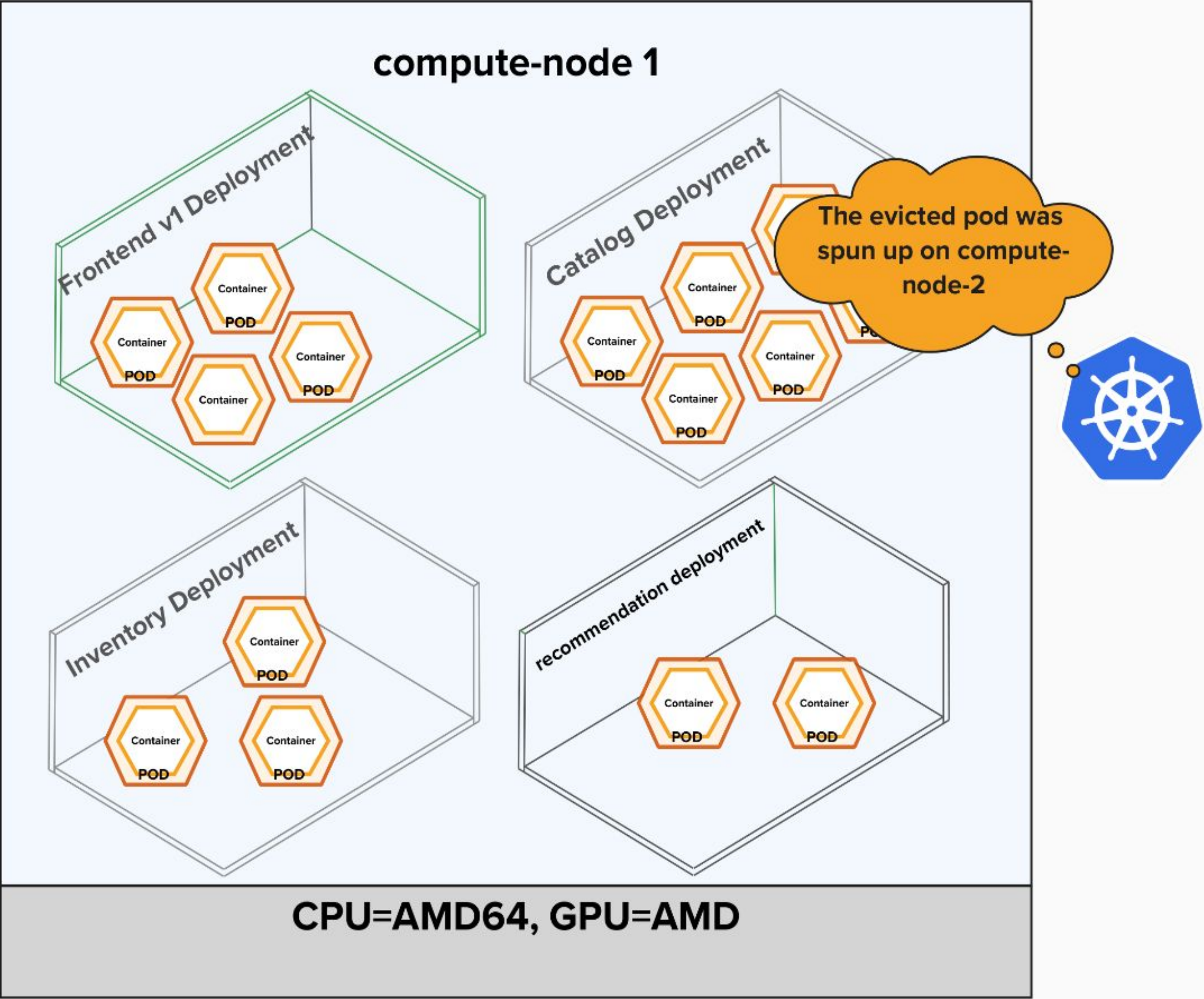
1. KUBERNETES



1. KUBERNETES



1. KUBERNETES



WHAT IS KUBERNETES

- Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.
- Created by three Google employees initially during the summer of 2014; grew exponentially and became the first project to get donated to the CNCF.
- Hit the first production-grade version v1.0.1 in July 2015. Has continually released a new minor version every three months since v1.2.0 in March 2016. Lately v1.13.0 was released in December 2018.

Decouples Infrastructure and Scaling

- **All services** within Kubernetes are natively Load Balanced.
- Can scale up and down dynamically.
- Used both to enable self-healing and seamless upgrading or rollback of applications.



Red Hat



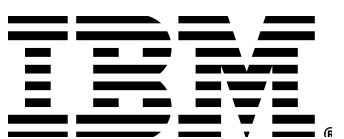
Self Healing

- Kubernetes will **ALWAYS** try and steer the cluster to its desired state.
- **Me:** “I want 3 healthy instances of redis to always be running.”
- **Kubernetes:** “Okay, I’ll ensure there are always 3 instances up and running.”
- **Kubernetes:** “Oh look, one has died. I’m going to attempt to spin up a new one.”

A Couple Key Concepts...

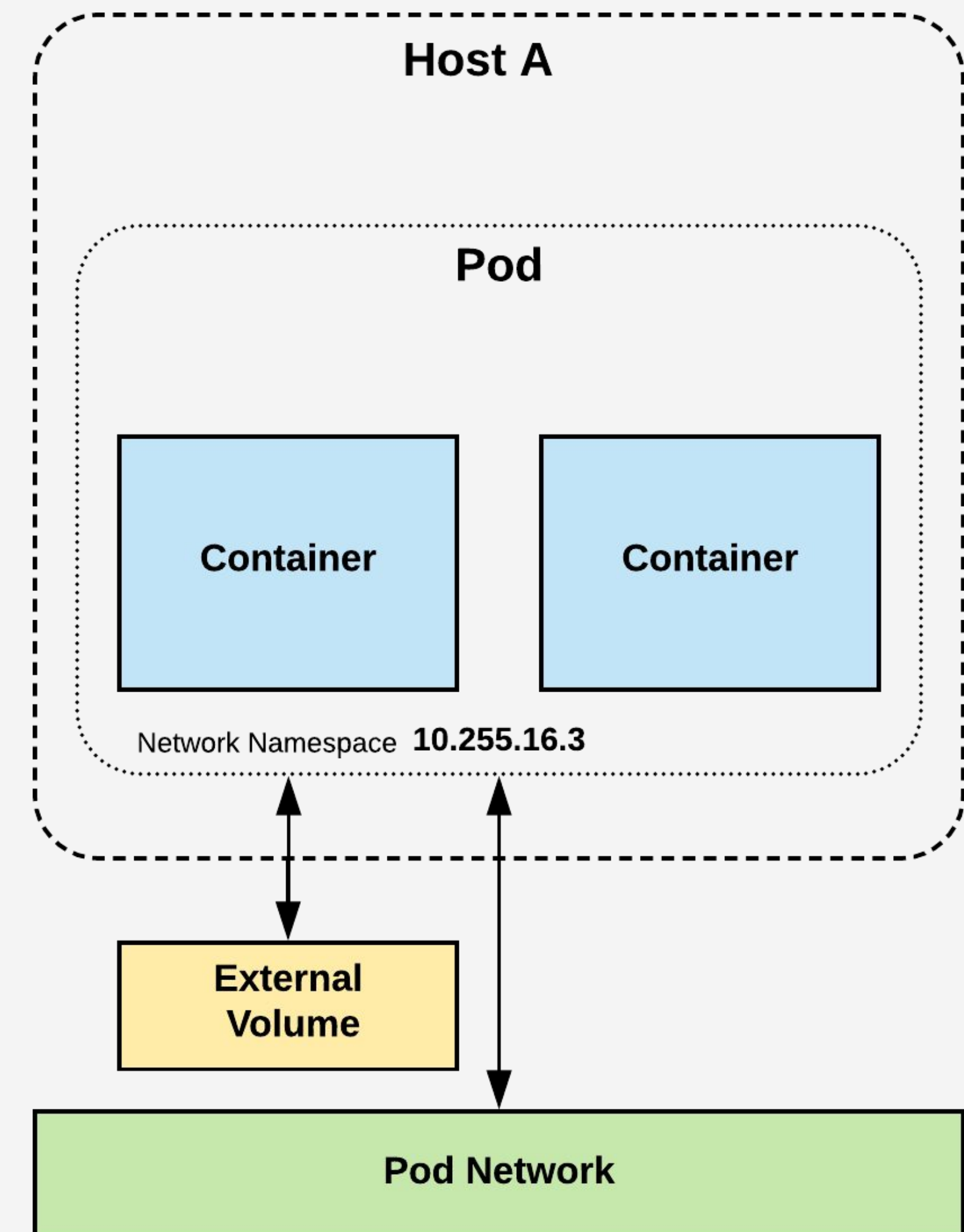


Red Hat



Pods

- **Atomic unit** or smallest “*unit of work*” of Kubernetes.
- Pods are **one or MORE containers** that share volumes and namespace.
- **They are also ephemeral!**

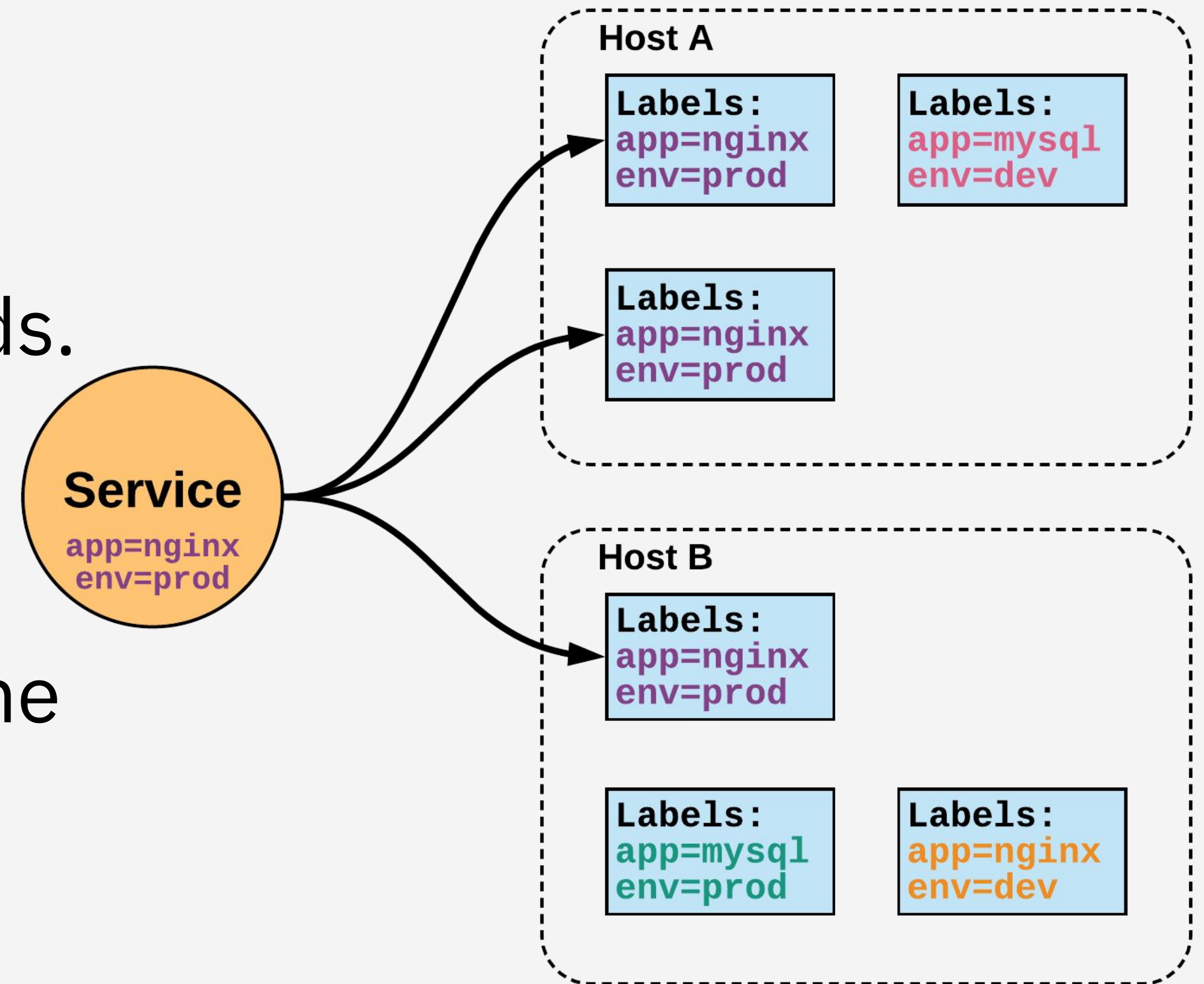


Red Hat



Services

- **Unified method of accessing** the exposed workloads of Pods.
- **Durable resource**
 - static cluster IP
 - static namespaced DNS name

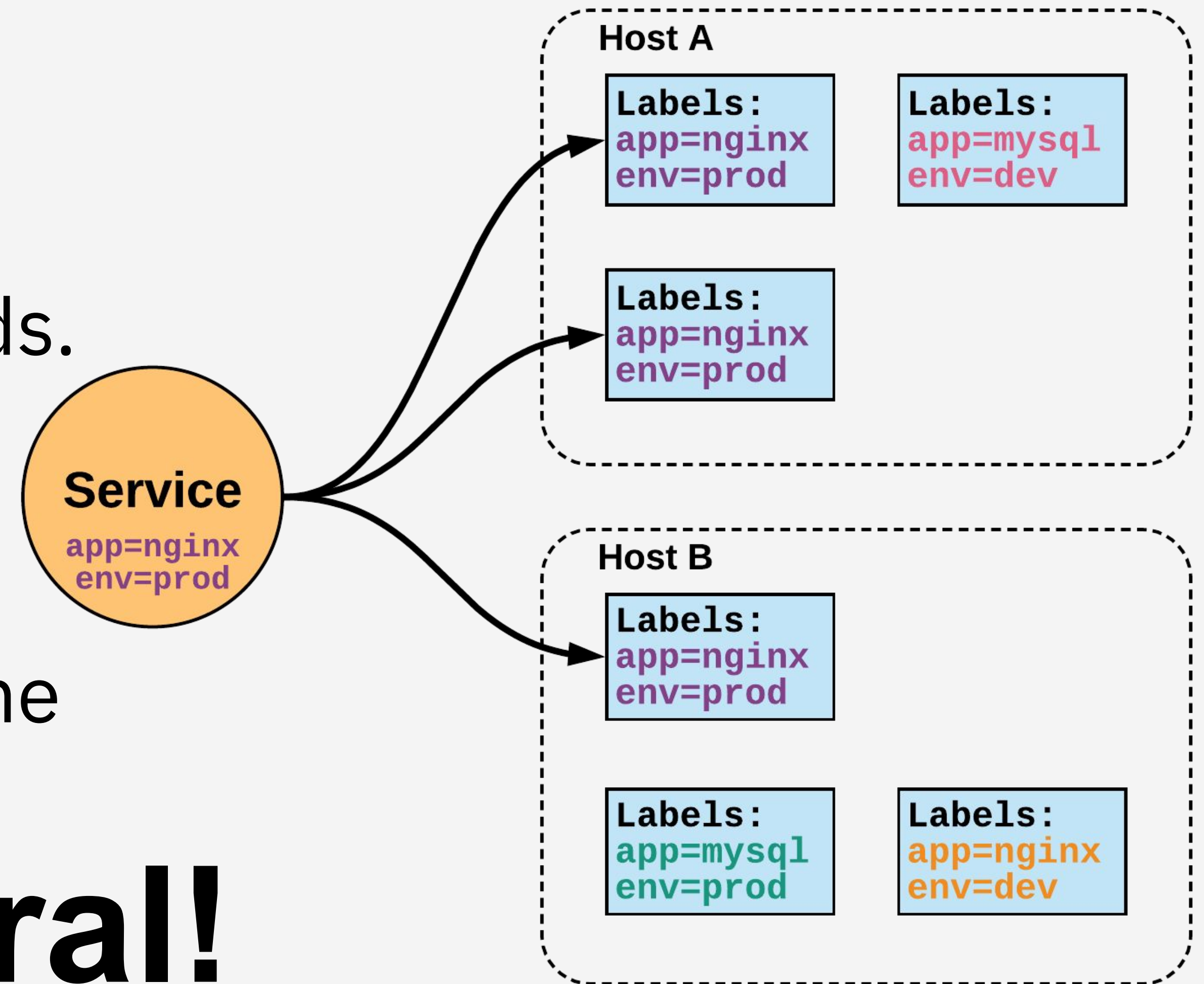


Red Hat



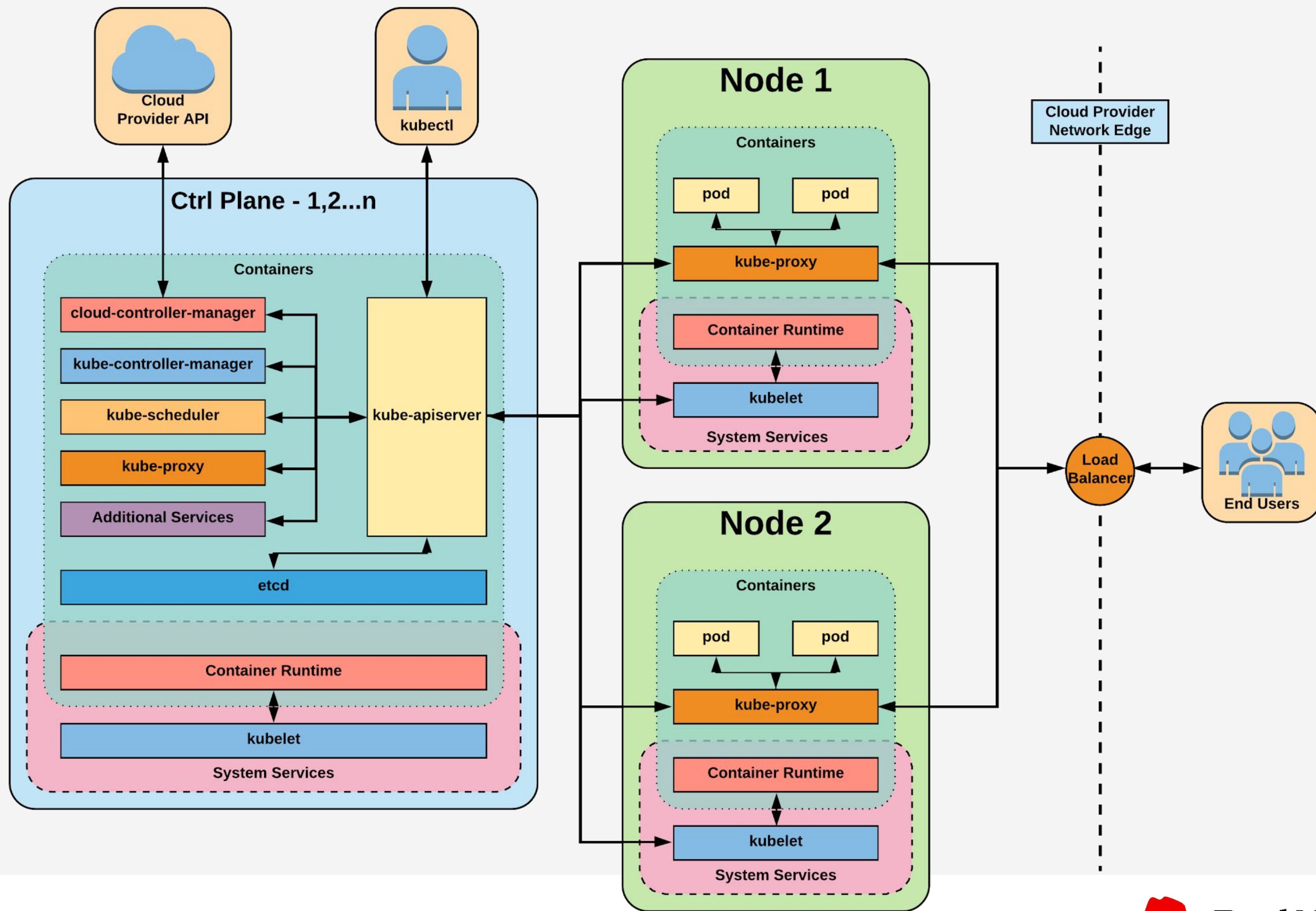
Services

- **Unified method of accessing** the exposed workloads of Pods.
- **Durable resource**
 - static cluster IP
 - static namespaced DNS name



NOT Ephemeral!

Architecture Overview

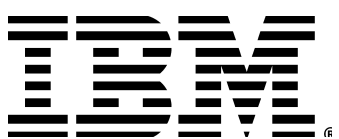


Control Plane Components

Architecture Overview

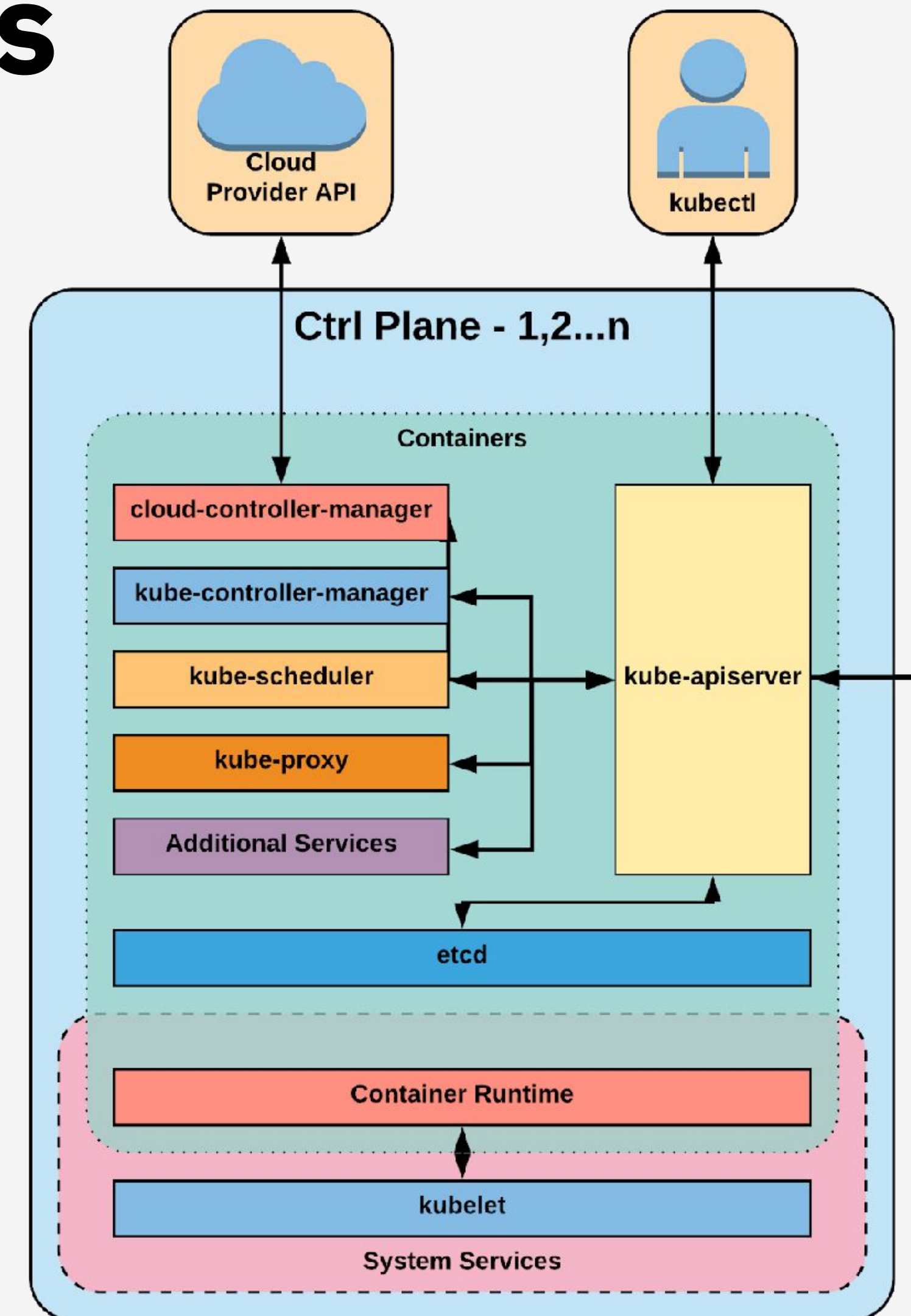


Red Hat



Control Plane Components

- kube-apiserver
- etcd
- kube-controller-manager
- kube-scheduler
- cloud-controller-manager



kube-apiserver

- Provides a forward facing REST interface into the kubernetes control plane and datastore.
- All clients and other applications interact with kubernetes strictly through the API Server.
- Acts as the gatekeeper to the cluster by handling authentication and authorization, request validation, mutation, and admission control in addition to being the front-end to the backing datastore.

etcd

- etcd acts as the cluster datastore.
- Purpose in relation to Kubernetes is to provide a strong, consistent and highly available key-value store for persisting cluster state.
- Stores objects and config information.

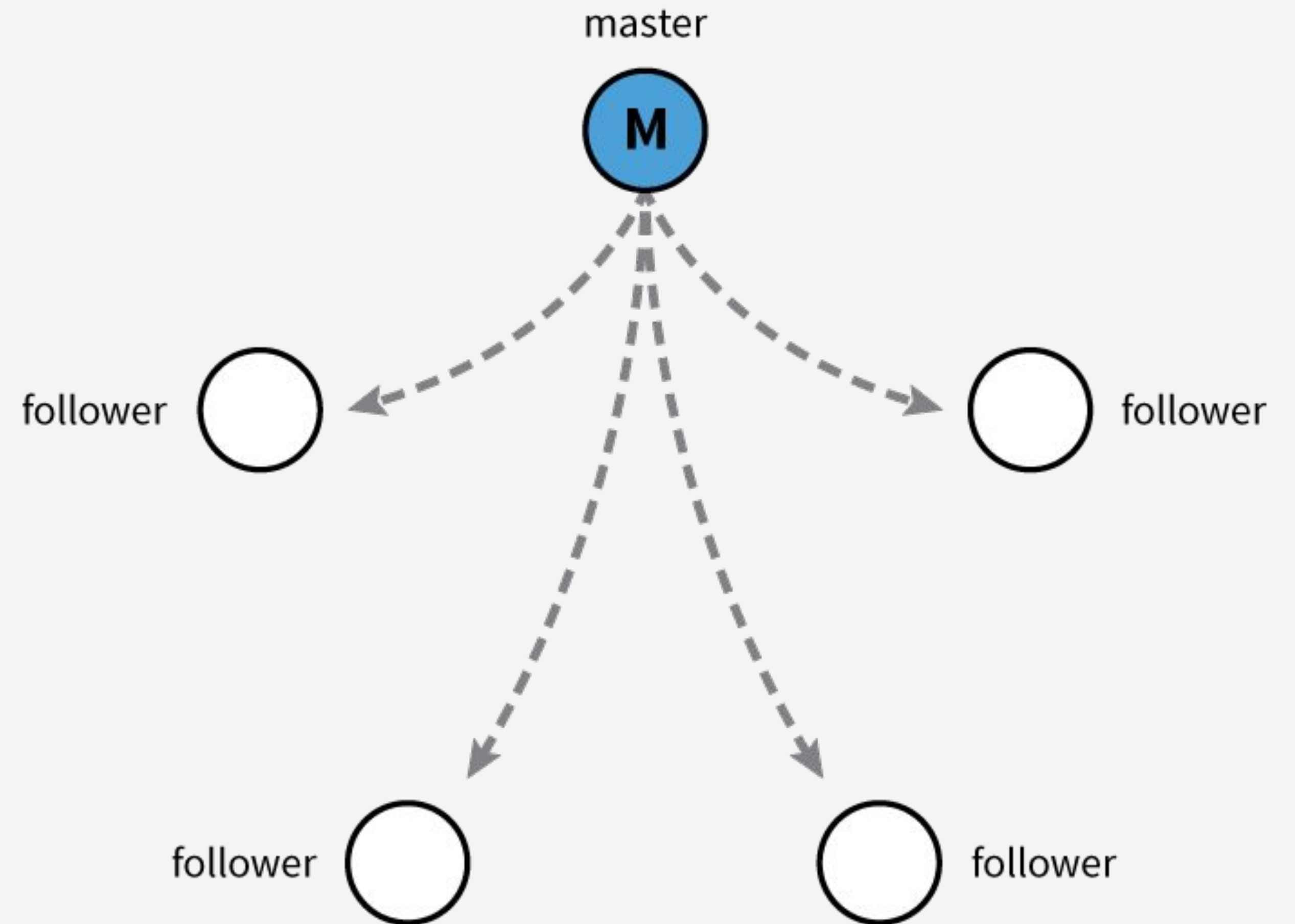


<https://github.com/etcd-io/etcd>

etcd

Uses “Raft Consensus”
among a quorum of
systems to create a
fault-tolerant consistent
“view” of the cluster.

<https://raft.github.io/>



Red Hat



kube-controller-manager

- Monitors the cluster state via the API server and **steers the cluster towards the desired state.**
- Node Controller: Responsible for noticing and responding when nodes go down.
- Replication Controller: Responsible for maintaining the correct number of pods for every replication controller object in the system.
- Endpoints Controller: Populates the Endpoints object (that is, joins Services & Pods).
- Service Account & Token Controllers: Create default accounts and API access tokens for new namespaces.

kube-scheduler

- Component on the master that watches newly created pods that have no node assigned, and selects a node for them to run on.
- Factors taken into account for scheduling decisions include individual and collective resource requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference and deadlines.

cloud-controller-manager

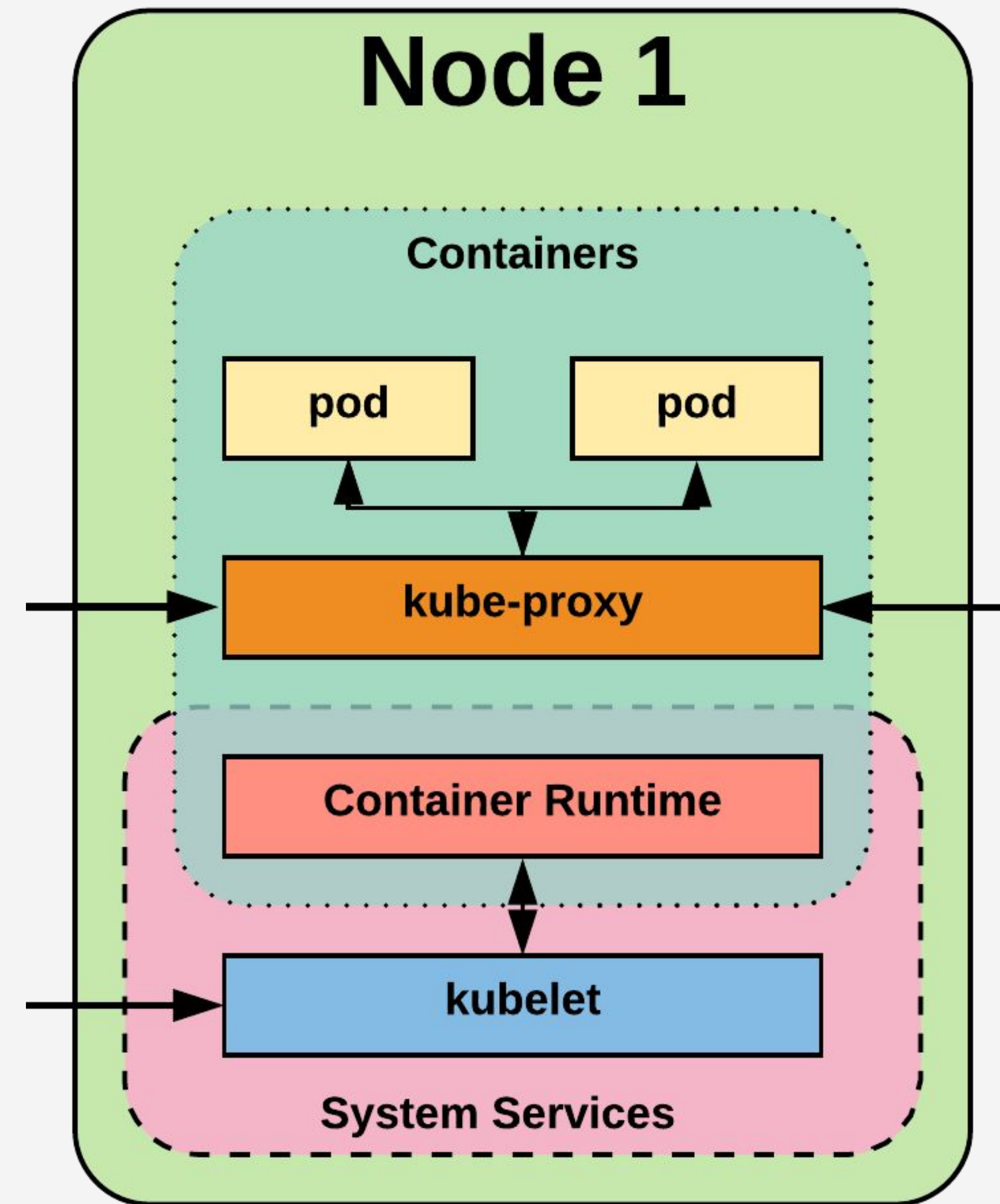
- Node Controller: For checking the cloud provider to determine if a node has been deleted in the cloud after it stops responding
- Route Controller: For setting up routes in the underlying cloud infrastructure
- Service Controller: For creating, updating and deleting cloud provider load balancers
- Volume Controller: For creating, attaching, and mounting volumes, and interacting with the cloud provider to orchestrate volumes

Node Components

Architecture Overview

Node Components

- kubelet
- kube-proxy
- Container Runtime Engine



Red Hat



kubelet

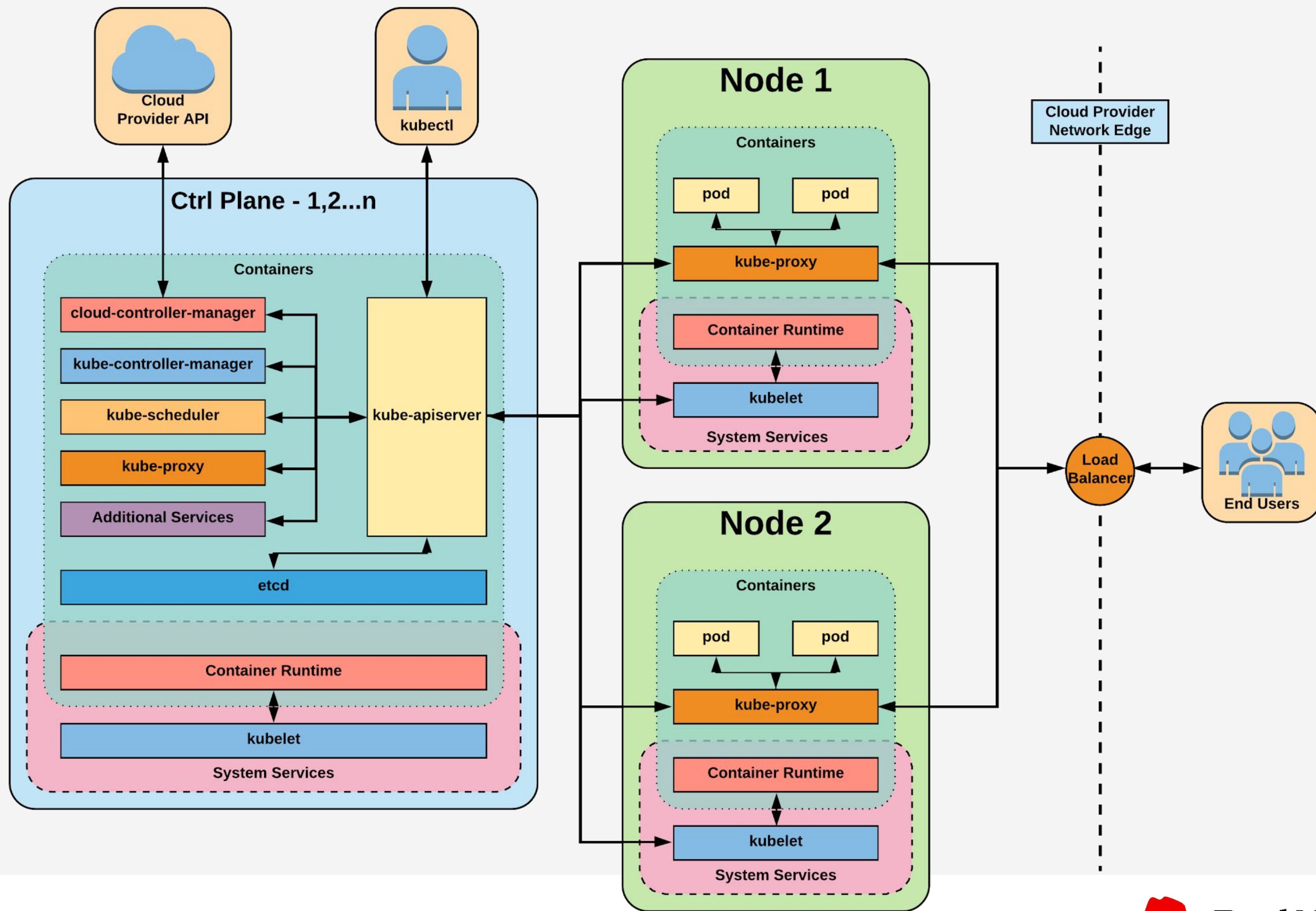
- An agent that runs on each node in the cluster. It makes sure that containers are running in a pod.
- The kubelet takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy.

kube-proxy

- Manages the network rules on each node.
- Performs connection forwarding or load balancing for Kubernetes cluster services.

Container Runtime Engine

- A container runtime is a CRI (Container Runtime Interface) compatible application that executes and manages containers.
 - Containerd (docker)
 - Cri-o
 - Rkt
 - Kata (formerly clear and hyper)
 - Virtlet (VM CRI compatible runtime)



Networking

Architecture Overview

Fundamental Networking Rules

- All containers within a pod can communicate with each other unimpeded.
- All Pods can communicate with all other Pods without NAT.
- All nodes can communicate with all Pods (and vice-versa) without NAT.
- The IP that a Pod sees itself as is the same IP that others see it as.

Fundamentals Applied

- **Container-to-Container**

- Containers within a pod exist within the **same network namespace** and share an IP.
- Enables intrapod communication over localhost.

- **Pod-to-Pod**

- Allocated **cluster unique IP** for the duration of its life cycle.
- Pods themselves are fundamentally ephemeral.



Red Hat



Fundamentals Applied

- **Pod-to-Service**

- managed by **kube-proxy** and given a **persistent cluster unique IP**
- exists beyond a Pod's lifecycle.

- **External-to-Service**

- Handled by **kube-proxy**.
- Works in cooperation with a cloud provider or other external entity (load balancer).

Core Objects and API

- **Namespaces**
- **Pods**
- **Labels**
- **Selectors**
- **Services**

Architecture Overview



Red Hat



Namespaces

Namespaces are a logical cluster or environment, and are the primary method of partitioning a cluster or scoping access.

```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    app: MyBigWebApp
```

```
$ kubectl get ns --show-labels
NAME          STATUS    AGE      LABELS
default       Active    11h      <none>
kube-public   Active    11h      <none>
kube-system   Active    11h      <none>
prod          Active    6s       app=MyBigWebApp
```


Pod Examples

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
```

Key Pod Container Attributes

- **name** - The name of the container
- **image** - The container image
- **ports** - array of ports to expose.
Can be granted a friendly name and protocol may be specified
- **env** - array of environment variables
- **command** - Entrypoint array (equiv to Docker **ENTRYPOINT**)
- **args** - Arguments to pass to the command (equiv to Docker **CMD**)

Container

```
name: nginx
image: nginx:stable-alpine
ports:
  - containerPort: 80
    name: http
    protocol: TCP
env:
  - name: MYVAR
    value: isAwesome
command: ["/bin/sh", "-c"]
args: ["echo ${MYVAR}"]
```



Red Hat



Key Pod Container Attributes

- **name** - The name of the container
- **image** - The container image
- **ports** - array of ports to expose.
Can be granted a friendly name and protocol may be specified
- **env** - array of environment variables
- **command** - Entrypoint array (equiv to Docker **ENTRYPOINT**)
- **args** - Arguments to pass to the command (equiv to Docker **CMD**)

Container

```
name: nginx
image: nginx:stable-alpine
ports:
  - containerPort: 80
    name: http
    protocol: TCP
env:
  - name: MYVAR
    value: isAwesome
command: ["/bin/sh", "-c"]
args: ["echo ${MYVAR}"]
```

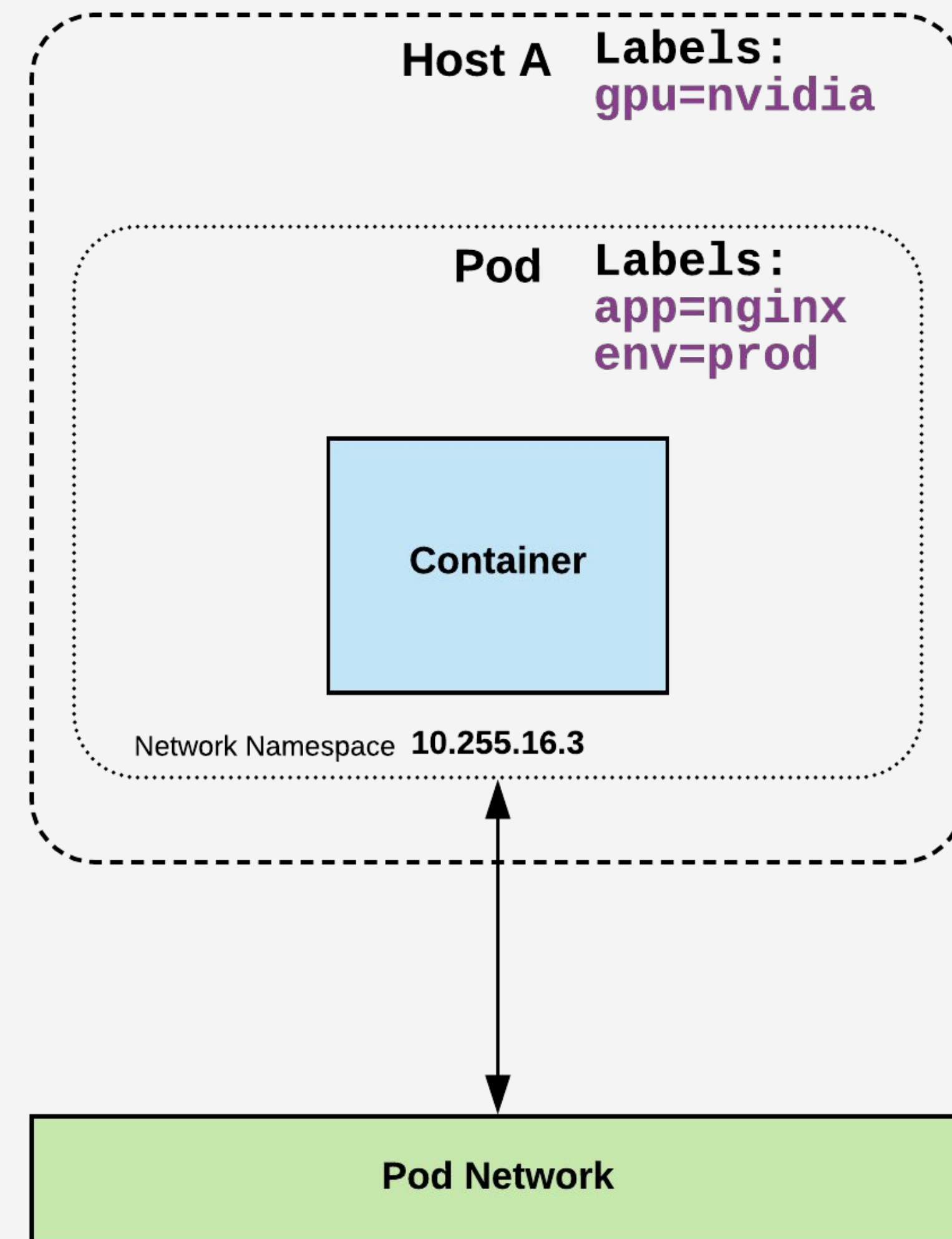


Red Hat



Labels

- key-value pairs that are used to identify, describe and group together related sets of objects or resources.
- **NOT** characteristic of uniqueness.
- Have a strict syntax with a slightly limited character set*.



Red Hat



Resource model

- **Request:** amount of a resource allowed to be used, with a strong guarantee of availability
 - CPU (seconds/second), RAM (bytes)
 - Scheduler will not over-commit requests
- **Limit:** max amount of a resource that can be used, regardless of guarantees
 - scheduler ignores limits

```
apiVersion: v1
kind: Pod
metadata:
  name: cpu-demo
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        cpu: "250m"
        memory: "64Mi"
      limits:
        cpu: "500m"
        memory: "128Mi"
```



Red Hat



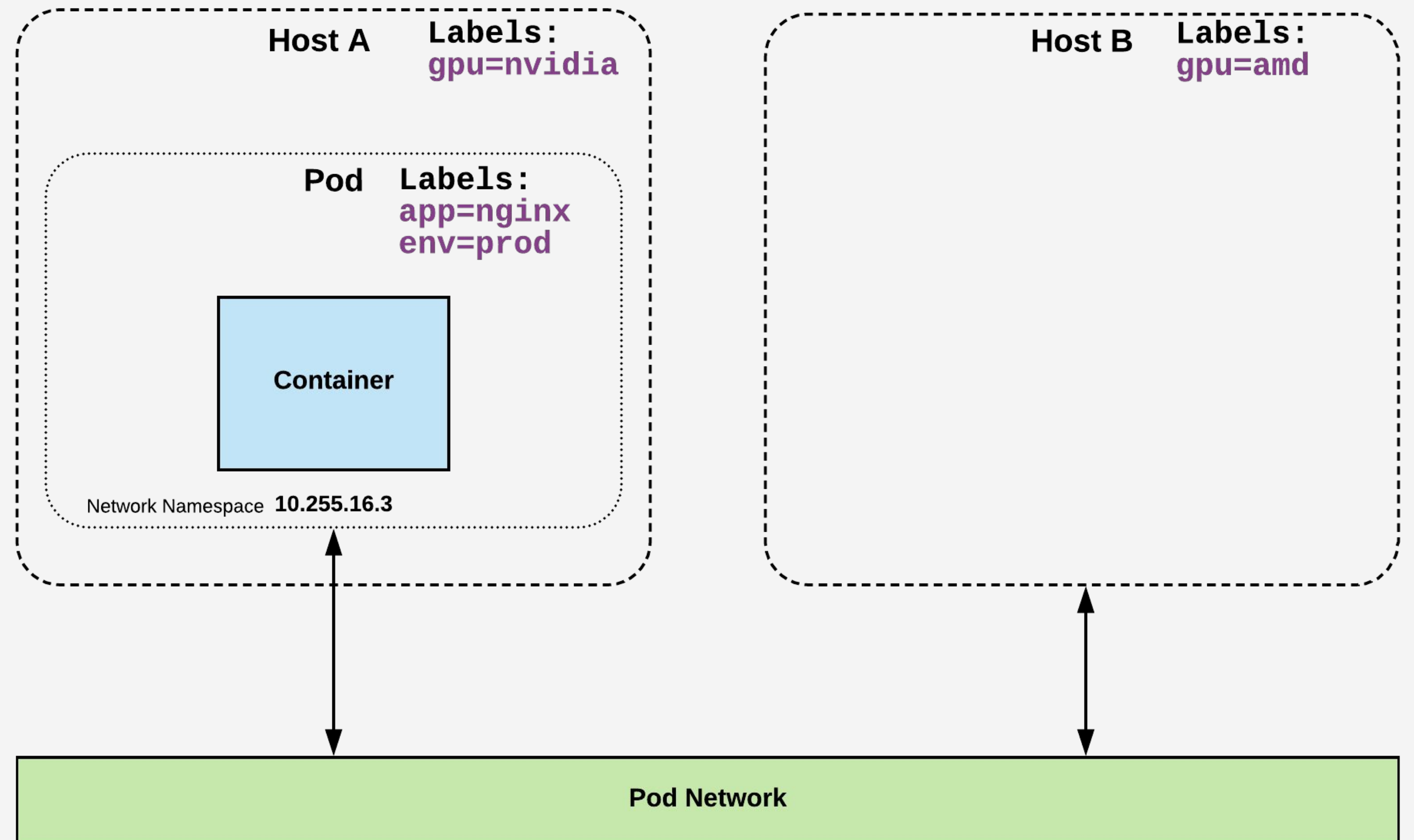
Selectors

Selectors use labels to filter or select objects, and are used throughout Kubernetes.

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-label-example
  labels:
    app: nginx
    env: prod
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
  nodeSelector:
    gpu: nvidia
```


Selector Example

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-label-example
  labels:
    app: nginx
    env: prod
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
  nodeSelector:
    gpu: nvidia
```



Selector Types

Equality based selectors allow for simple filtering (=, ==, or !=).

```
selector:  
  matchLabels:  
    gpu: nvidia
```

Set-based selectors are supported on a limited subset of objects. However, they provide a method of filtering on a set of values, and supports multiple operators including: **in**, **notin**, and **exist**.

```
selector:  
  matchExpressions:  
    - key: gpu  
      operator: in  
      values: ["nvidia"]
```



Red Hat



Services

- **Unified method of accessing** the exposed workloads of Pods.
- **Durable resource** (unlike Pods)
 - static cluster-unique IP
 - static namespaced DNS name

<service name>.<namespace>.svc.cluster.local

Services

- Target Pods using **equality based selectors**.
- Uses **kube-proxy** to provide simple load-balancing.
- **kube-proxy** acts as a daemon that creates **local entries** in the host's iptables for every service.

Service Types

There are 4 major service types:

- ClusterIP (default)
- NodePort
- LoadBalancer
- ExternalName

ClusterIP Service

ClusterIP services exposes a service on a strictly cluster internal virtual IP.

```
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  selector:
    app: nginx
    env: prod
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```



Red Hat

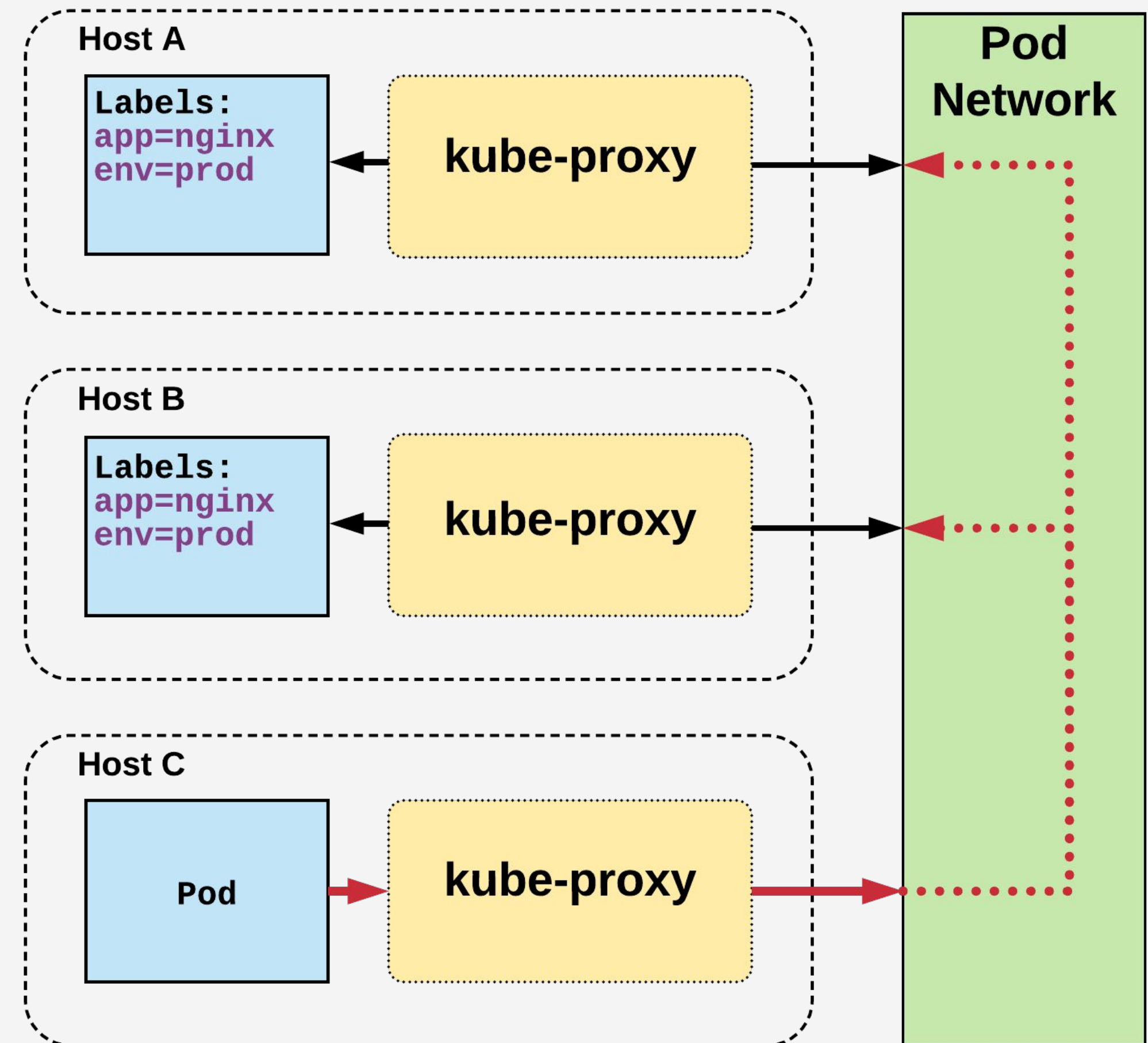


Cluster IP Service

Name: example-prod
Selector: app=nginx,env=prod
Type: ClusterIP
IP: 10.96.28.176
Port: <unset> 80/TCP
TargetPort: 80/TCP
Endpoints: 10.255.16.3:80,
10.255.16.4:80

```
/ # nslookup example-prod.default.svc.cluster.local
```

Name: example-prod.default.svc.cluster.local
Address 1: 10.96.28.176 example-prod.default.svc.cluster.local



ClusterIP Service Without Selector

```
kind: Service
apiVersion: v1
metadata:
  name: my-service
spec:
  ports:
  - protocol: TCP
    port: 80
    targetPort: 9376
```

```
kind: Endpoints
apiVersion: v1
metadata:
  name: my-service
subsets:
  - addresses:
    - ip: 1.2.3.4
    ports:
    - port: 9376
```



Red Hat



NodePort Service

- **NodePort** services extend the **ClusterIP** service.
- Exposes a port on every node's IP.
- Port can either be statically defined, or dynamically taken from a range between 30000-32767.

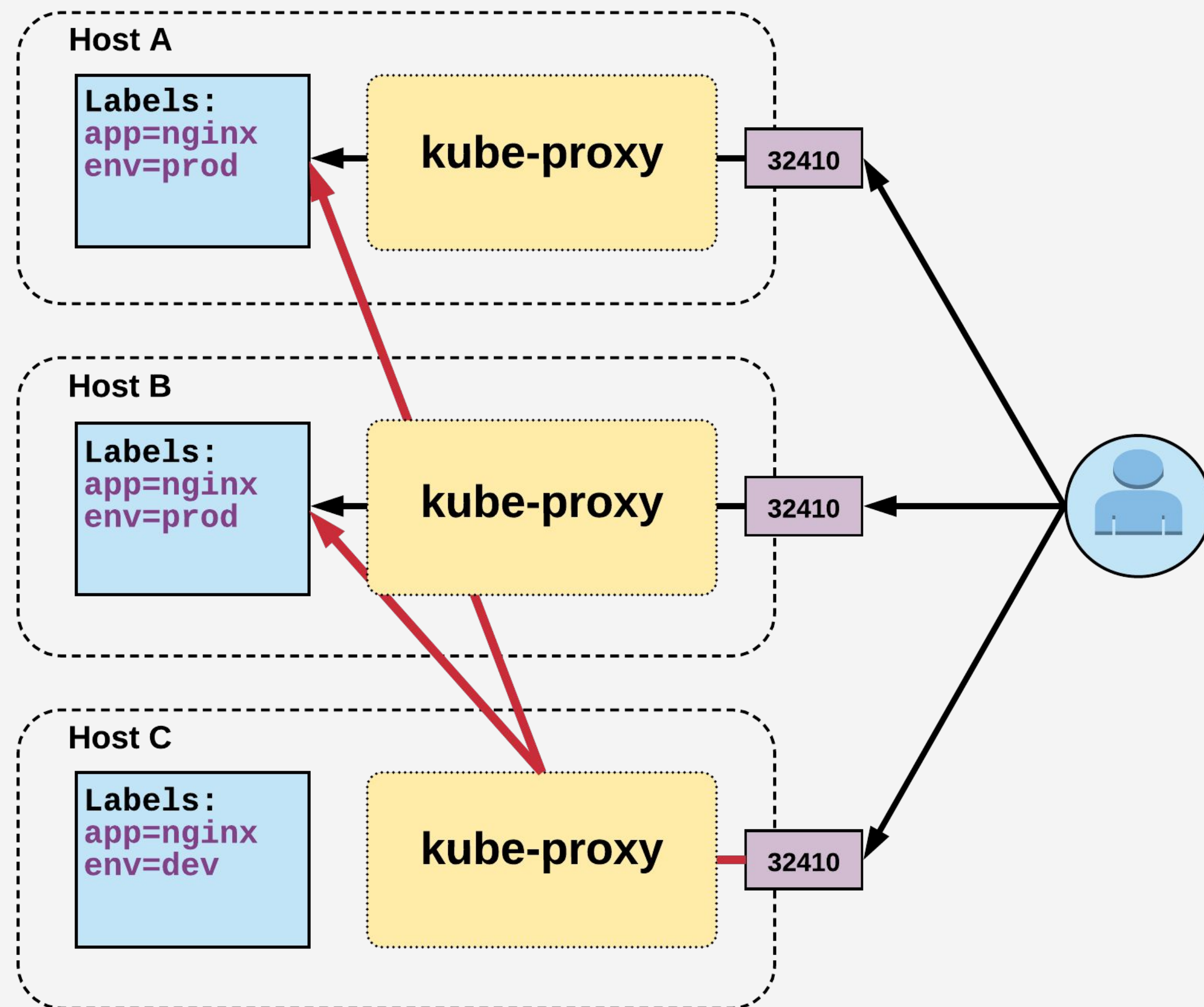
```
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  type: NodePort
  selector:
    app: nginx
    env: prod
  ports:
    - nodePort: 32410
      protocol: TCP
      port: 80
      targetPort: 80
```



Red Hat



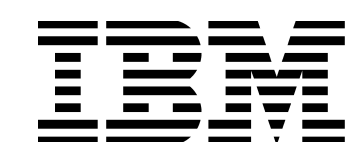
NodePort Service



Name:	example-prod
Selector:	app=nginx,env=prod
Type:	NodePort
IP:	10.96.28.176
Port:	<unset> 80/TCP
TargetPort:	80/TCP
NodePort:	<unset> 32410/TCP
Endpoints:	10.255.16.3:80, 10.255.16.4:80



Red Hat



LoadBalancer Service

- **LoadBalancer** services extend **NodePort**.
- Works in conjunction with an external system to map a cluster external IP to the exposed service.

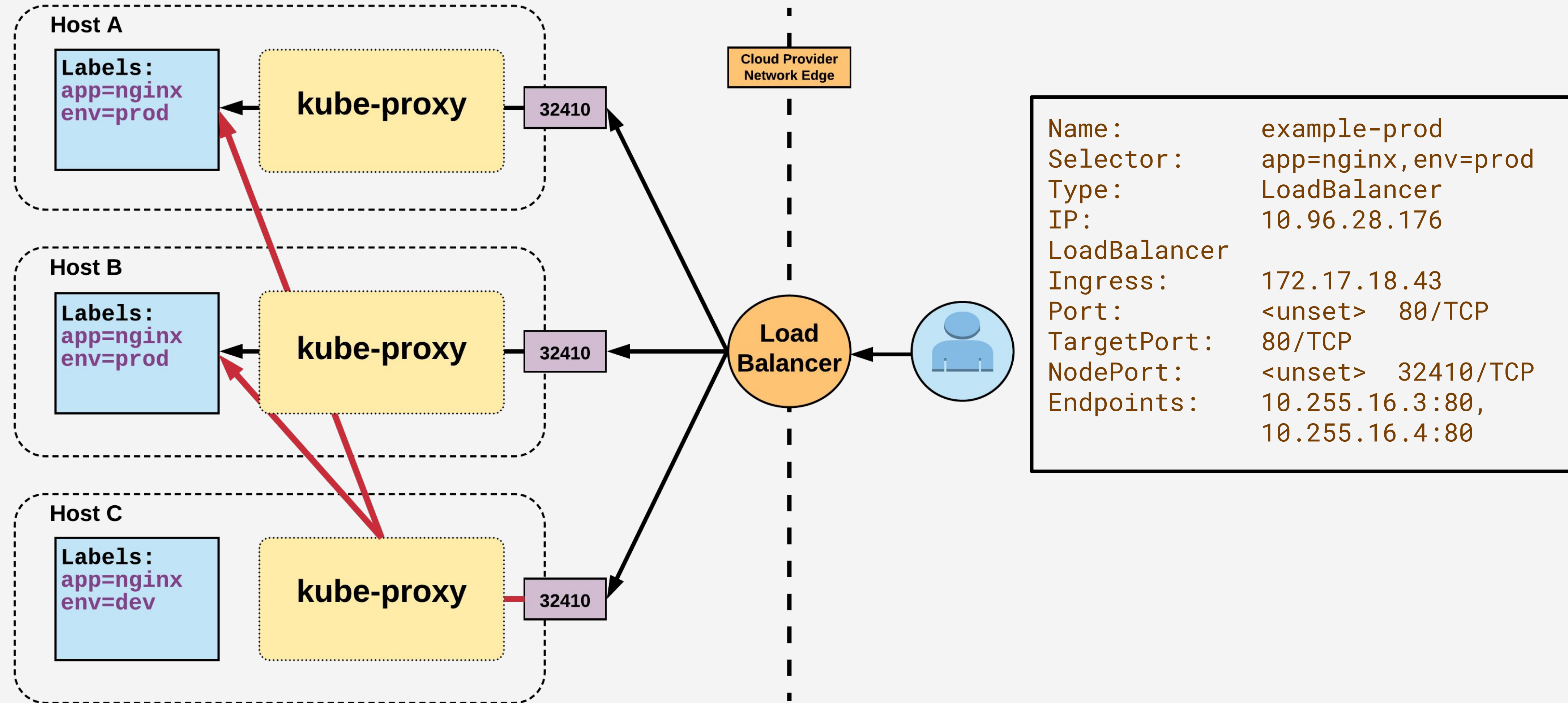
```
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  type: LoadBalancer
  selector:
    app: nginx
    env: prod
  ports:
    protocol: TCP
    port: 80
    targetPort: 80
```



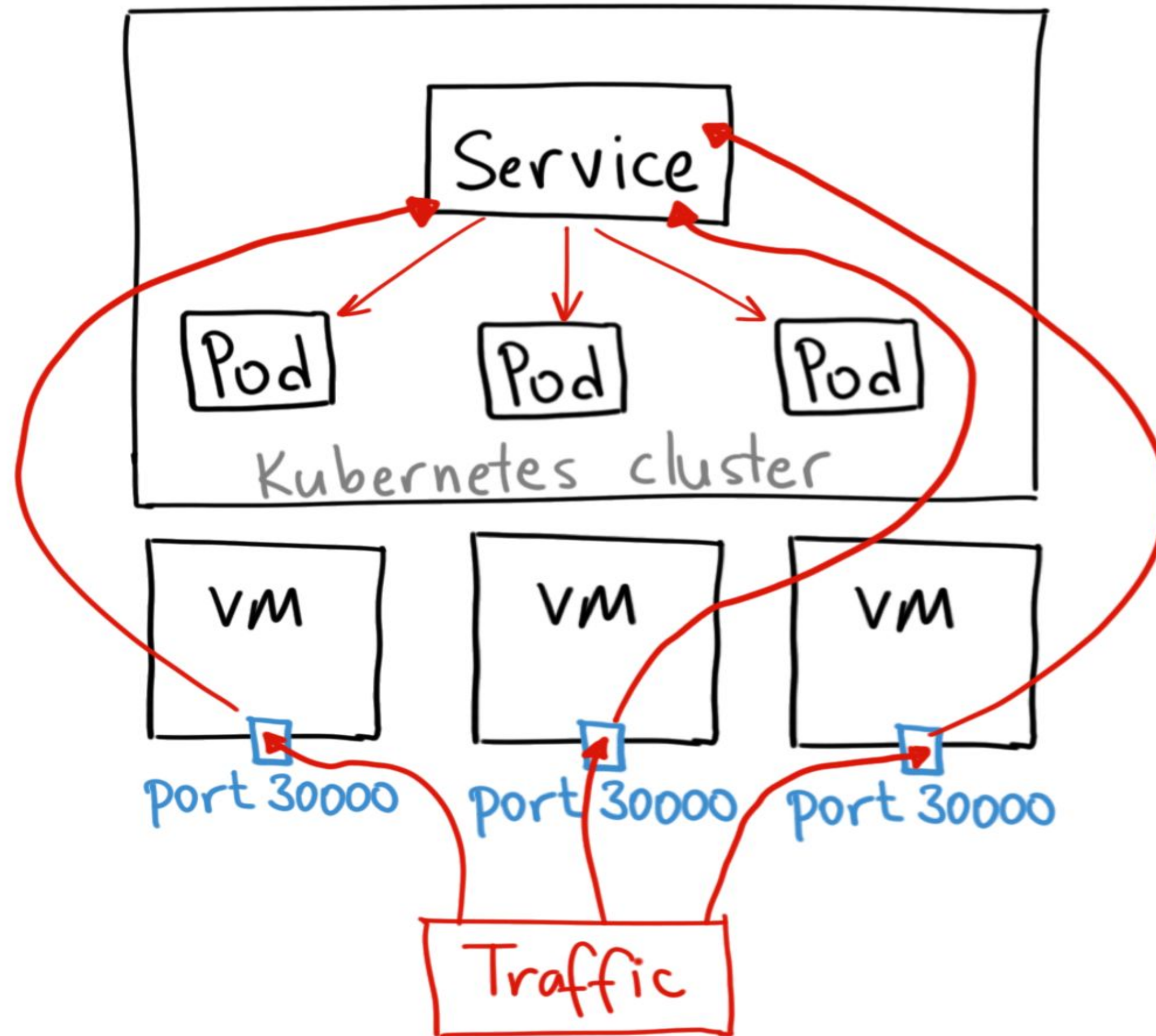
Red Hat



LoadBalancer Service



LoadBalancer Service



ExternalName Service

- **ExternalName** is used to reference endpoints **OUTSIDE** the cluster.
- Creates an internal **CNAME** DNS entry that aliases another.

```
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  type: ExternalName
spec:
  externalName: example.com
```



Ingress – Name Based Routing

- An API object that manages external access to the services in a cluster
- Provides load balancing, SSL termination and name/path-based virtual hosting
- Gives services externally-reachable URLs

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: name-virtual-host-ingress
spec:
  rules:
  - host: first.bar.com
    http:
      paths:
      - backend:
          serviceName: service1
          servicePort: 80
  - host: second.foo.com
    http:
      paths:
      - backend:
          serviceName: service2
          servicePort: 80
  - http:
      paths:
      - backend:
          serviceName: service3
          servicePort: 80
```



Red Hat



Ingress – Path Based Routing

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: simple-fanout-example
spec:
  rules:
  - host: foo.bar.com
    http:
      paths:
      - path: /foo
        backend:
          serviceName: service1
          servicePort: 4200
      - path: /bar
        backend:
          serviceName: service2
          servicePort: 8080
```



Red Hat



Core workloads

- **ReplicaSet**
- **Deployment**
- **DaemonSet**
- **StatefulSet**
- **Job**
- **CronJob**

Concepts and resources



Red Hat



ReplicaSet

- Primary method of managing pod replicas and their lifecycle.
- Includes their scheduling, scaling, and deletion.
- Their job is simple: **Always ensure the desired number of pods are running.**



Red Hat



ReplicaSet

- **replicas**: The desired number of instances of the Pod.
- **selector**: The label selector for the **ReplicaSet** will manage **ALL** Pod instances that it targets; whether it's desired or not.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-example
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  template:
    <pod template>
```



Red Hat



ReplicaSet

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-example
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  template:
    metadata:
      labels:
        app: nginx
        env: prod
    spec:
      containers:
        - name: nginx
          image: nginx:stable-alpine
          ports:
            - containerPort: 80
```

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
rs-example-9l4dt	1/1	Running	0	1h
rs-example-b7bcg	1/1	Running	0	1h
rs-example-mk1l2	1/1	Running	0	1h

```
$ kubectl describe rs rs-example
```

```
Name:          rs-example
Namespace:     default
Selector:      app=nginx,env=prod
Labels:        app=nginx
               env=prod
Annotations:   <none>
Replicas:      3 current / 3 desired
Pods Status:   3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=nginx
          env=prod
  Containers:
    nginx:
      Image:      nginx:stable-alpine
      Port:       80/TCP
      Environment: <none>
      Mounts:      <none>
      Volumes:     <none>
Events:
  Type    Reason            Age   From                    Message
  ----    -
  Normal  SuccessfulCreate  16s   replicaset-controller   Created pod: rs-example-mk1l2
  Normal  SuccessfulCreate  16s   replicaset-controller   Created pod: rs-example-b7bcg
  Normal  SuccessfulCreate  16s   replicaset-controller   Created pod: rs-example-9l4dt
```



Red Hat



Deployment

- Way of managing Pods via **ReplicaSets**.
- Provide rollback functionality and update control.
- Updates are managed through the **pod-template-hash** label.
- Each iteration creates a unique label that is assigned to both the **ReplicaSet** and subsequent Pods.



Red Hat

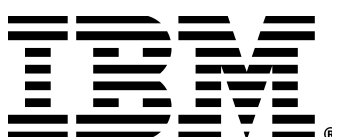


Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.7.9
          ports:
            - containerPort: 80
```



Red Hat



Deployment

- **revisionHistoryLimit**: The number of previous iterations of the Deployment to retain.
- **strategy**: Describes the method of updating the Pods based on the **type**. Valid options are **Recreate** or **RollingUpdate**.
 - **Recreate**: All existing Pods are killed before the new ones are created.
 - **RollingUpdate**: Cycles through updating the Pods according to the parameters: **maxSurge** and **maxUnavailable**.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    <pod template>
```



Red Hat



Deployment

```
$ kubectl create deployment test --image=nginx
$ kubectl set image deployment test nginx=nginx:1.9.1 --record

$ kubectl rollout history deployment test
deployments "test"
REVISION  CHANGE-CAUSE
1          <none>
2          kubectl set image deployment test nginx=nginx:1.9.1 --record=true

$ kubectl annotate deployment test kubernetes.io/change-cause="image updated to 1.9.1"

$ kubectl rollout undo deployment test
$ kubectl rollout undo deployment test --to-revision=2

$ kubectl rollout history deployment test
deployments "test"
REVISION  CHANGE-CAUSE
2          kubectl set image deployment test nginx=nginx:1.9.1 --record=true
3          <none>

kubectl scale deployment test --replicas=10
kubectl rollout pause deployment test
kubectl rollout resume deployment test
```

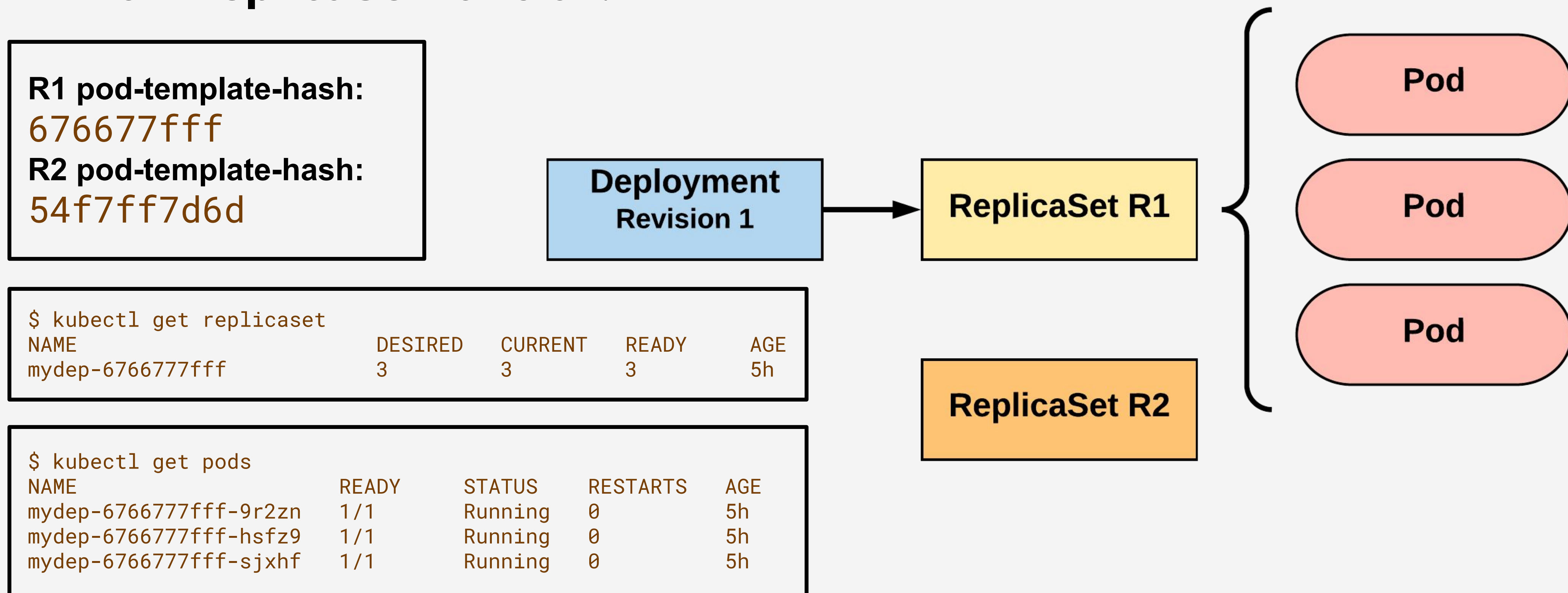


Red Hat



RollingUpdate Deployment

Updating pod template generates a new **ReplicaSet** revision.



Red Hat



RollingUpdate Deployment

New **ReplicaSet** is initially scaled up based on **maxSurge**.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d

Deployment
Revision 2

ReplicaSet R1

Pod

Pod

Pod

ReplicaSet R2

Pod

```
$ kubectl get replicaset
NAME                                DESIRED  CURRENT  READY  AGE
mydep-54f7ff7d6d                    1         1        1      5s
mydep-6766777fff                    2         3        3      5h
```

```
$ kubectl get pods
NAME                                READY    STATUS    RESTARTS  AGE
mydep-54f7ff7d6d-9gv1l             1/1      Running   0          2s
mydep-6766777fff-9r2zn             1/1      Running   0          5h
mydep-6766777fff-hsfz9             1/1      Running   0          5h
mydep-6766777fff-sjxhf             1/1      Running   0          5h
```


RollingUpdate Deployment

Phase out of old Pods managed by
maxSurge and **maxUnavailable**.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d

Deployment
Revision 2

```
$ kubectl get replicaset
NAME                                DESIRED  CURRENT  READY  AGE
mydep-54f7ff7d6d                    2         2        2      8s
mydep-6766777fff                    2         2        2      5h
```

```
$ kubectl get pods
NAME                                READY    STATUS    RESTARTS  AGE
mydep-54f7ff7d6d-9gv1l             1/1     Running   0          5s
mydep-54f7ff7d6d-cqv1q             1/1     Running   0          2s
mydep-6766777fff-9r2zn             1/1     Running   0          5h
mydep-6766777fff-hsfz9             1/1     Running   0          5h
```

ReplicaSet R1

Pod

Pod

~~Pod~~

ReplicaSet R2

Pod

Pod



Red Hat



RollingUpdate Deployment

Phase out of old Pods managed by
maxSurge and **maxUnavailable**.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d

Deployment
Revision 2

```
$ kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
mydep-54f7ff7d6d	3	3	3	10s
mydep-6766777fff	0	1	1	5h

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
mydep-54f7ff7d6d-9gv1l	1/1	Running	0	7s
mydep-54f7ff7d6d-cqvlq	1/1	Running	0	5s
mydep-54f7ff7d6d-gccr6	1/1	Running	0	2s
mydep-6766777fff-9r2zn	1/1	Running	0	5h

ReplicaSet R1

Pod

~~Pod~~

~~Pod~~

ReplicaSet R2

Pod

Pod

Pod



Red Hat



RollingUpdate Deployment

Phase out of old Pods managed by
maxSurge and **maxUnavailable**.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d

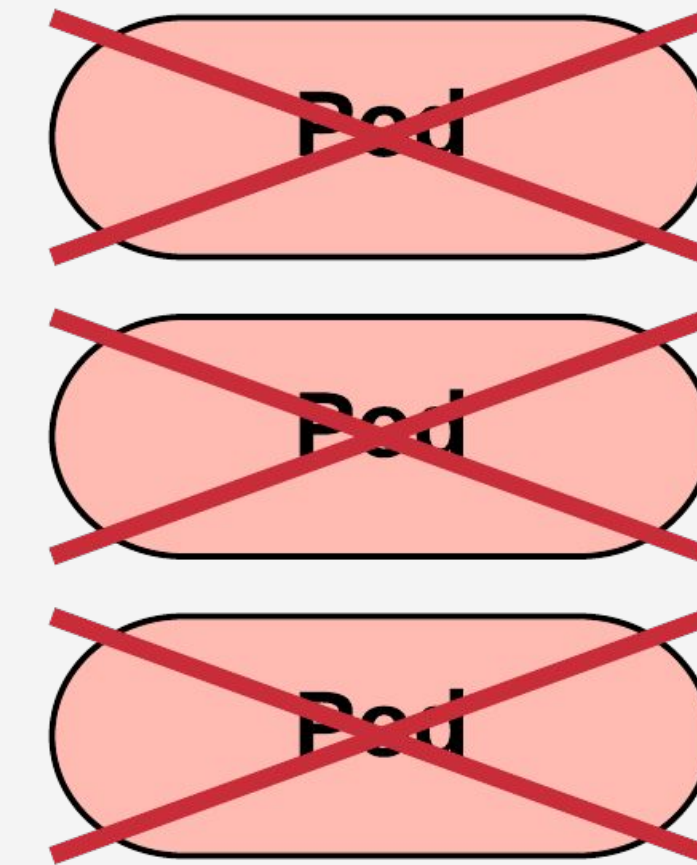
Deployment
Revision 2

```
$ kubectl get replicaset
NAME                               DESIRED  CURRENT  READY  AGE
mydep-54f7ff7d6d                   3        3        3      13s
mydep-6766777fff                   0        0        0      5h
```

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS  AGE
mydep-54f7ff7d6d-9gv1l             1/1    Running   0         10s
mydep-54f7ff7d6d-cqvlq             1/1    Running   0         8s
mydep-54f7ff7d6d-gccr6             1/1    Running   0         5s
```

ReplicaSet R1

ReplicaSet R2

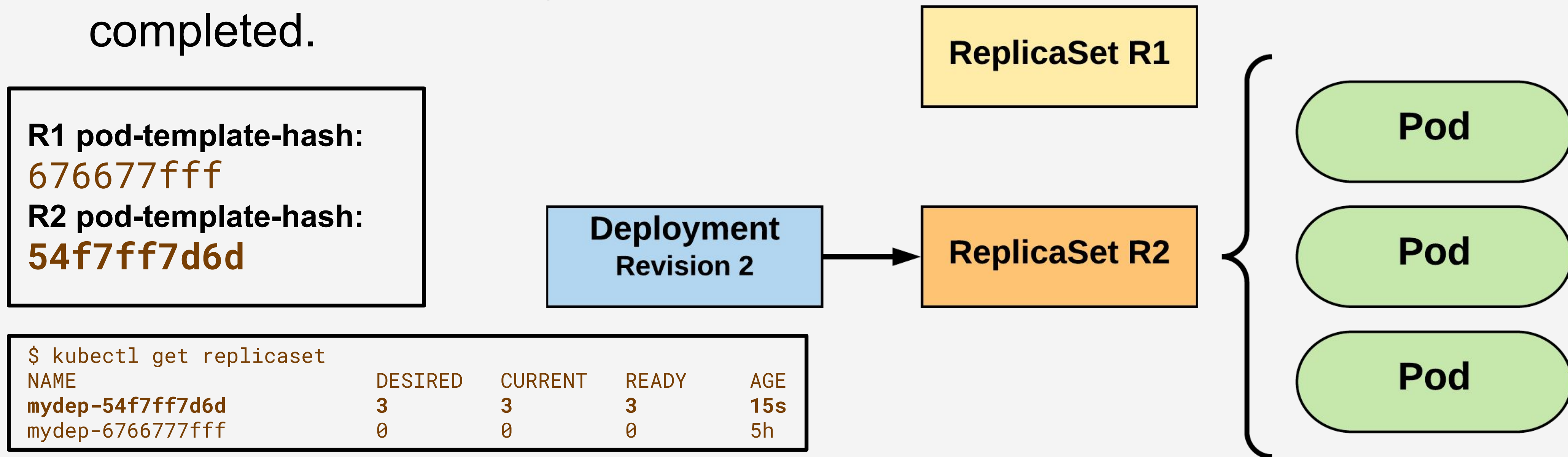


Red Hat



RollingUpdate Deployment

Updated to new deployment revision completed.



```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
mydep-54f7ff7d6d-9gv1l	1/1	Running	0	12s
mydep-54f7ff7d6d-cqvlq	1/1	Running	0	10s
mydep-54f7ff7d6d-gccr6	1/1	Running	0	7s



Red Hat



Taints and Tolerations

```
$ kubectl taint nodes node1 key=value:NoSchedule
```

```
tolerations:  
- key: "key"  
  operator: "Equal"  
  value: "value"  
  effect: "NoSchedule"
```

```
tolerations:  
- operator: "Exists"
```

```
tolerations:  
- key: "key"  
  operator: "Exists"
```

```
tolerations:  
- key: "key1"  
  operator: "Equal"  
  value: "value1"  
  effect: "NoExecute"  
  tolerationSeconds: 3600
```

```
$ kubectl taint nodes node1 gpu=nvidia:NoSchedule
```

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx  
spec:  
  containers:  
  - image: nginx  
    name: nginx  
  tolerations:  
  - key: gpu  
    value: nvidia  
    effect: NoSchedule
```



Red Hat



DaemonSet

- Ensure that all nodes matching certain criteria will run an instance of the supplied Pod.
- Are ideal for cluster wide services such as log forwarding or monitoring.



Red Hat



StatefulSet

- Tailored to managing Pods that must persist or maintain state.
- Pod lifecycle will be ordered and follow consistent patterns.
- Assigned a unique ordinal name following the convention of '*<statefulset name>-<ordinal index>*'.



Red Hat



StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-example
spec:
  replicas: 2
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: stateful
  serviceName: app
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 0
  template:
    metadata:
      labels:
        app: stateful
    <continued>
```

```
<continued>
spec:
  containers:
    - name: nginx
      image: nginx:stable-alpine
      ports:
        - containerPort: 80
      volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
    - metadata:
        name: www
      spec:
        accessModes: [ "ReadWriteOnce" ]
        storageClassName: standard
        resources:
          requests:
            storage: 1Gi
```



Red Hat



StatefulSet

- **revisionHistoryLimit:** The number of previous iterations of the StatefulSet to retain.
- **serviceName:** The name of the associated headless service; or a service without a **ClusterIP**.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-example
spec:
  replicas: 2
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: stateful
  serviceName: app
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 0
  template:
    <pod template>
```



Red Hat



Headless Service

<StatefulSet Name>-<ordinal>.<service name>.<namespace>.svc.cluster.local

```
apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  clusterIP: None
  selector:
    app: stateful
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

```
/ # dig app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> app.default.svc.cluster.local +noall +answer
;; global options: +cmd
app.default.svc.cluster.local. 2 IN A 10.255.0.5
app.default.svc.cluster.local. 2 IN A 10.255.0.2
```

```
/ # dig sts-example-0.app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> sts-example-0.app.default.svc.cluster.local +noall +answer
;; global options: +cmd
sts-example-0.app.default.svc.cluster.local. 20 IN A 10.255.0.2
```

```
$ kubectl get pods
NAME           READY   STATUS    RESTARTS   AGE
sts-example-0   1/1     Running   0           11m
sts-example-1   1/1     Running   0           11m
```

```
/ # dig sts-example-1.app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> sts-example-1.app.default.svc.cluster.local +noall +answer
;; global options: +cmd
sts-example-1.app.default.svc.cluster.local. 30 IN A 10.255.0.5
```



Red Hat



Headless Service

<StatefulSet Name>-<ordinal>.<service name>.<namespace>.svc.cluster.local

```
apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  clusterIP: None
  selector:
    app: stateful
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

```
/ # dig app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> app.default.svc.cluster.local +noall +answer
;; global options: +cmd
app.default.svc.cluster.local. 2 IN A 10.255.0.5
app.default.svc.cluster.local. 2 IN A 10.255.0.2
```

```
/ # dig sts-example-0.app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> sts-example-0.app.default.svc.cluster.local +noall +answer
;; global options: +cmd
sts-example-0.app.default.svc.cluster.local. 20 IN A 10.255.0.2
```

```
$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
sts-example-0 1/1     Running   0           11m
sts-example-1 1/1     Running   0           11m
```

```
/ # dig sts-example-1.app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> sts-example-1.app.default.svc.cluster.local +noall +answer
;; global options: +cmd
sts-example-1.app.default.svc.cluster.local. 30 IN A 10.255.0.5
```

Headless Service

<StatefulSet Name>-<ordinal>.<service name>.<namespace>.svc.cluster.local

```
apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  clusterIP: None
  selector:
    app: stateful
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

```
/ # dig app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> app.default.svc.cluster.local +noall +answer
;; global options: +cmd
app.default.svc.cluster.local. 2 IN A 10.255.0.5
app.default.svc.cluster.local. 2 IN A 10.255.0.2
```

```
/ # dig sts-example-0.app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> sts-example-0.app.default.svc.cluster.local +noall +answer
;; global options: +cmd
sts-example-0.app.default.svc.cluster.local. 20 IN A 10.255.0.2
```

```
$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
sts-example-0 1/1     Running   0           11m
sts-example-1 1/1     Running   0           11m
```

```
/ # dig sts-example-1.app.default.svc.cluster.local +noall +answer

; <<>> DiG 9.11.2-P1 <<>> sts-example-1.app.default.svc.cluster.local +noall +answer
;; global options: +cmd
sts-example-1.app.default.svc.cluster.local. 30 IN A 10.255.0.5
```


CronJob

An extension of the Job Controller, it provides a method of executing jobs on a cron-like schedule.

CronJobs within Kubernetes
use **UTC ONLY**.



Red Hat



CronJob

- **schedule:** The cron schedule for the job.
- **successfulJobHistoryLimit:** The number of successful jobs to retain.
- **failedJobHistoryLimit:** The number of failed jobs to retain.

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: cronjob-example
spec:
  schedule: "*/1 * * * *"
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      completions: 4
      parallelism: 2
      template:
        <pod template>
```


CronJob

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: cronjob-example
spec:
  schedule: "*/1 * * * *"
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      completions: 4
      parallelism: 2
      template:
        spec:
          containers:
            - name: hello
              image: alpine:latest
              command: ["/bin/sh", "-c"]
              args: ["echo hello from $HOSTNAME!"]
              restartPolicy: Never
```

```
$ kubectl get jobs
```

NAME	DESIRED	SUCCESSFUL	AGE
cronjob-example-1519053240	4	4	2m
cronjob-example-1519053300	4	4	1m
cronjob-example-1519053360	4	4	26s

```
$ kubectl describe cronjob cronjob-example
Name:                cronjob-example
Namespace:            default
Labels:               <none>
Annotations:          <none>
Schedule:             */1 * * * *
Concurrency Policy:   Allow
Suspend:              False
Starting Deadline Seconds: <unset>
Selector:             <unset>
Parallelism:          2
Completions:          4
Pod Template:
  Labels:  <none>
  Containers:
    hello:
      Image:  alpine:latest
      Port:   <none>
      Command:
        /bin/sh
        -c
      Args:
        echo hello from $HOSTNAME!
      Environment:  <none>
      Mounts:       <none>
      Volumes:      <none>
  Last Schedule Time: Mon, 19 Feb 2018 09:54:00 -0500
  Active Jobs:       cronjob-example-1519052040
Events:
  Type      Reason            Age   From                  Message
  ----      -
  Normal    SuccessfulCreate   3m    cronjob-controller    Created job cronjob-example-1519051860
  Normal    SawCompletedJob    2m    cronjob-controller    Saw completed job: cronjob-example-1519051860
  Normal    SuccessfulCreate   2m    cronjob-controller    Created job cronjob-example-1519051920
  Normal    SawCompletedJob    1m    cronjob-controller    Saw completed job: cronjob-example-1519051920
  Normal    SuccessfulCreate   1m    cronjob-controller    Created job cronjob-example-1519051980
```



Red Hat



Storage

- **Volumes**
- **Persistent Volumes**
- **Persistent Volume Claims**
- **StorageClass**

Concepts and resources



Red Hat



Storage

Pods by themselves are useful, but many workloads require exchanging data between containers, or persisting some form of data.

For this we have **Volumes**, **PersistentVolumes**, **PersistentVolumeClaims**, and **StorageClasses**.



Red Hat



StorageClass

- Storage classes are an abstraction on top of an external storage resource (PV)
- Work hand-in-hand with the external storage system to enable **dynamic provisioning** of storage by eliminating the need for the cluster admin to pre-provision a PV

StorageClass

- **provisioner**: Defines the ‘*driver*’ to be used for provisioning of the external storage.
- **parameters**: A hash of the various configuration parameters for the provisioner.
- **reclaimPolicy**: The behaviour for the backing storage when the PVC is deleted.
 - **Retain** - manual clean-up
 - **Delete** - storage asset deleted by provider

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: standard
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
  zones: us-central1-a, us-central1-b
reclaimPolicy: Delete
```



Red Hat



Available StorageClasses

- AWSElasticBlockStore
- AzureFile
- AzureDisk
- CephFS
- Cinder
- FC
- Flocker
- GCEPersistentDisk
- Glusterfs
- iSCSI
- Quobyte
- NFS
- RBD
- VsphereVolume
- PortworxVolume
- ScaleIO
- StorageOS
- Local

 Internal Provisioner

Volumes

- Storage that is tied to the **Pod's Lifecycle**.
- A pod can have one or more types of volumes attached to it.
- Can be consumed by any of the containers within the pod.
- Survive Pod restarts; however their durability beyond that is dependent on the Volume Type.

Volume Types

- awsElasticBlockStore
- azureDisk
- azureFile
- cephfs
- configMap
- csi
- downwardAPI
- emptyDir
- fc (fibre channel)
- flocker
- gcePersistentDisk
- gitRepo
- glusterfs
- hostPath
- iscsi
- local
- nfs
- persistentVolume Claim
- projected
- portworxVolume
- quobyte
- rbd
- scaleIO
- secret
- storageos
- vsphereVolume



Persistent Volume Supported



Red Hat



Volumes

- **volumes:** A list of volume objects to be attached to the Pod. Every object within the list must have its own unique **name**.
- **volumeMounts:** A container specific list referencing the Pod volumes by **name**, along with their desired **mountPath**.

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-example
spec:
  containers:
    - name: nginx
      image: nginx:stable-alpine
      volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
          ReadOnly: true
    - name: content
      image: alpine:latest
      command: ["/bin/sh", "-c"]
      args:
        - while true; do
            date >> /html/index.html;
            sleep 5;
          done
      volumeMounts:
        - name: html
          mountPath: /html
  volumes:
    - name: html
      emptyDir: {}
```



Red Hat



Volumes

- **volumes:** A list of volume objects to be attached to the Pod. Every object within the list must have its own unique **name**.
- **volumeMounts:** A container specific list referencing the Pod volumes by **name**, along with their desired **mountPath**.

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    volumeMounts:
    - name: html
      mountPath: /usr/share/nginx/html
      ReadOnly: true
  - name: content
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args:
    - while true; do
      date >> /html/index.html;
      sleep 5;
    done
    volumeMounts:
    - name: html
      mountPath: /html
  volumes:
  - name: html
    emptyDir: {}
```



Red Hat



Volumes

- **volumes:** A list of volume objects to be attached to the Pod. Every object within the list must have its own unique **name**.
- **volumeMounts:** A container specific list referencing the Pod volumes by **name**, along with their desired **mountPath**.

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    volumeMounts:
    - name: html
      mountPath: /usr/share/nginx/html
      ReadOnly: true
  - name: content
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args:
    - while true; do
      date >> /html/index.html;
      sleep 5;
    done
    volumeMounts:
    - name: html
      mountPath: /html
  volumes:
  - name: html
    emptyDir: {}
```



Red Hat



Persistent Volumes

- A **PersistentVolume** (PV) represents a storage resource.
- PVs are a **cluster wide resource** linked to a backing storage provider: NFS, GCEPersistentDisk, RBD etc.
- Generally provisioned by an administrator.
- Their lifecycle is handled independently from a pod
- **CANNOT** be attached to a Pod directly. Relies on a **PersistentVolumeClaim**



Red Hat



PersistentVolumeClaims

- A **PersistentVolumeClaim** (PVC) is a **namespaced** request for storage.
- Satisfies a set of requirements instead of mapping to a storage resource directly.
- Ensures that an application's '*claim*' for storage is portable across numerous backends or providers.



Red Hat



PersistentVolume

- `capacity.storage`: The total amount of available storage.
- `volumeMode`: The type of volume, this can be either `Filesystem` or `Block`.
- `accessModes`: A list of the supported methods of accessing the volume.
Options include:
 - `ReadWriteOnce`
 - `ReadOnlyMany`
 - `ReadWriteMany`

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfsserver
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Delete
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /exports
    server: 172.22.0.42
```



Red Hat



PersistentVolume

- **persistentVolumeReclaimPolicy**: The behaviour for PVC's that have been deleted. Options include:
 - **Retain** - manual clean-up
 - **Delete** - storage asset deleted by provider.
- **storageClassName**: Optional name of the storage class that PVC's can reference. If provided, **ONLY** PVC's referencing the name consume use it.
- **mountOptions**: Optional mount options for the PV.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfsserver
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Delete
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /exports
    server: 172.22.0.42
```



Red Hat



PersistentVolumeClaim

- **accessModes**: The selected method of accessing the storage. This **MUST** be a subset of what is defined on the target PV or Storage Class.
 - **ReadWriteOnce**
 - **ReadOnlyMany**
 - **ReadWriteMany**
- **resources.requests.storage**: The desired amount of storage for the claim
- **storageClassName**: The name of the desired Storage Class

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-sc-example
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: slow
```



Red Hat



PVs and PVCs with Selectors

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: pv-selector-example
  labels:
    type: hostpath
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/mnt/data"
```

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-selector-example
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  selector:
    matchLabels:
      type: hostpath
```



Red Hat



PVs and PVCs with Selectors

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: pv-selector-example
  labels:
    type: hostpath
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/mnt/data"
```

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-selector-example
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  selector:
    matchLabels:
      type: hostpath
```



Red Hat



PV Phases

Available

PV is ready and available to be consumed.

Bound

The PV has been bound to a claim.

Released

The binding PVC has been deleted, and the PV is pending reclamation.

Failed

An error has been encountered.



Red Hat



StorageClass

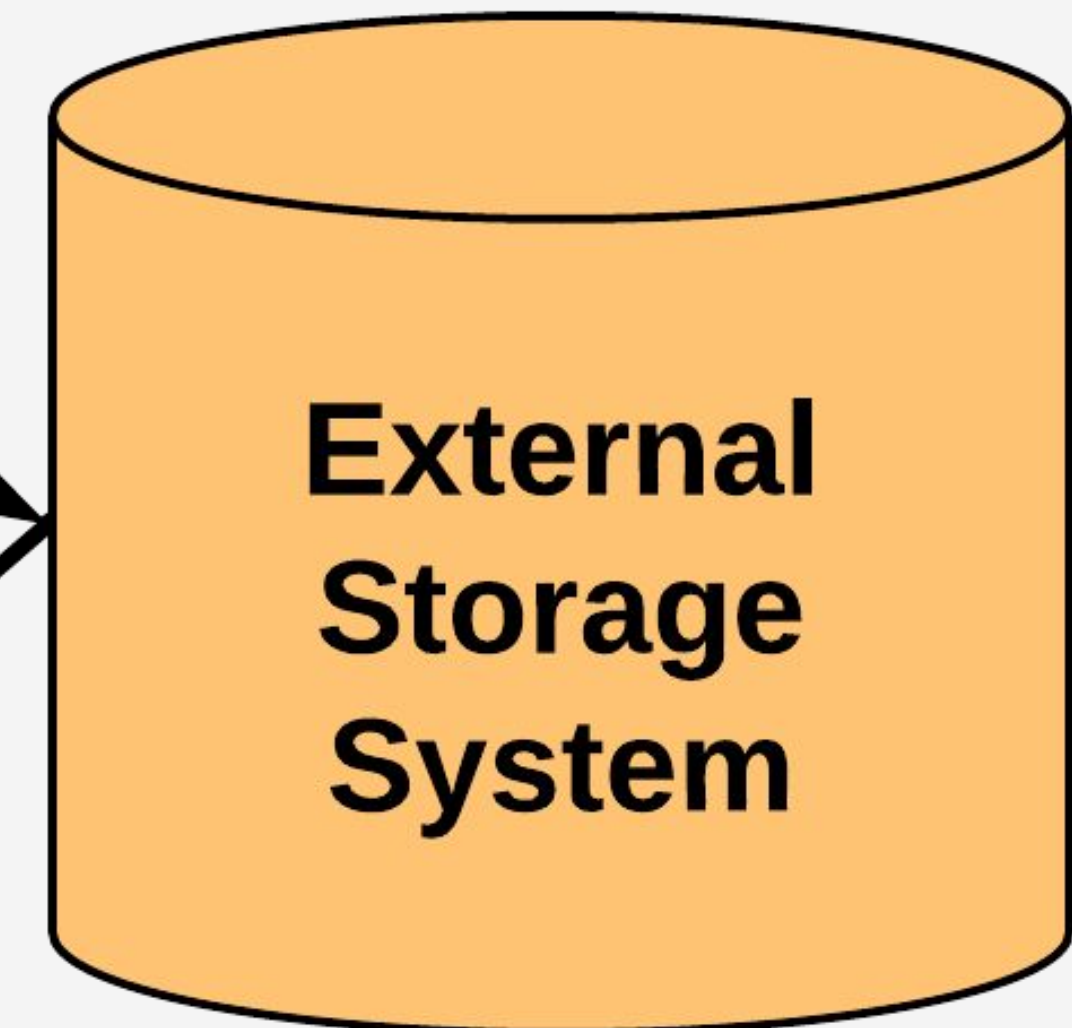
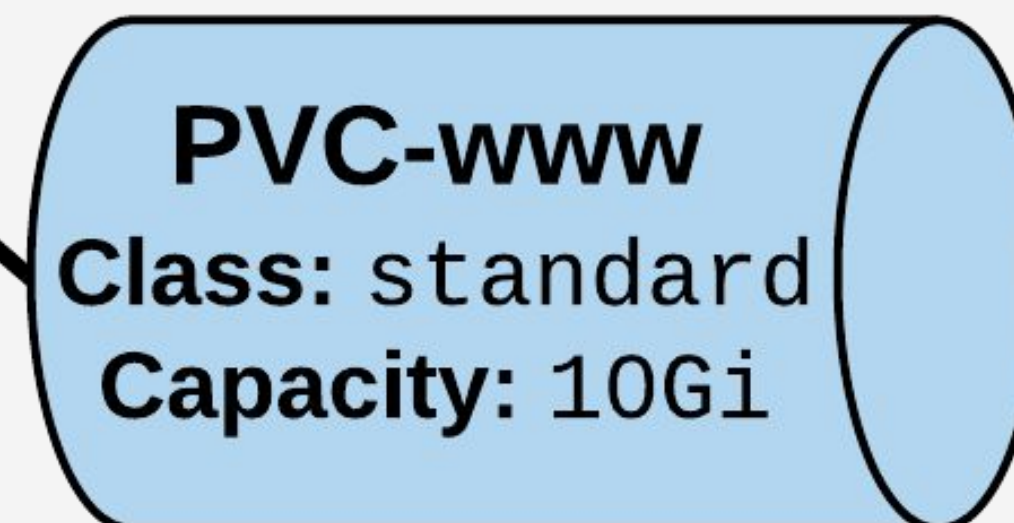
1. PVC makes a request of the StorageClass.

2. StorageClass provisions request through API with external storage system.

3. External storage system creates a PV strictly satisfying the PVC request.

4. provisioned PV is bound to requesting PVC.

uid: 9df65c6e-1a69-11e8-ae10-080027a3682b



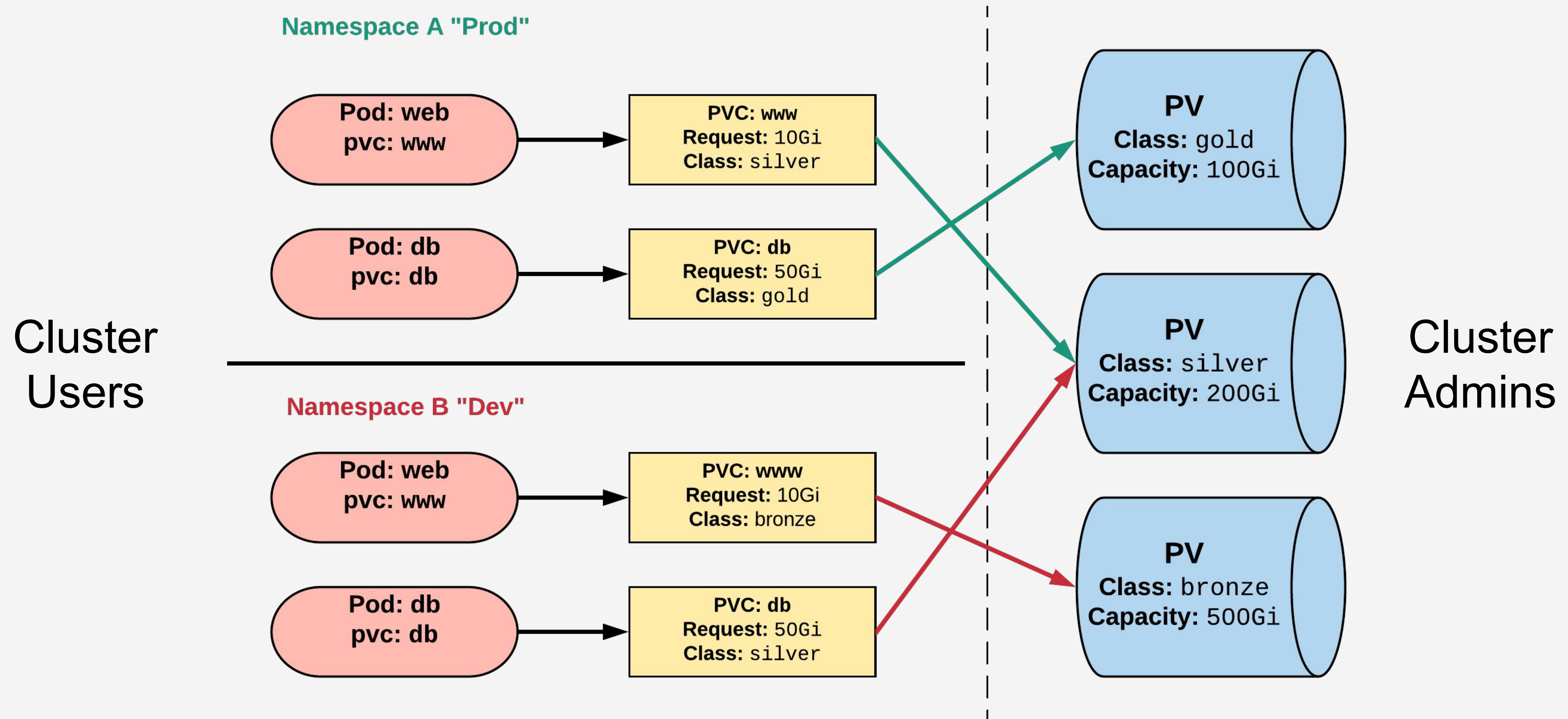
pv: pvc-9df65c6e-1a69-11e8-ae10-080027a3682b



Red Hat



Persistent Volumes and Claims



Red Hat



Working with Volumes

Concepts and resources



Red Hat



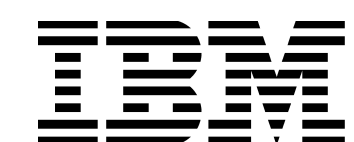
Configurations

- **ConfigMap**
- **Secret**

Concepts and resources



Red Hat



Configuration

Kubernetes has an integrated pattern for decoupling configuration from application or container.

This pattern makes use of two Kubernetes components: **ConfigMaps** and **Secrets**.



Red Hat



ConfigMap

- Externalized data stored within kubernetes.
- Can be referenced through several different means:
 - environment variable
 - a command line argument (via env var)
 - injected as a file into a volume mount
- Can be created from a manifest, literals, directories, or files directly.

ConfigMap

data: Contains key-value pairs of ConfigMap contents.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  state: Michigan
  city: Ann Arbor
  content: |
    Look at this,
    its multiline!
```



Red Hat



ConfigMap Example

All produce a **ConfigMap** with the same content!

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  city: Ann Arbor
  state: Michigan
```

```
$ kubectl create configmap literal-example \
> --from-literal="city=Ann Arbor" --from-literal=state=Michigan
configmap "literal-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap dir-example --from-file=cm/
configmap "dir-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap file-example --from-file=cm/city --from-file=cm/state
configmap "file-example" created
```



Red Hat



ConfigMap Example

All produce a **ConfigMap** with the same content!

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  city: Ann Arbor
  state: Michigan
```

```
$ kubectl create configmap literal-example \
> --from-literal="city=Ann Arbor" --from-literal=state=Michigan
configmap "literal-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap dir-example --from-file=cm/
configmap "dir-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap file-example --from-file=cm/city --from-file=cm/state
configmap "file-example" created
```



Red Hat



ConfigMap Example

All produce a **ConfigMap** with the same content!

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  city: Ann Arbor
  state: Michigan
```

```
$ kubectl create configmap literal-example \
> --from-literal="city=Ann Arbor" --from-literal=state=Michigan
configmap "literal-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap dir-example --from-file=cm/
configmap "dir-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap file-example --from-file=cm/city --from-file=cm/state
configmap "file-example" created
```



Red Hat



ConfigMap Example

All produce a **ConfigMap** with the same content!

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  city: Ann Arbor
  state: Michigan
```

```
$ kubectl create configmap literal-example \
> --from-literal="city=Ann Arbor" --from-literal=state=Michigan
configmap "literal-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap dir-example --from-file=cm/
configmap "dir-example" created
```

```
$ cat info/city
Ann Arbor
$ cat info/state
Michigan
$ kubectl create configmap file-example --from-file=cm/city --from-file=cm/state
configmap "file-example" created
```



Red Hat



Secret

- Functionally identical to a ConfigMap.
- Stored as **base64 encoded content**.
- Encrypted at rest within etcd (**if configured!**).
- Stored on each worker node in tmpfs directory.
- Ideal for username/passwords, certificates or other sensitive information that should not be stored in a container.

Secret

- **type:** There are three different types of secrets within Kubernetes:
 - **docker-registry** - credentials used to authenticate to a container registry
 - **generic/Opaque** - literal values from different sources
 - **tls** - a certificate based secret
- **data:** Contains key-value pairs of base64 encoded content.

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-secret
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```



Red Hat



Secret Example

All produce a **Secret** with the same content!

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-example
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

```
$ kubectl create secret generic literal-secret \
> --from-literal=username=example \
> --from-literal=password=mypassword
secret "literal-secret" created
```

```
$ cat info/username
example
$ cat info/password
mypassword
$ kubectl create secret generic dir-secret --from-file=secret/
Secret "file-secret" created
```

```
$ cat secret/username
example
$ cat secret/password
mypassword
$ kubectl create secret generic file-secret --from-file=secret/username --from-file=secret/password
Secret "file-secret" created
```



Red Hat



Secret Example

All produce a **Secret** with the same content!

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-example
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

```
$ kubectl create secret generic literal-secret \
> --from-literal=username=example \
> --from-literal=password=mypassword
secret "literal-secret" created
```

```
$ cat info/username
example
$ cat info/password
mypassword
$ kubectl create secret generic dir-secret --from-file=secret/
Secret "file-secret" created
```

```
$ cat secret/username
example
$ cat secret/password
mypassword
$ kubectl create secret generic file-secret --from-file=secret/username --from-file=secret/password
Secret "file-secret" created
```

Secret Example

All produce a **Secret** with the same content!

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-example
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

```
$ kubectl create secret generic literal-secret \
> --from-literal=username=example \
> --from-literal=password=mypassword
secret "literal-secret" created
```

```
$ cat info/username
example
$ cat info/password
mypassword
$ kubectl create secret generic dir-secret --from-file=secret/
Secret "file-secret" created
```

```
$ cat secret/username
example
$ cat secret/password
mypassword
$ kubectl create secret generic file-secret --from-file=secret/username --from-file=secret/password
Secret "file-secret" created
```


Secret Example

All produce a **Secret** with the same content!

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-example
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

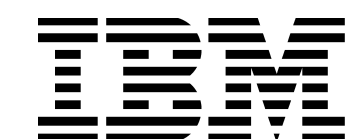
```
$ kubectl create secret generic literal-secret \
> --from-literal=username=example \
> --from-literal=password=mypassword
secret "literal-secret" created
```

```
$ cat info/username
example
$ cat info/password
mypassword
$ kubectl create secret generic dir-secret --from-file=secret/
Secret "file-secret" created
```

```
$ cat secret/username
example
$ cat secret/password
mypassword
$ kubectl create secret generic file-secret --from-file=secret/username --from-file=secret/password
Secret "file-secret" created
```



Red Hat



Injecting as Environment Variable

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cm-env-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["printenv CITY"]
        env:
        - name: CITY
          valueFrom:
            configMapKeyRef:
              name: manifest-example
              key: city
      restartPolicy: Never
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: secret-env-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["printenv USERNAME"]
        env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: manifest-example
              key: username
      restartPolicy: Never
```



Red Hat



Injecting as Environment Variable

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cm-env-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["printenv CITY"]
        env:
        - name: CITY
          valueFrom:
            configMapKeyRef:
              name: manifest-example
              key: city
        restartPolicy: Never
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: secret-env-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["printenv USERNAME"]
        env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: manifest-example
              key: username
        restartPolicy: Never
```



Red Hat



Injecting in a Command

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cm-cmd-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["echo Hello ${CITY}!"]
        env:
        - name: CITY
          valueFrom:
            configMapKeyRef:
              name: manifest-example
              key: city
      restartPolicy: Never
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: secret-cmd-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["echo Hello ${USERNAME}!"]
        env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: manifest-example
              key: username
      restartPolicy: Never
```



Red Hat



Injecting in a Command

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cm-cmd-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["echo Hello ${CITY}!"]
        env:
        - name: CITY
          valueFrom:
            configMapKeyRef:
              name: manifest-example
              key: city
      restartPolicy: Never
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: secret-cmd-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["echo Hello ${USERNAME}!"]
        env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: manifest-example
              key: username
      restartPolicy: Never
```



Red Hat



Injecting as a Volume

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cm-vol-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["cat /myconfig/city"]
        volumeMounts:
        - name: config-volume
          mountPath: /myconfig
      restartPolicy: Never
      volumes:
      - name: config-volume
        configMap:
          name: manifest-example
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: secret-vol-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["cat /mysecret/username"]
        volumeMounts:
        - name: secret-volume
          mountPath: /mysecret
      restartPolicy: Never
      volumes:
      - name: secret-volume
        secret:
          secretName: manifest-example
```



Red Hat



Injecting as a Volume

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cm-vol-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["cat /myconfig/city"]
        volumeMounts:
        - name: config-volume
          mountPath: /myconfig
      restartPolicy: Never
      volumes:
      - name: config-volume
        configMap:
          name: manifest-example
```

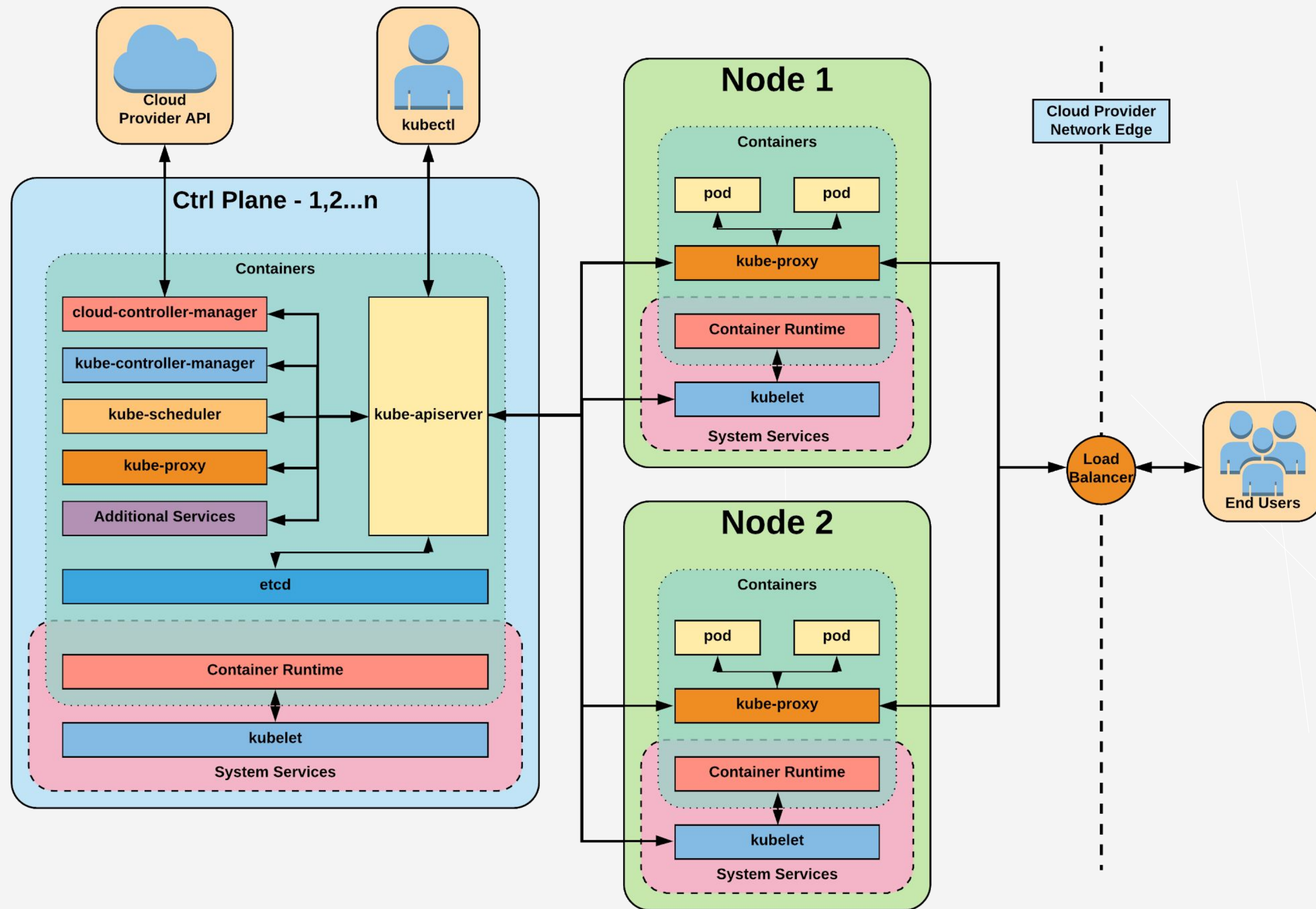
```
apiVersion: batch/v1
kind: Job
metadata:
  name: secret-vol-example
spec:
  template:
    spec:
      containers:
      - name: mypod
        image: alpine:latest
        command: ["/bin/sh", "-c"]
        args: ["cat /mysecret/username"]
        volumeMounts:
        - name: secret-volume
          mountPath: /mysecret
      restartPolicy: Never
      volumes:
      - name: secret-volume
        secret:
          secretName: manifest-example
```



Red Hat



Summary



Questions?

Adapted from slides by Joe Beda (Gluecon 2017).

Licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

Changes were made.